

《计算机组成原理》

（第四讲）

厦门大学信息学院软件工程系 张海英

2026年4月

目录

- 第1章 计算机系统概论
- 第2章 数据信息的表示
- 第3章 运算方法与运算器
- 第4章 存储系统
- 第5章 指令系统
- 第6章 中央处理器
- 第7章 指令流水线
- 第8章 总线系统
- 第9章 输入输出系统



第4章 存储系统

- 4.1 存储器概述
- 4.2 半导体存储器
- 4.3 主存的组织及与CPU 的连接
- 4.4 并行主存系统
- 4.5 高速缓冲存储器
- 4.6 虚拟存储器

4.1 存储器概述

• 4.1.1 存储器分类

– 1、按存储介质分类

- (1) 磁存储器
 - 硬盘、磁带，早期的软盘、磁芯（属于内存）
- (2) 半导体存储器
 - ① 双极型存储器（TTL型和ECL型）
 - ② 金属氧化物半导体存储器（MOS存储器）：静态MOS存储器（SRAM）、动态MOS存储器（DRAM）；如内存条就属于DRAM
- (3) 光存储器
 - 光盘：CD-ROM、DVD-ROM

TTL: Transistor Transistor Logic, 晶体管-晶体管逻辑

ECL: Emitter Coupled Logic, 发射极耦合逻辑

MOS: Metal-Oxide-Semiconductor Field-Effect Transistor, 金属-氧化物半导体场效应晶体管

SRAM: Static Random-Access Memory, 静态随机存取存储器

DRAM: Dynamic Random Access Memory, 动态随机存取存储器

内存最新技术发展

1. HBM4 / HBM4E: AI训练的核心引擎

高带宽内存（HBM）是AI加速卡（如NVIDIA B300、GB300）的标配。相比HBM3E，**HBM4**将接口位宽从1024位翻倍至2048位，并通过更先进的堆叠技术实现更高密度。

- **性能**：当前HBM4带宽已达**2.8 TB/s**（如美光），功耗效率提升40%以上；三星已宣布量产，SK海力士也于2026年3月推出新品。
- **演进**：**HBM4E**预计将带宽推至**4.0 TB/s**，采用16层堆叠，单堆栈容量可达48GB或更高，主要用于2026年底至2027年的旗舰AI芯片。
- **意义**：直接决定AI训练集群的吞吐能力，是破除“内存墙”的第一道关口。

HBM: high bandwidth memory 高带宽内存，专为高性能计算设计的**3D**堆叠高端内存
目前AI训练卡的标配

2. LPDDR6 与 1c 工艺: 端侧AI的能效革命

为了支撑PC、手机等终端的本地AI（如AI PC、AI手机），低功耗内存加速迭代。

- **LPDDR6**：相比LPDDR5X，速度从8.5Gbps提升至**10.7Gbps以上**，带宽提升33%的同时，通过电压调节技术降低功耗超20%。SK海力士于2026年3月率先发布基于**1c工艺**的16Gb LPDDR6，能耗比领先。
- **国产进展**：长鑫存储也已送样LPDDR6产品，满足国内端侧AI芯片的配套需求。
- **应用**：2026年下半年至2027年的旗舰移动设备将成为主要落地场景。

LPDDR: low power double data rate 低功耗双倍数据率
主要针对移动设备和轻薄型电子产品

内存最新技术发展

3. CXL 内存池化：打破物理边界

Compute Express Link (CXL) 技术允许将内存从服务器内部解耦，形成跨CPU/GPU共享的“内存池”，大幅提升利用率。

- **技术：**CXL 3.0 交换机（如Marvell Structera S系列）可连接多个主机，动态分配内存资源。宜鼎等厂商已推出CXL内存扩展卡，单卡通过PCIe插槽提供**256GB**容量。
- **价值：**传统服务器内存利用率常低于50%，池化后可提升至80%以上，且允许在AI推理中灵活分配容量，降低总体拥有成本（TCO）。
- **生态：**Linux内核及主流AI框架已开始原生支持CXL内存分层管理。

4. GDDR 垂直堆叠：推理侧的性价比之选

美光在GTC 2026上提出了一条介于GDDR和HBM之间的新路线——通过垂直堆叠GDDR芯片，在提供远高于单颗GDDR带宽的同时，成本远低于HBM。

- **方案：**利用成熟的GDDR7基础，通过3D堆叠实现更高带宽密度，满足AI推理卡（尤其是边缘端）的需求。
- **进度：**目前处于概念和样品开发阶段，预计**2027年发布样品**。若成功，将显著降低AI推理硬件的内存成本。
- **GDDR：**图形双倍数据率（**Graphics double data rate**）专门为显卡**GPU**设计的显存。追求高位宽，支持高分辨率纹理渲染。
- **DDR：**注重低延迟和低功耗，更侧重快速响应**CPU**的指令。

内存最新技术发展

5. 国产封装突破：HBM3e 2.5D 封装

在先进存储封装领域，长电科技等国内厂商已实现关键突破，为国产AI芯片提供自主供应链。

- **技术：**长电展出的**HBM3e 2.5D**封装方案，通过硅中介层实现逻辑芯片与HBM堆栈的集成，带宽可达**960GB/s**，适配3nm以下AI芯片。
- **意义：**此前2.5D封装多由台积电（CoWoS）主导，国产方案的突破为国内AI芯片（如华为昇腾、寒武纪等）提供了HBM集成的可行性路径，缓解了先进封装环节的瓶颈。

4.1.1 存储器分类

– 2、按存取方式分类

- (1) 随机存储器: **RAM** (Random Access Memory)
 - 可按照地址随机读写数据存储单元, 且存取访问时间与存储单元的位置无关, 如**SRAM**、**DRAM**
- (2) 顺序存储器: **SAM** (Sequential Access Memory)
 - 存储单元中的内容只能依地址顺序访问, 且访问的速度与存储单元的位置有关, 如**磁带**
- (3) 直接存储器: **DAM** (Direct Access Memory)
 - 不必经过顺序搜索就能在存储器中直接存取信息, 如**磁盘**: 磁道随机访问, 扇区顺序访问
 - 硬盘的读写原理详解 - 知乎 <https://zhuanlan.zhihu.com/p/379550433>

– 3、按信息的可改写性分类

- (1) 读写存储器: 如**RAM** (Random Access Memory)
- (2) 只读存储器: **ROM** (Read Only Memory)

– 4、按信息的可保存性分类

- (1) 易失性存储器: 断电后存储器中的信息会丢失, 如**RAM**
- (2) 非易失性存储器: 断电后存储器中的信息不会丢失, 如**ROM**、**硬盘**

4.1.1 存储器分类

– 5、按功能和存取速度分类

- (1) 寄存器存储器
 - 由多个寄存器（在CPU中）组成的存储器
- (2) 高速缓存存储器
 - 高速缓存`cache`，位于寄存器（在CPU中）和主存之间的一个高速小容量存储器，一般采用SRAM构成，用于缓冲CPU与慢速主存之间的性能差异
- (3) 主存储器
 - 简称`主存`，一般由半导体存储器构成；如内存条就属于主存，采用`DRAM`存储器
 - 主存还包括BIOS（Basic Input Output System）、硬件端口等
- (4) 外存储器
 - 简称外存或辅助存储器
 - 包括：硬盘、磁带、光盘存储器、磁盘阵列和网络存储系统

4.1.2 存储器技术指标

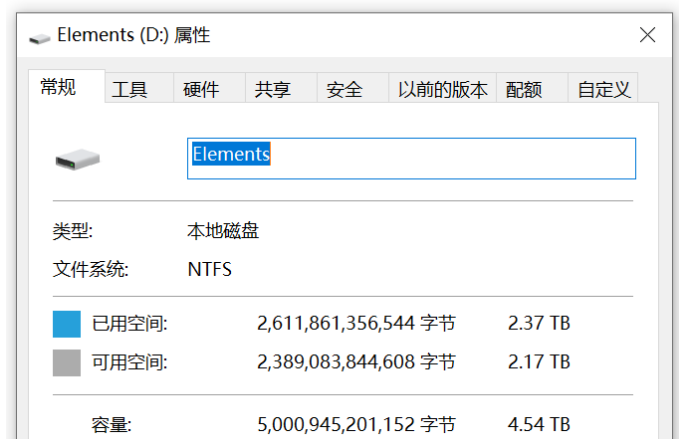
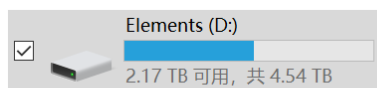
– 1、存储容量

- (1) 位表示法

- 1Kx4位：表示有1K个单元（1024个单元），每个单元存放4位二进制数

- (2) 字节表示法

- 128B：表示有128个单元，每个单元存放1个字节（1B，Byte）



1KiB=1024B=2¹⁰B
1MiB=1024KB=2²⁰B
1GiB=1024MB=2³⁰B
1TiB=1024GB=2⁴⁰B
1PiB=1024TB=2⁵⁰B

1KB=1000B=10³B
1MB=1000KiB=10⁶B
1GB=1000MiB=10⁹B
1TB=1000GiB=10¹²B
1PB=1000TiB=10¹⁵B

5TB移动硬盘

5TB=5x1000x1000x1000x1000
=5x10¹²B/(1024x1024x1024x1024)≈4.54TiB

5,000,945,201,152/(1024x1024x1024x1024)
=4.54TiB

4.1.2 存储器技术指标

– 2、存取速度

- (1) 存取时间

- 也称存储器的访问时间，指启动一次存储器操作（读或写）到该操作完成所经历的时间，读时间和写时间可能不同，例如DRAM读慢写快，闪存（U盘）读快写慢

- (2) 存取周期

- 指连续启动两次访问操作之间的最短时间间隔

- 存取周期 > 存取时间

- 存取周期 = 存取时间 + 恢复时间（存储器状态的稳定恢复时间）

- (3) 存储器带宽

- 指单位时间内存储器所能传输的信息量
- 存储器带宽的单位：位/秒，b/s；字节/秒，B/s
- 存储器带宽与存取时间的长短和一次传输的数据位的多少有关
- 存取时间越短、数据位越宽，存储器带宽越高

4.1.3 存储系统层次结构

- 存储系统层次结构利用程序局部性的原理，从系统级角度将**速度**、**容量**、**成本**各异的存储器有机组合在一起，全方位优化存储系统的各项性能指标
- 上层存储器可以为下层存储器做缓冲，将经常使用数据的副本调度到上层，这样CPU只需要访问上层快速的小容量存储器，即可获得大部分数据
- 寄存器：通常在CPU中
- 高速缓冲（cache）：通常采用SRAM，位于CPU与主存之间
- 主存：通常采用DRAM，也称内存条

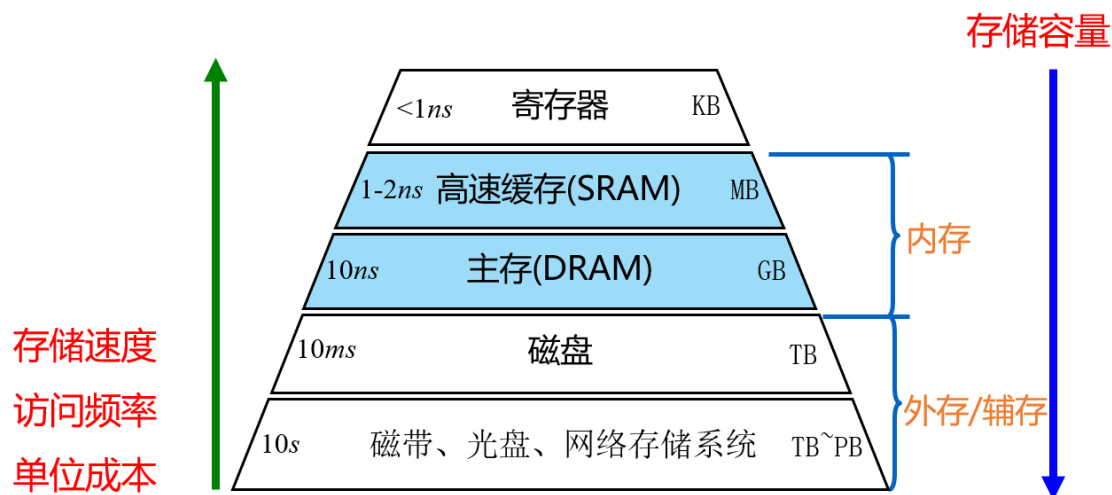
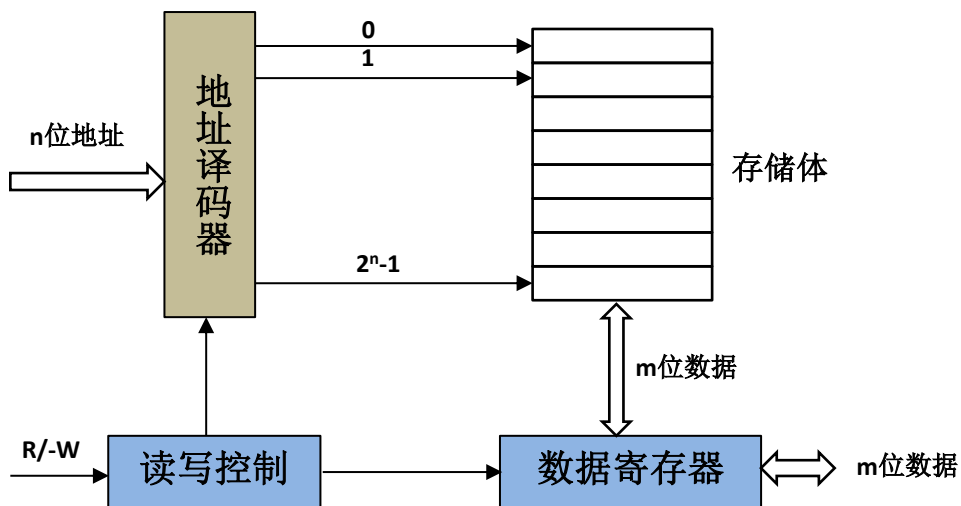


图4.1 存储系统的分级结构

4.1.4 主存的基本结构

- 主存可以看作是一个一维数组：mem[addr]
- 假设主存的地址addr为n位，数组mem[]的宽度为m位；则主存数组有 2^n 个存储单元（地址为： $0 \sim 2^n - 1$ ），每个存储单元存放m位数据。
- C语言的指针：本质上就是主存地址



W的低电平表示:

$\overline{W}$ /W -W

图4.2 主存的硬件内部结构

4.1.5 主存中数据的存放

– 1、存储字长与数据字长的概念

- (1) 存储字长：主存的一个存储单元所存储的二进制位数（如**m位**）
- (2) 数据字长（机器字长，简称**字长**）：计算机一次能处理的二进制数的位数
- 存储字长和数据字长不一定相同，如字长为**32位**的计算机，存储字长可以是**16位**、**32位**或**64位**

– 2、地址访问模式

- 存储字长都是字节的整数倍：如**8位**、**16位**、**32位**、.....
- **主存通常按字节进行编址**
- 例如**32位**计算机，主存既可以按字节访问（**字节地址**），也可以按**16位**半字访问（**半字地址**），还可以按**32位**字进行访问（**字地址**）

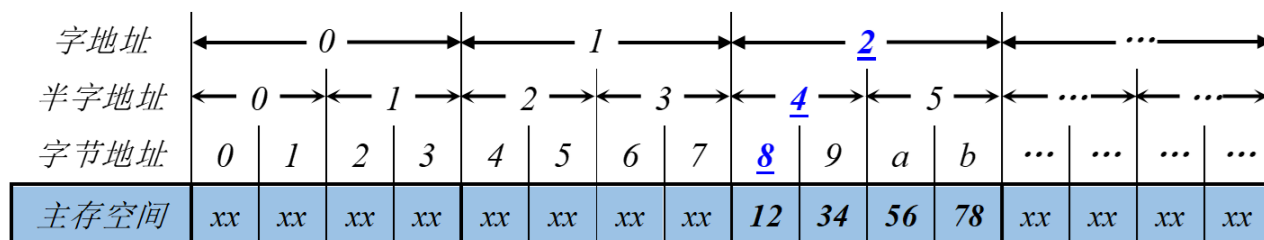


图4.3 不同的主存地址访问模式

4.1.5 主存中数据的存放

- 举例：

小端对齐

- `mov ah,[8]` ;按字节访存, `ah=0x12`
- `mov ax,[8]` ;按半字访存, `ax=0x3412`
- `mov eax,[8]` ;按字访存, `eax=0x78563412`
- `mov eax,[9]`; 未对齐, 产生异常
- `mov eax,[0] mov eax, [4], mov eax,[8]`为正确指令

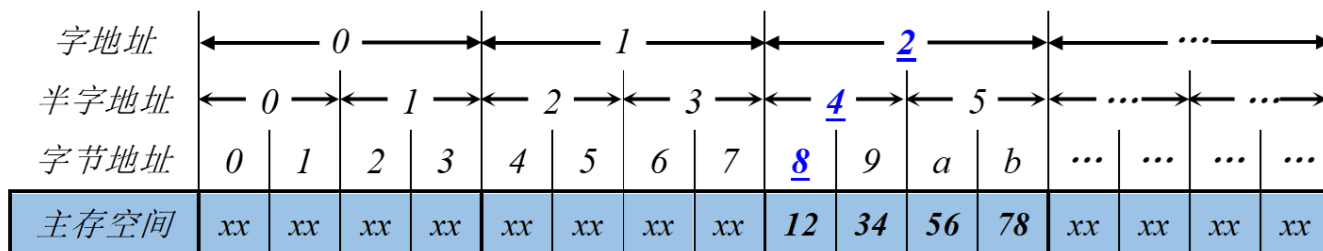


图4.3 不同的主存地址访问模式

注：指令中给出的主存地址实际上是字节地址，并且是起始地址

4.1.5 主存中数据的存放

– 3、大端和小端方式

- **小端方式（Little-Endian）**：存储器的**低**字节地址单元中存放的是数据的**最低**字节
- **大端方式（Big-Endian）**：存储器的**低**字节地址单元中存放的是数据的**最高**字节

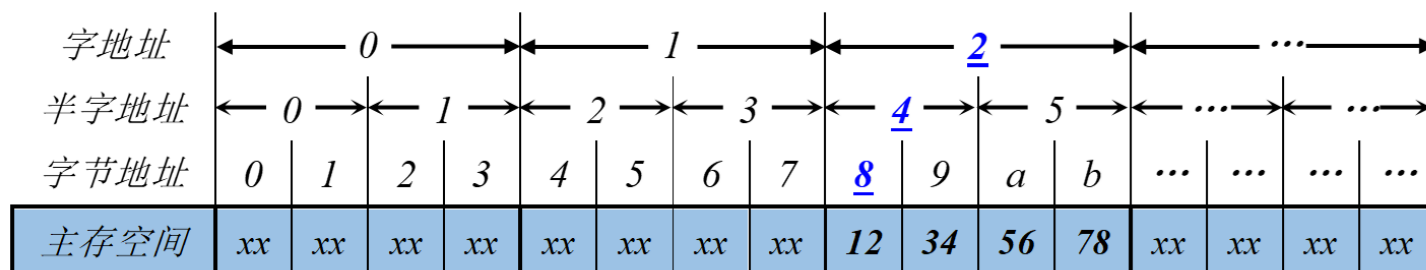


图4.3 不同的主存地址访问模式

- 如果按照**小端**方式，指令“MOV EAX, [8]”得到的数据是：EAX=0x78563412
- 如果按照**大端**方式，指令“MOV EAX, [8]”得到的数据是：EAX=0x12345678
- 采用**大端**方式更符合**人的思维习惯**，而采用**小端**方式则有利于**计算机处理**
- 主流处理器（如Intel x86、IA64处理器、RISC-V处理器）一般都采用小端方式进行数据存放；PowerPC处理器则采用大端方式；ARM、MIPS处理器同时支持小端方式和大端方式

4.1.5 主存中数据的存放

– 4、数据的边界对齐

- C语言中不同数据类型变量的字节长度（表4.1）：

x86: x86通常是指32位系统

IA-64: 64位的英特尔架构

表4.1 不同数据类型变量的字节字长

C语言数据类型	x86	IA-64	C语言数据类型	x86	IA-64
char	1	1	float	4	4
short	2	2	double	8	8
int	4	4	long double	N/A	16
long	4	8	pointer	4	8

- 假设计算机的字长为32位（即1个字=32位=4字节），则边界对齐要求：
 - 双字数据（64位，8字节）起始字节地址最末3位为000（8的倍数）
 - 单字数据（32位，4字节）起始字节地址最末2位为00（4的倍数）
 - 半字数据（16位，2字节）起始字节地址最末1位为0（2的倍数）
 - 单字节数据（8位，1字节）起始字节地址可以任意

4.1.5 主存中数据的存放

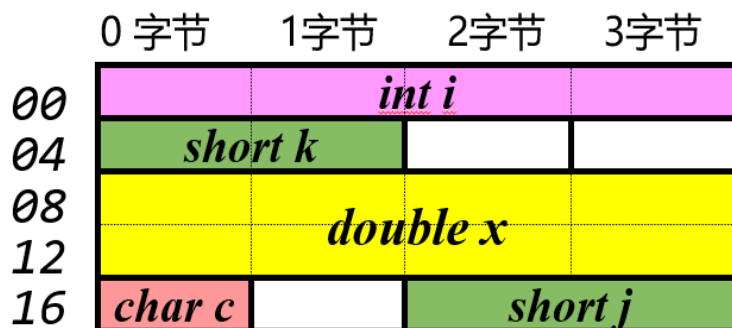


图4.5 对齐方式

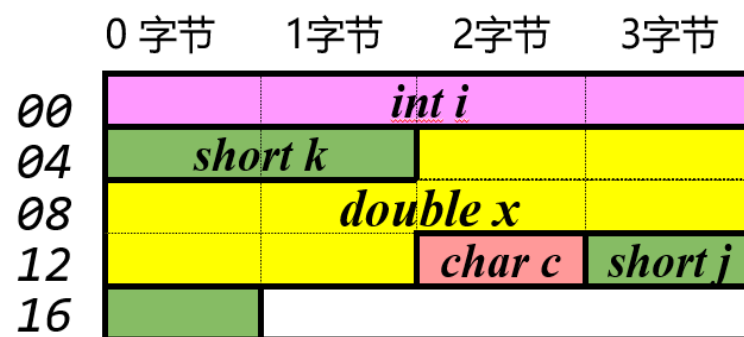


图4.4 未对齐方式

对齐方式:

优点: 访问变量*i*、*k*、*c*、*j*只需要1个存储周期，访问变量*x*需要2个存储周期

缺点: 造成存储空间的浪费（浪费3个字节的存储空间）

未对齐方式:

优点: 不浪费存储空间

缺点: 访问变量*x*需要3个存储周期，访问变量*j*需要2个存储周期，只有访问变量*i*、*k*、*c*需要1个存储周期

参: [内存对齐问题](#)
[内存地址对齐](#)

4.2 半导体存储器

• 4.2.1 静态MOS存储器

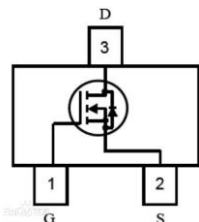
- **MOS**: Metal Oxide Semiconductor Field Effect Transistor, 金属氧化物半导体场效应晶体管
- 存储体以静态MOS存储元为基本单元组成的存储器称为静态MOS存储器, 也称为静态随机存取存储器 (**SRAM**: Static Random Access Memory)
- 1、静态MOS存储单元
 - 存储单元: 也称为存储元, 存放1位二进制数
 - 存储单元电路的基本功能:
 - ① 具有两种稳定状态
 - ② 两种稳定状态经外部信号控制可以相互转换
 - ③ 经控制后能读出其中的信息
 - ④ 无外部原因, 其中的信息能长期保存

参: [DDR - 基础知识](#)

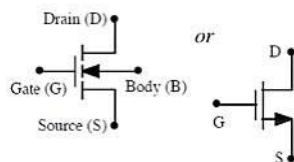
4.2.1 静态MOS存储器

- MOS，是MOSFET（Metal Oxide Semiconductor Field Effect Transistor）的缩写，金属氧化物半导体场效应晶体管，简称**场效应管**。
- G: Gate 栅极；S: Source 源极；D: Drain 漏极。
- 场效应管分为**PMOS**管（P沟道型）和**NMOS**管（N沟道型），属于绝缘栅场效应管。

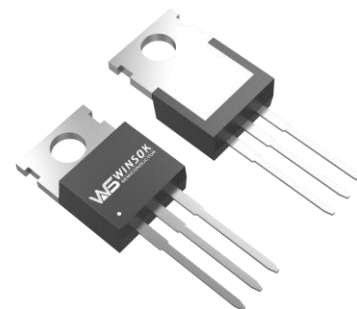
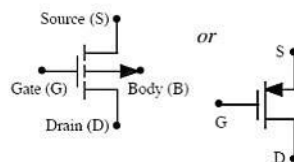
引脚排列图:



n-Channel (NMOS)



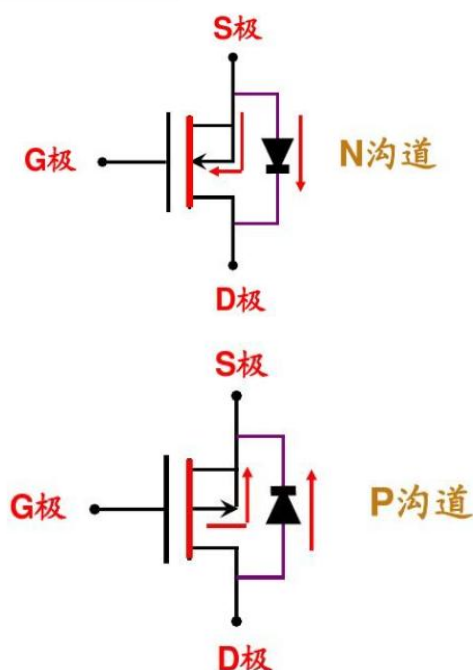
p-Channel (PMOS)



4.2.1 静态MOS存储器

— (1) 6管MOS存储单元

- 6个MOS管 (T_1 、 T_2 、 T_3 、 T_4 、 T_5 、 T_6) 组成存放1位二进制数的SRAM存储单元 (图4.6)
- T_1 、 T_2 : 工作管; T_3 、 T_4 : 负载管; T_5 、 T_6 、 T_7 、 T_8 : 门控管; 其中, T_7 、 T_8 为各个存储单元共用



存放1位二进制数

G: 栅极
S: 源极
D: 漏极

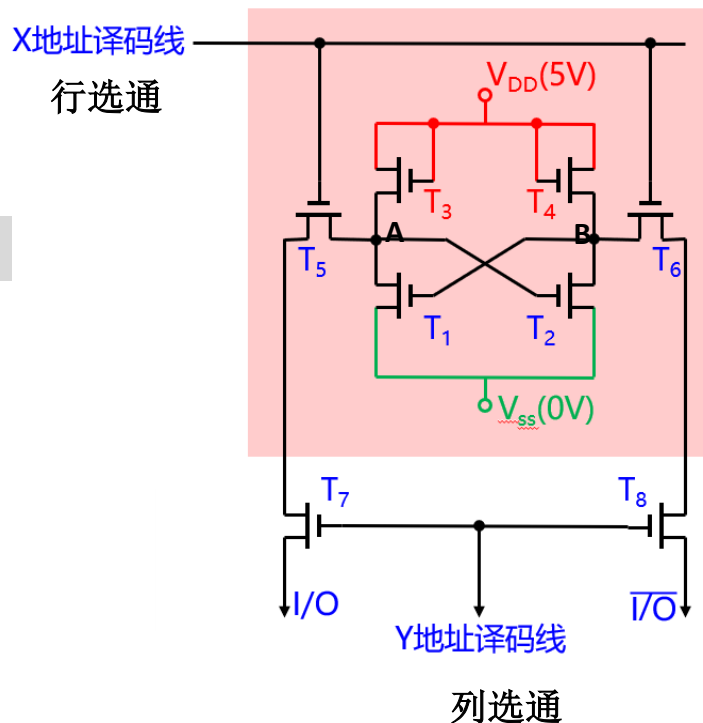


图4.6 6管SRAM存储单元

4.2.1 静态MOS存储器

— (2) 电路状态

- A点高电平、B点低电平，表示存储数据“1” T1截止，T2导通
- A点低电平、B点高电平，表示存储数据“0” T1导通，T2截止

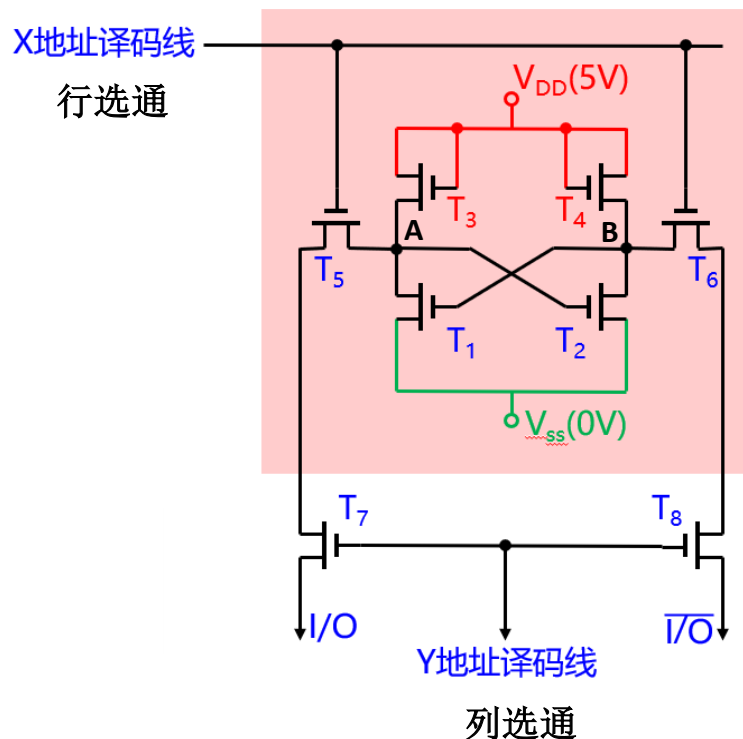


图4.6 6管SRAM存储单元

4.2.1 静态MOS存储器

— (3) 写操作

- 首先给出行选通和列选通信号，使 T_5 、 T_6 、 T_7 、 T_8 门控管打开
- **写入数据“1”**：输入数据为高电平（ I/O =高电平， $\bar{I/O}$ =低电平），则使A点高电平、B点低电平，则写入数据“1”
- **写入数据“0”**：输入数据为低电平（ I/O =低电平， $\bar{I/O}$ =高电平），则使A点低电平、B点高电平，则写入数据“0”

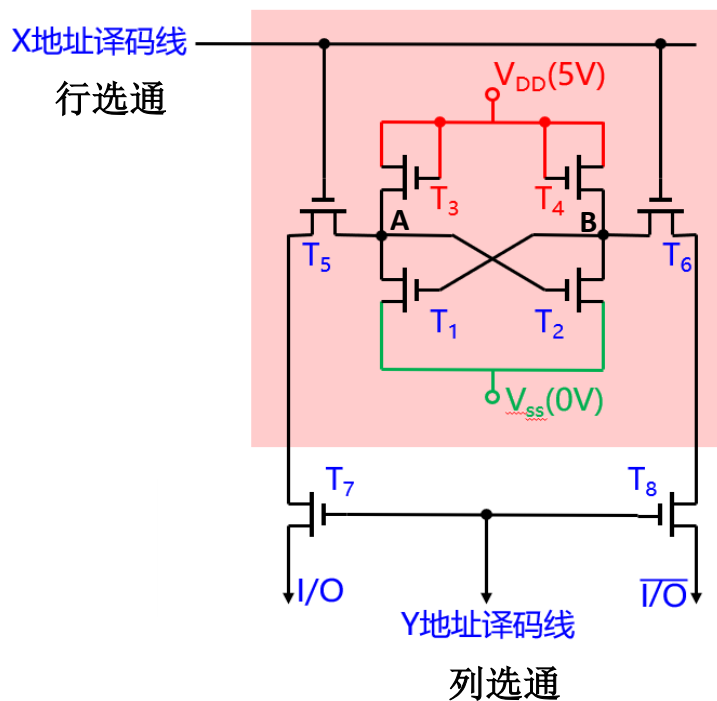


图4.6 6管SRAM存储单元

4.2.1 静态MOS存储器

— (4) 读操作

- 首先给出行选通和列选通信号，使 T_5 、 T_6 、 T_7 、 T_8 门控管打开
- 读出数据“1”：因为A点是高电平、B点是低电平，因此输出数据为高电平（ I/O =高电平， $\bar{I/O}$ =低电平），表示读出数据“1”
- 读出数据“0”：因为A点是低电平、B点是高电平，因此输出数据为低电平（ I/O =低电平， $\bar{I/O}$ =高电平），表示读出数据“0”

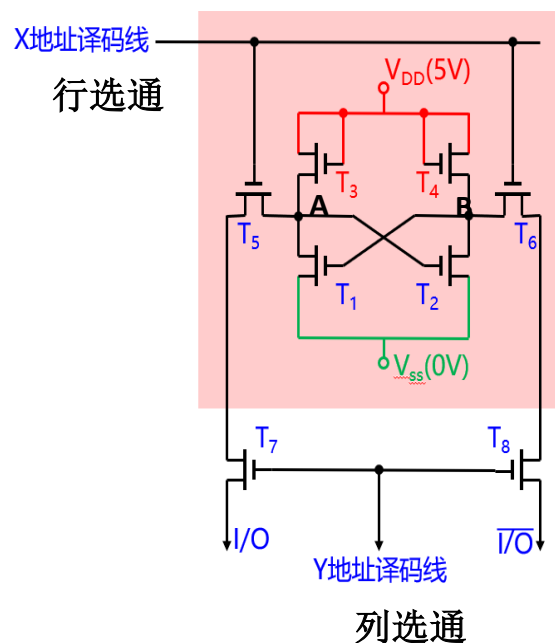


图4.6 6管SRAM存储单元

4.2.1 静态MOS存储器

— (5) 信息的保持

- 不给出行选通和列选通信号，因此 T_5 、 T_6 、 T_7 、 T_8 门控管截止
- 只要电源不断，A点和B点的信息将保持不变，存储单元的状态就一直不变，并且在读出时也不会破坏原有数据

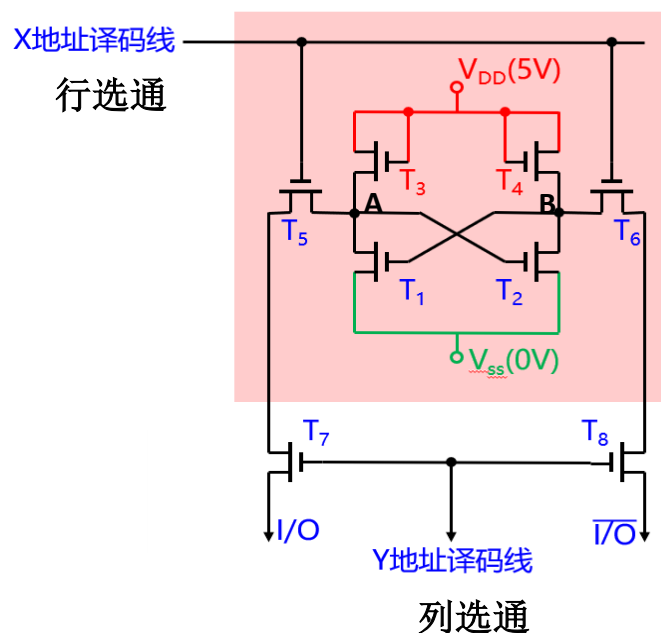


图4.6 6管SRAM存储单元

4.2.1 静态MOS存储器

– 2、存储单元扩展

- 存放1位二进制数的6管SRAM存储单元（图4.7）
- SRAM存储单元的**行扩展**（图4.8a）：4行、1列，存放4位二进制数
- SRAM存储单元的**行列扩展**（图4.8b）：4行、2列，存放8位二进制数

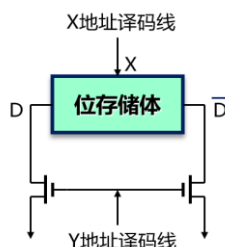


图4.7 6管SRAM存储单元

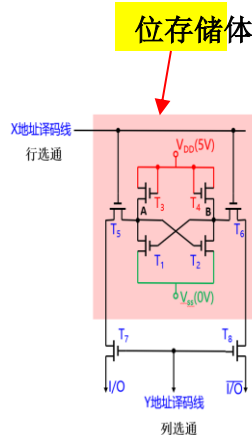
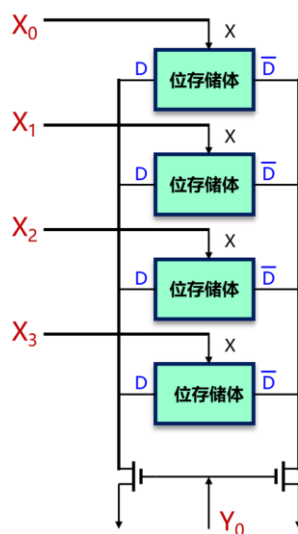
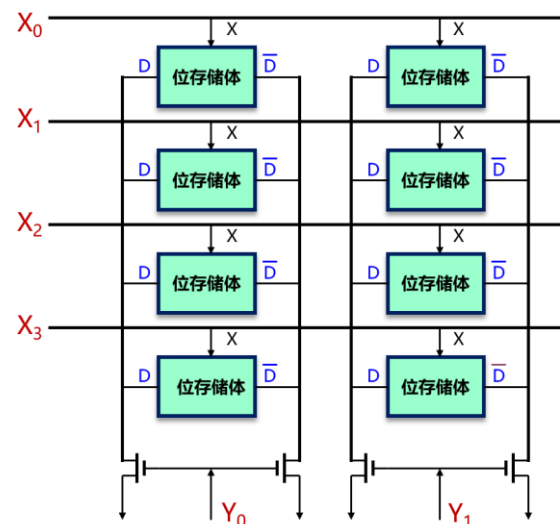


图4.6 6管SRAM存储单元



(a) 一维行扩展



(b) 二维行列扩展

图4.8 SRAM存储单元的行列扩展

4.2.1 静态MOS存储器

- **单译码结构**：图4.9a， n 位地址译码后得到 2^n 个译码线（ $0 \sim 2^n$ ），共有 2^n 个存储单元
- **双译码结构**：图4.9b， X 地址译码后得到 k 行， Y 地址译码后得到 n 列，共有 kn 个存储单元

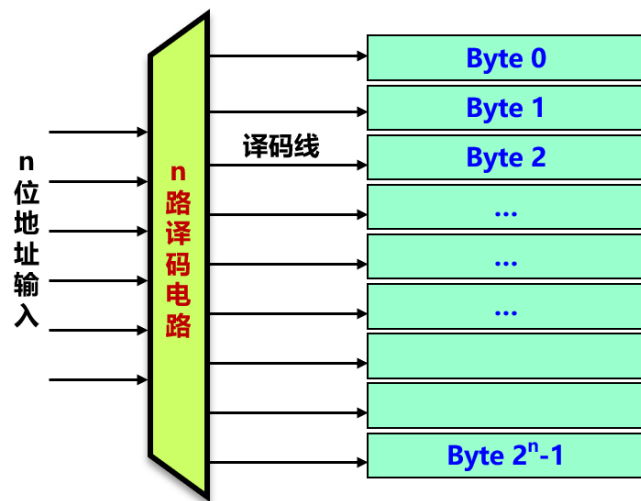


图4.9a 单译码结构

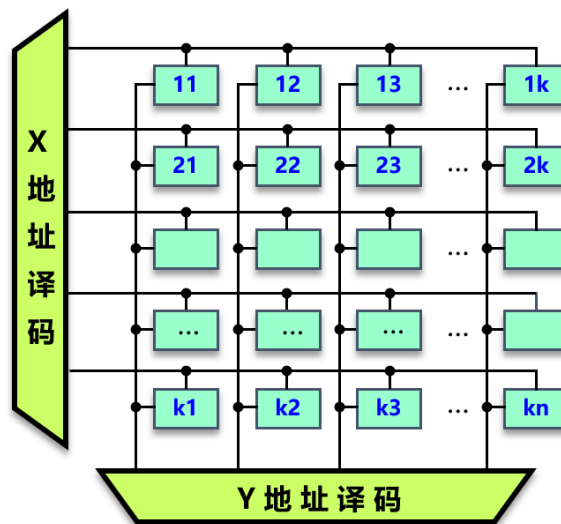


图4.9b 双译码结构

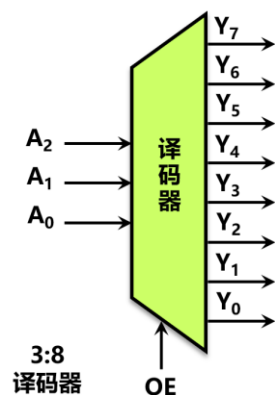
4.2.1 静态MOS存储器

- 译码器：3-8译码器

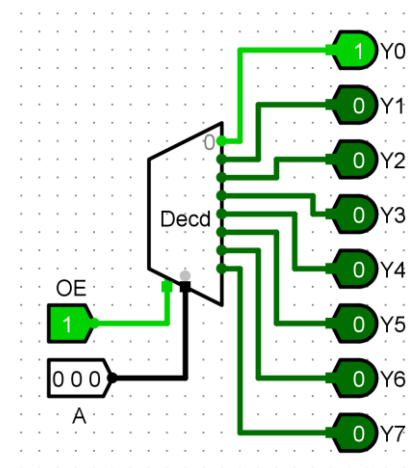
- 输入：A₂、A₁、A₀，OE（OE为输出允许信号，OE=1译码器工作，OE=0译码器不工作）
- 输出：Y₇、Y₆、Y₅、Y₄、Y₃、Y₂、Y₁、Y₀
- x：表示任意（可能是0，也可能是1）

3-8译码器的输入输出关系

输入				输出							
A ₂	A ₁	A ₀	OE	Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀
0	0	0	1	0	0	0	0	0	0	0	1
0	0	1	1	0	0	0	0	0	0	1	0
0	1	0	1	0	0	0	0	0	1	0	0
0	1	1	1	0	0	0	0	1	0	0	0
1	0	0	1	0	0	0	1	0	0	0	0
1	0	1	1	0	0	1	0	0	0	0	0
1	1	0	1	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0
x	x	x	0	x	x	x	x	x	x	x	x



3-8译码器的符号



Logisim实现的3-8译码器

4.2.1 静态MOS存储器

— 3、静态MOS存储器的结构

- 静态MOS存储器包括：**存储体**（存储单元阵列）、**地址译码电路**（X向地址译码器、Y向地址译码器、X向驱动器、Y向驱动器）、**I/O电路和控制电路**等（图4.10）

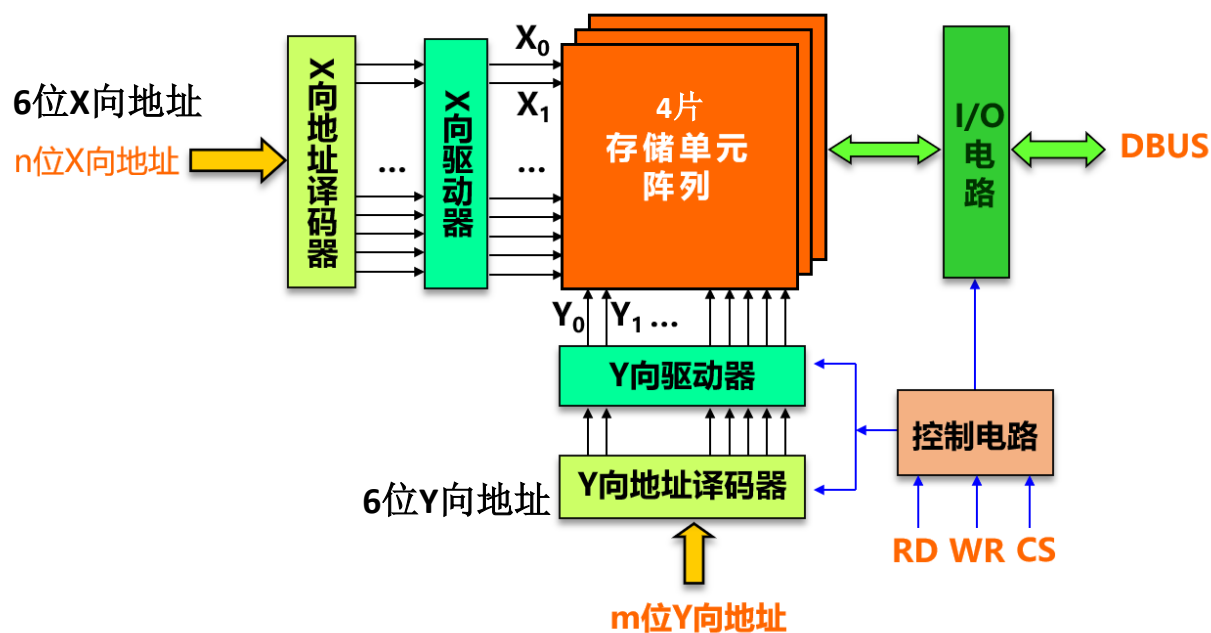
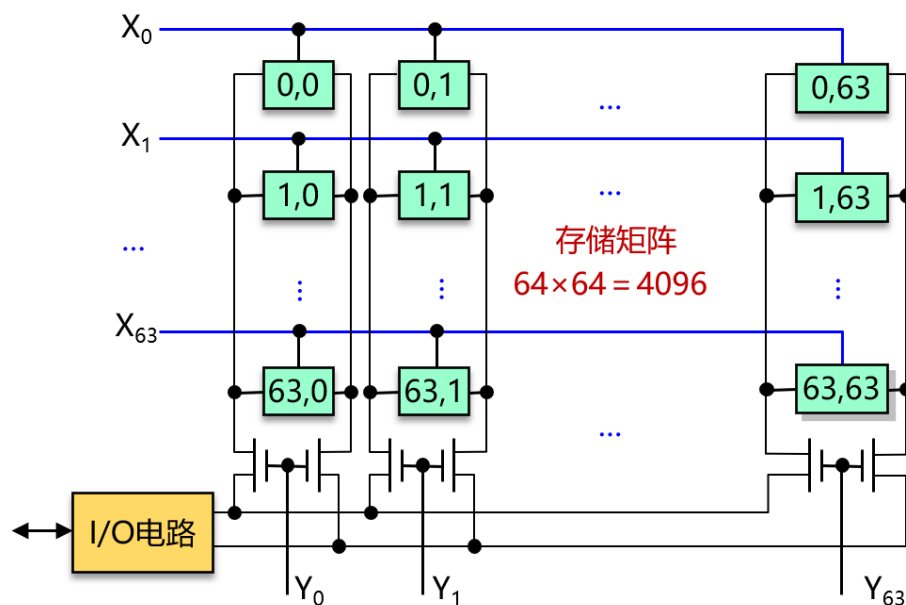


图4.10 4096x4位的静态MOS存储器结构框图

4.2.1 静态MOS存储器

- (1) 存储体

– 图4.10的存储单元阵列（存储体）包括4片64x64位的存储矩阵



共有4片

64x64位存储矩阵

4.2.1 静态MOS存储器

- (2) 地址译码器
 - 包括：X向地址译码器、Y向地址译码器；其中X向地址译码器和Y向地址译码器都是**6-64译码器**（6个输入、64个输出）
- (3) 驱动器
 - 包括：X向驱动器、Y向驱动器；驱动器的输出和输入在逻辑上是相等的，其目的是增强负载能力

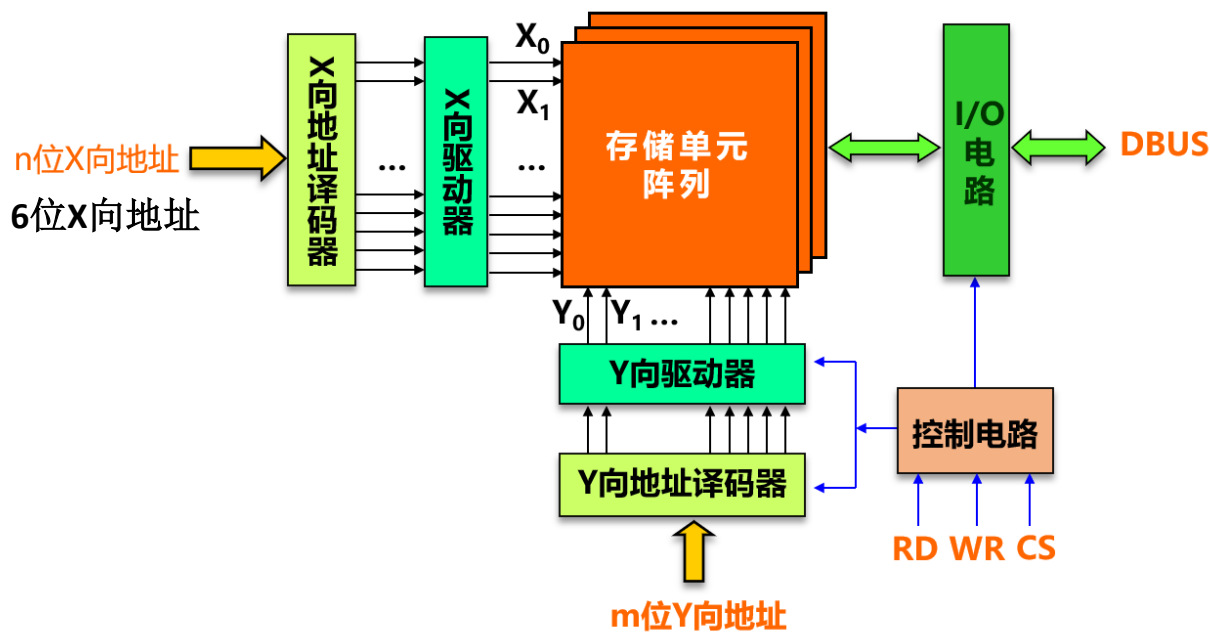


图4.10 4096x4位的静态MOS存储器结构框图

4.2.1 静态MOS存储器

- (4) I/O电路

- I/O电路用来控制数据的输入（写操作）和输出（读操作）
- DBUS: 数据总线（Data Bus）

- (5) 片选和读写控制电路

- CS: Chip Select, 片选信号
- WR: 写信号
- RD: 读信号

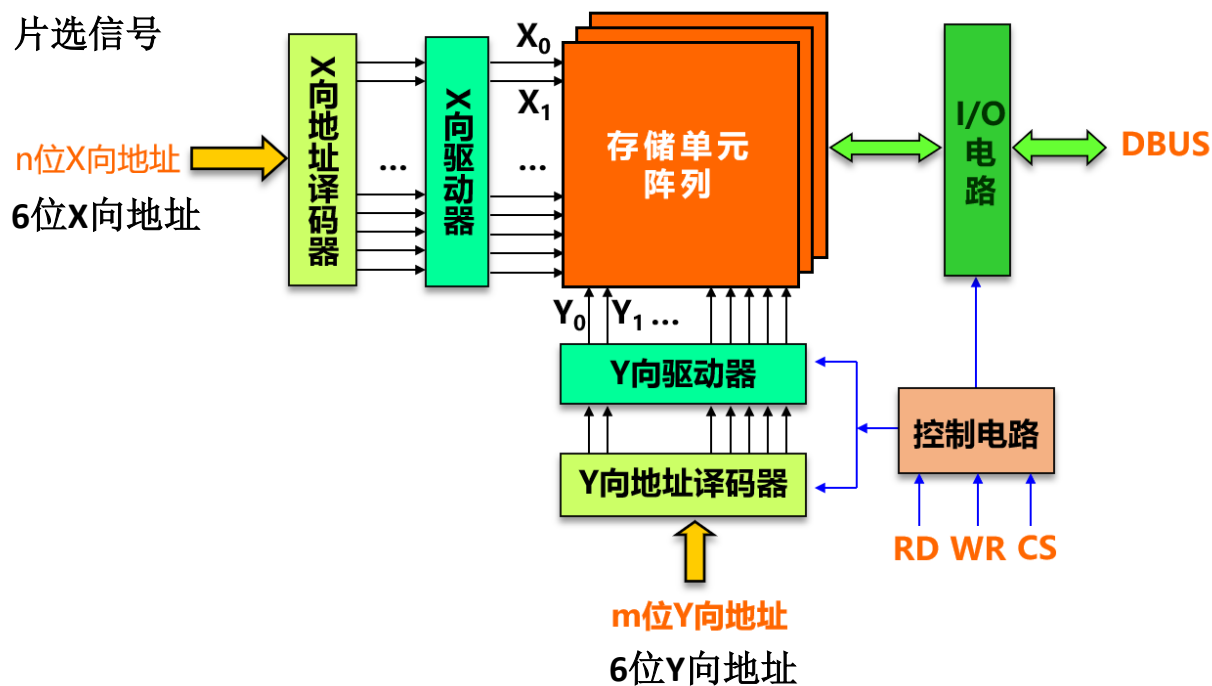
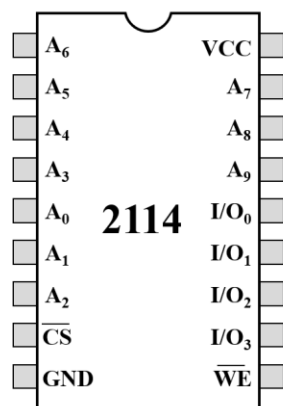


图4.10 4096x4位的静态MOS存储器结构框图

4.2.1 静态MOS存储器

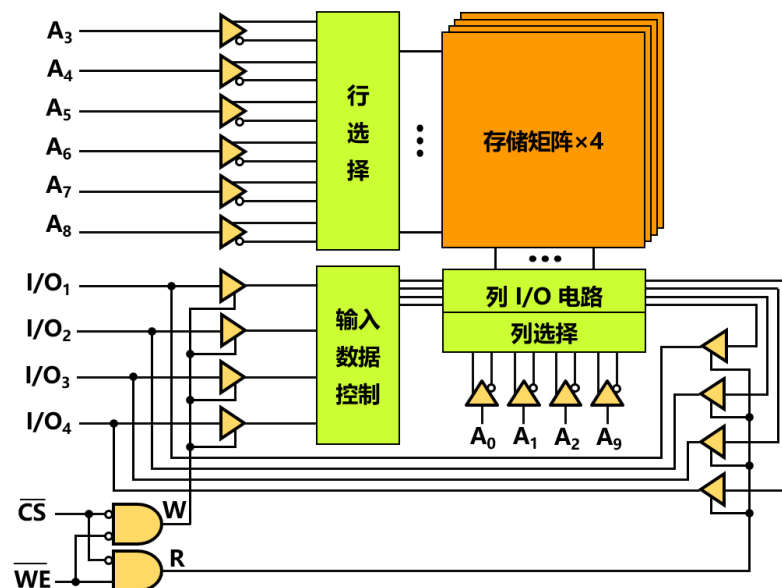
– 4、静态MOS存储器芯片实例

- **Intel 2114**: Intel公司推出的1Kx4位SRAM芯片，其容量为1024x4位，即有1024个单元、每个单元存放4位二进制数
- 地址线：10根（ $A_9 \sim A_0$ ， $2^{10}=1024$ ）；数据线：4根（ $I/O_3 \sim I/O_0$ ）
- $/CS$ ：片选信号（低电平有效）； $/WE$ ：读写控制信号
- 读操作时： $/WE$ =高电平；写操作时： $/WE$ =低电平



(a) 封装引脚

- 地址线（单向， $A_9 \sim A_0$ ）
- 数据线（双向， $I/O_3 \sim I/O_0$ ）
- 读写控制线（Write Enable，低电平有效）
- 片选线（Chip Select，低电平有效）
- 电源线（VCC）
- 地线（GND）

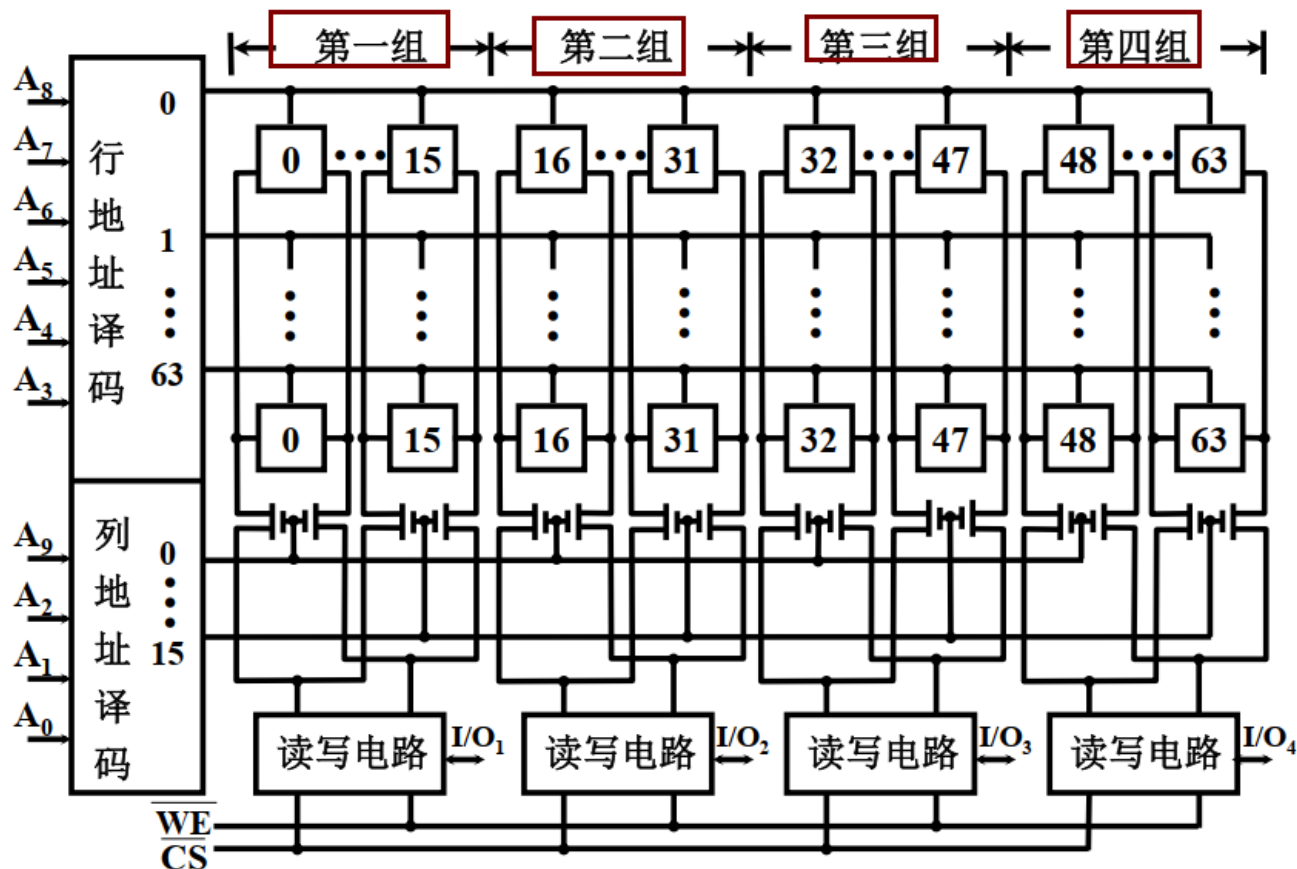


(b) 内部结构

图4.11 Intel 2114 结构

4.2.1 静态MOS存储器

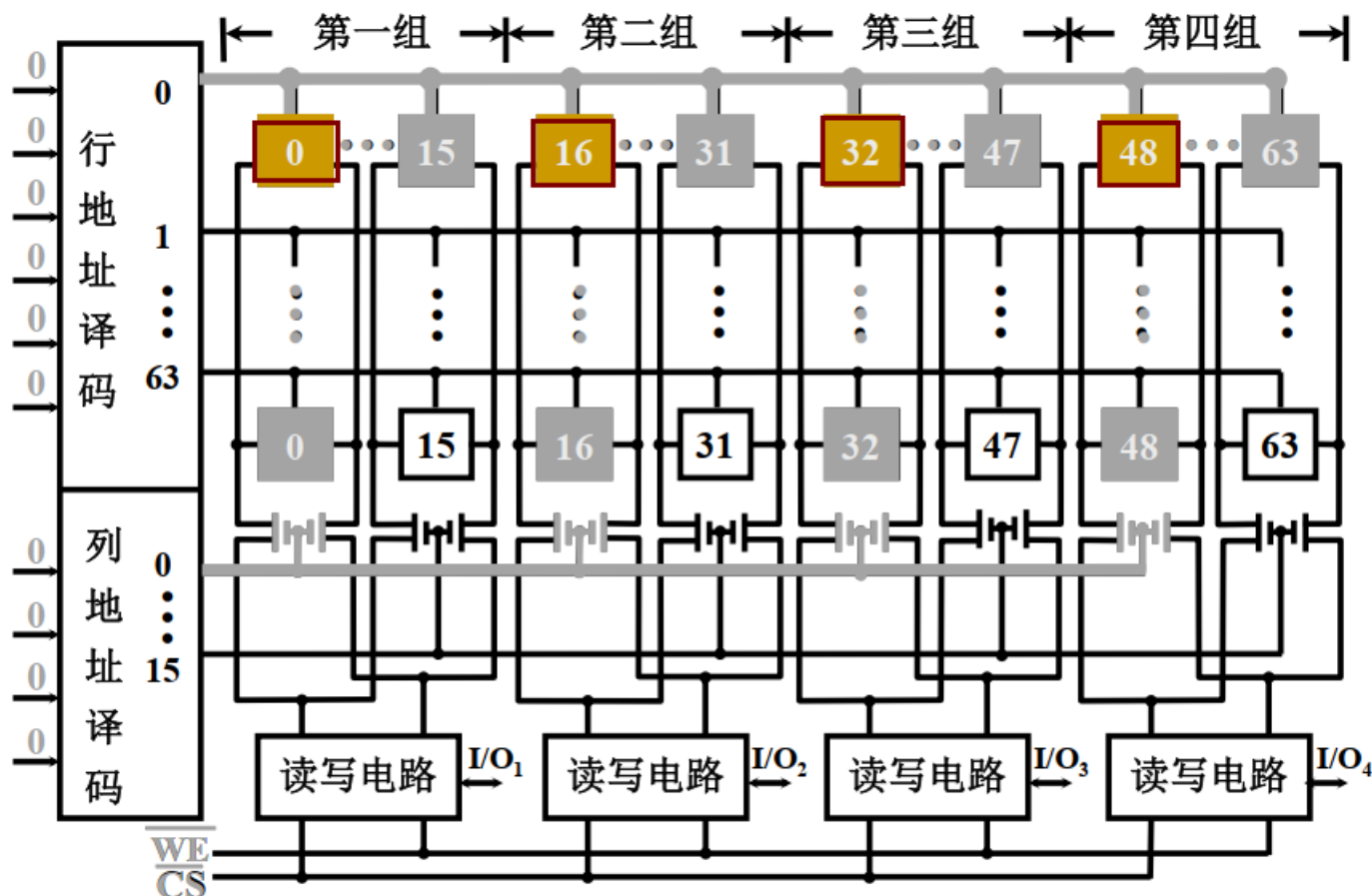
Intel 2114 RAM 矩阵 (64 × 64) 读



注：64行*64列（4组*16列）行地址译码后选中某一行，列地址译码后，选中4组中的某一列，从而满足4路I/O输出

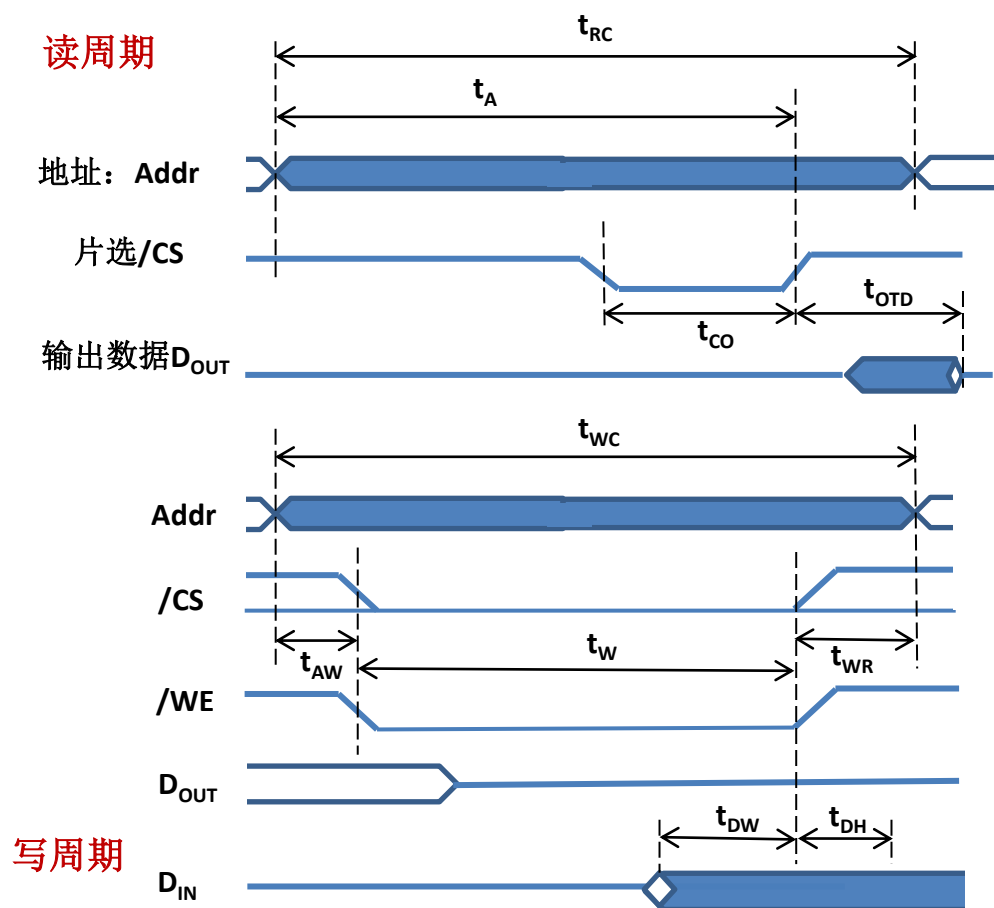
4.2.1 静态MOS存储器

Intel 2114 RAM 矩阵 (64×64) 读



4.2.1 静态MOS存储器

— 5、SRAM读写时序



t_{RC} : 读周期时间

t_A : 读数据时间

t_{CO} : 片选有效到数据稳定输出之间的时间延迟

t_{OTD} : 片选断开到输出端数据还能保持的时间

t_{WC} : 写周期时间

t_W : 写数据时间

t_{WR} : 写恢复时间

t_{DW} : 数据有效时间

t_{DH} : 写信号无效后数据保持时间

t_{AW} : 地址有效到写信号有效之间的时间间隔

图4.12 Intel 2114的读写周期时序及时间参数

4.2.2 动态MOS存储器

- 存储体以动态MOS存储元为基本单元组成的存储器称为动态MOS存储器，也称为**动态随机存取存储器**（**DRAM**: Dynamic Random Access Memory）
- 静态MOS存储器（SRAM）存储1位二进制数需要6个MOS管，其存储密度较低
- 动态MOS存储器（DRAM）则采用数量较少的MOS管（如4个MOS管、3个MOS管、1个MOS管）和**存储电容**来存储1位二进制数，其**存储密度较高**

4.2.2 动态MOS存储器

1、单管动态MOS存储单元

- 使用1个MOS管 (T_1) 和1个电容 (C) 构成存储单元，存放1位二进制数
- 电容C有电荷，表示存放数据“1”；电容C无电荷，表示存放数据“0”

单管DRAM的特点：

- ① 因为电容的充放电过程较慢，因此单管DRAM的读写速度比SRAM慢
- ② 单管DRAM读出时，会引起电容电荷的减少，从而破坏原有数据，为避免数据丢失，在读出后需要将数据重新写入，该过程称为数据恢复（再生）
注：在读操作中，电容器中的电荷通过位线传输到放大器，放大器放大信号并保持原有单元的内容，这个过程不仅读取了存取单元的数据，还通过位线的电压变化恢复电容器的电荷，从而实现了数据的再生和刷新（隐式刷新）
- ③ 数据恢复以及读出信号需要再生放大电路，该电路为该列上所有的存储单元共享
- ④ 因为电容上的电荷会逐渐泄露，数据只能保存较短的时间，为避免数据丢失，必须定期采用类似读操作的方式对存储单元补充电荷，这个过程称为刷新

图4.13 单管DRAM内部电路

图4.13 单管DRAM内部电路

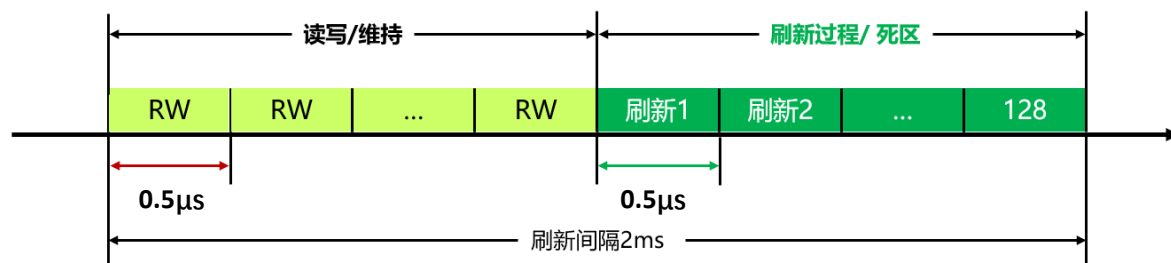
4.2.2 动态MOS存储器

3、动态MOS存储的刷新

- 因为DRAM电容上的电荷会逐渐泄露，数据只能保存较短的时间，为避免数据丢失，必须定期采用类似读操作的方式对存储单元补充电荷，这个过程称为刷新
- 最大刷新周期：信息存储到数据丢失之前的时间称为最大刷新周期
- 刷新周期：存储器实际完成两次完整刷新之间的时间间隔，常见的有2ms、4ms、8ms等
- DRAM的刷新是按行进行的，每次完成1行的刷新，刷新操作只需要给出存储器的行地址，不需要给出存储器的列地址
- 内存争用：DRAM刷新是由内存控制器完成，此时CPU不能访问存储器，称为内存争用问题（也称死区）
- 常见的刷新方式：
 - ① 集中刷新方式
 - ② 分散刷新方式
 - ③ 异步刷新方式

假设DRAM存储体为128行x128列的结构，存储器的读写周期=0.5μs

2ms内需要完成128行的刷新



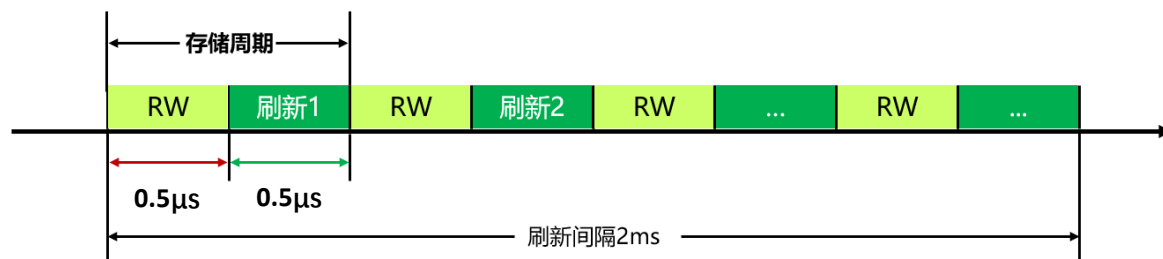
集中刷新方式

优点：读写操作期间不受刷新操作的影响，因此存储器的速度比较快

缺点：存在较长时间的“死区”

死区率=128/4000=3.2%

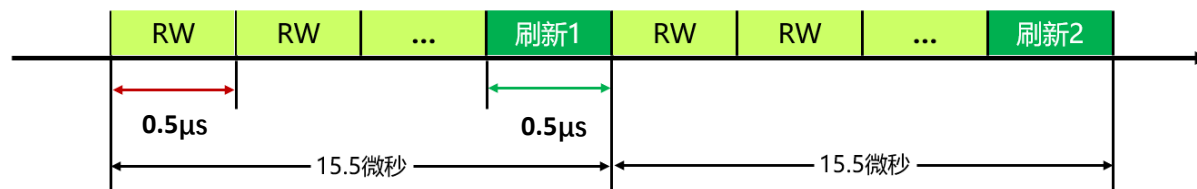
4000=2ms/0.5μs



分散刷新方式

优点：不存在“死区”

缺点：存储周期增加（存储周期=1μs），降低存储器的速度
刷新太过频繁
刷新周期是128*1us=128us<2ms



异步刷新方式

是集中刷新方式和分散刷新方式的结合

既没有增加存储周期，也没有较长时间的“死区”

是常用的刷新方式
保证2ms内刷新一次就好

2ms/128=15.625μs≈15.5μs

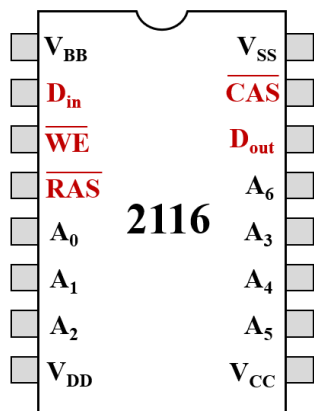
图4.14 刷新时间分配

4.2.2 动态MOS存储器

4、动态MOS存储芯片举例

- **Intel 2116**: 是Intel公司生产的容量为16Kx1位的DRAM；存储矩阵为 $128 \times 128 = 16K \times 1$
- 采用**地址复用技术**，由/RAS和/CAS先后将7位地址分别送到行地址锁存器和列地址锁存器中
- **DRAM的结构与SRAM的结构的不同点**:
 - ① **DRAM**的地址线一般采用复用技术，即CPU分时传送行、列地址，节省一半的地址线
 - ② **DRAM**无片选信号，通常可由/RAS和/CAS选通信号代替
 - ③ 数据输入（DIN）和数据输出（DOUT）分开且可锁存

[深入内存/主存：解剖DRAM存储器](#)



(a) 封装引脚

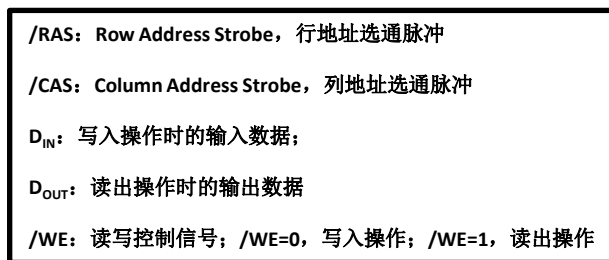
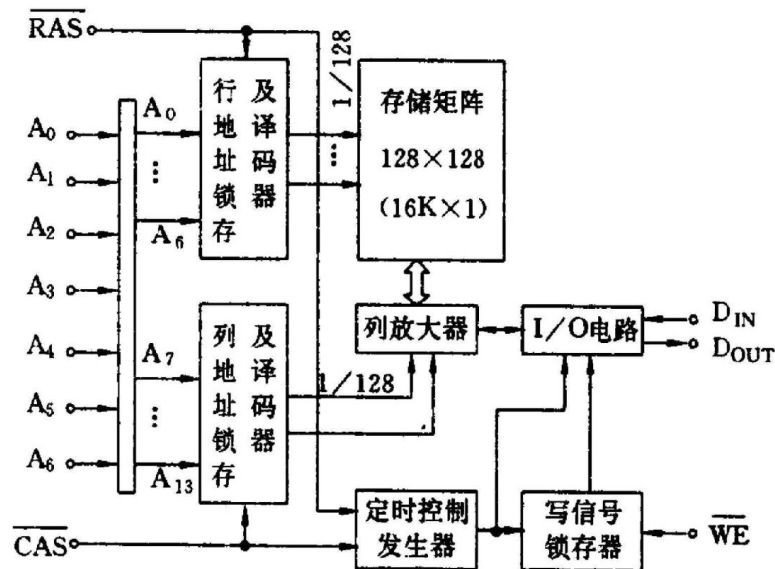
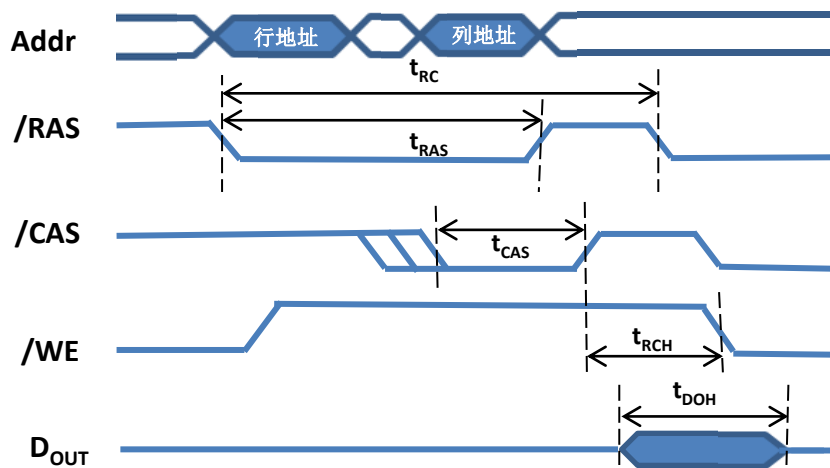


图4.15 Intel 2116 结构



(b) 内部结构

– 5、动态MOS存储器的读写周期



读周期

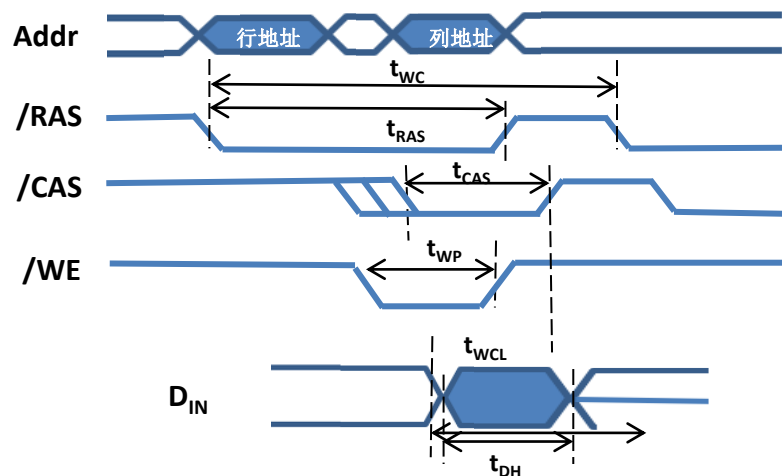
t_{RC} : 读周期时间

t_{RAS} : /RAS的脉冲宽度

t_{CAS} : /CAS的脉冲宽度

t_{RCH} : 读命令保持时间

t_{DOH} : 数据输出保持时间



写周期

t_{WC} : 写周期时间

t_{RAS} : /RAS的脉冲宽度

t_{CAS} : /CAS的脉冲宽度

t_{WP} : 写命令的脉冲宽度

t_{WCL} : 从/CAS无效到写命令开始的时间

t_{DH} : 写入数据保持时间

图4.16 DRAM芯片的读写周期时序示意图

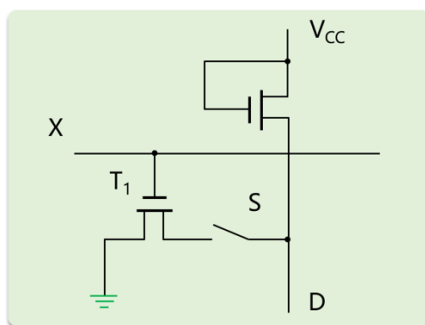
4.2.3 只读存储器

- **ROM: Read Only Memory**, 只读存储器
- **ROM的特点**: 具有非易失性, 即使断电保存的信息也不会丢失, **ROM**常用于存放一些不需要修改的程序, 如微程序、系统软件 (**ROM-BIOS**) 等
- **ROM通常分为**:
 - ①掩模式只读存储器: **MROM**, **Mask ROM**
 - ②可编程只读存储器: **PROM**, **Programmable ROM**
 - ③可擦除可编程只读存储器: **EPROM**, **Erasable Programmable ROM**
 - ④电可擦除可编程只读存储器: **EEPROM**, **Electrically Erasable Programmable ROM**, **E²PROM**, **E2PROM**
 - ⑤闪存: **Flash ROM**

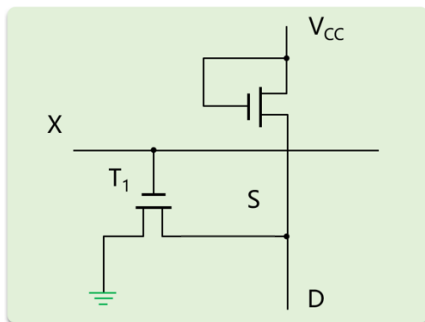
4.2.3 只读存储器

1、掩模式只读存储器（MROM, Mask ROM）

- **Mask ROM**的存储单元（存放1位二进制数）：
 - 如果开关s是打开的，则D的输出为高电平，表示读出“1”
 - 如果开关s是闭合的，则D的输出为低电平，表示读出“0”
- **Mask ROM**阵列：4行x4列，存放16位二进制数
- **Mask ROM**中的内容在厂家生产时就按要求做好



开关s打开，表示“1”



开关s闭合，表示“0”

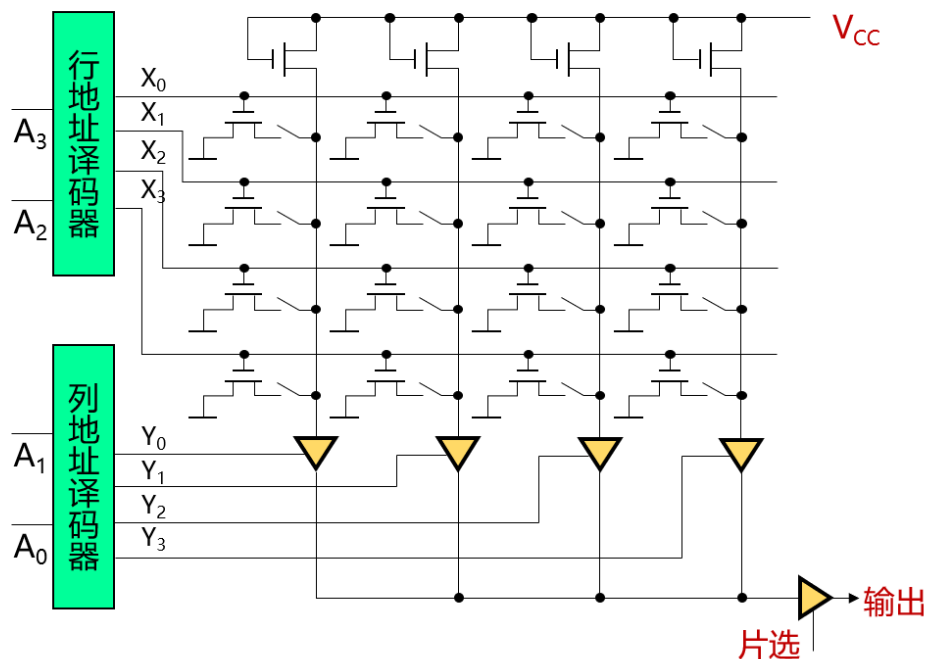
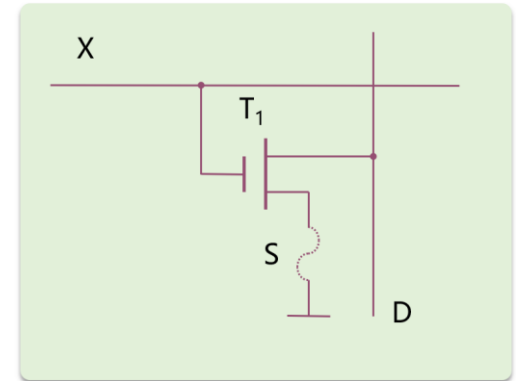


图4.17 (a) MROM存储单元

4.2.3 只读存储器

- 2、可编程只读存储器（PROM，Programmable ROM）
- PROM的存储单元（存放1位二进制数）：
 - 用熔丝S代替Mask ROM的开关；新购买的PROM芯片，熔丝S是连接的，此时输出D为低电平，表示读出“0”，即新的PROM芯片中所有的数据都是“0”
 - 可以通过编程器将熔丝熔断，则输出D为高电平，表示读出“1”；熔丝熔断后不能再接通，因此PROM芯片的编程（将部分存储单元的“0”变为“1”）是一次性的
- 反向二极管的PROM：
 - 采用反向二极管代替熔丝；新的PROM其反向二极管是不导通的，此时输出D为高电平，表示读出“1”，即新的PROM芯片中使用的数据都是“1”
 - 可以通过编程器将反向二极管导通，此时输出D为低电平，表示读出“0”，即编程的目的是将部分“1”变为“0”，且反向二极管导通后，将永远导通（编程是一次性的）



熔丝连接，表示“0”

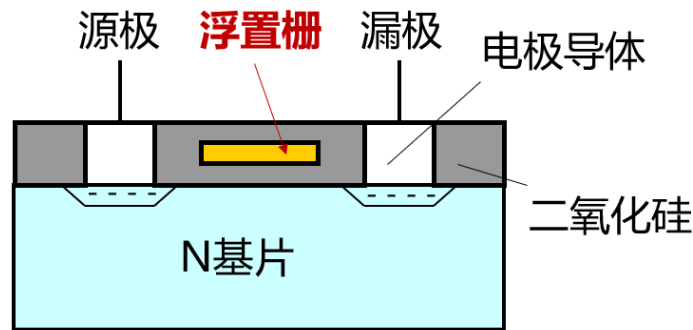
熔丝熔断，表示“1”

图4.17 PROM存储单元

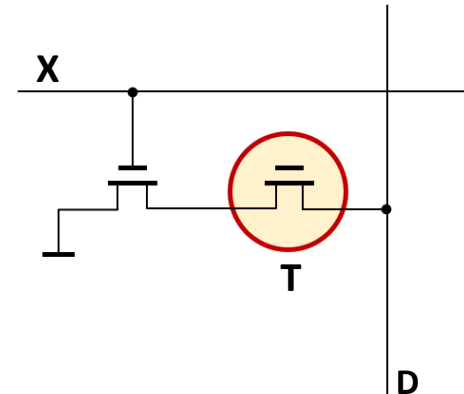
4.2.3 只读存储器

– 3、可擦除可编程只读存储器（EPROM, Erasable Programmable ROM）

- **EPROM**: 采用浮栅型MOS存储单元的结构
- 新购买的EPROM其MOS管T全部截止（右下角的图），输出D为高电平，表示读出“1”，即新购买的EPROM中全部存放“1”
- **编程**: 通过施加编程电压（通常为+12.5V），使部分存储单元的MOS管T导通，输出D为低电平，表示写入“0”，即将EPROM的部分存储单元由“1”变为“0”
- **读出**: 编程后的EPROM在正常电压（通常为+5V）下即可读出其中的内容，其中没有编程过的存储单元输出为“1”，编程过的存储单元输出为“0”
- **擦除**: 通过紫外线持续照射，EPROM所有MOS管T全部截止，EPROM的内容又恢复为全部“1”

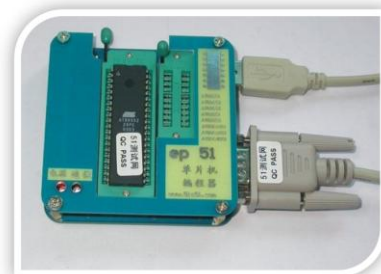


EPROM存储单元



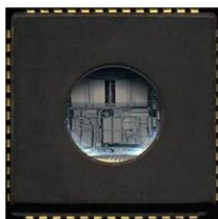
EPROM等效电路

图4.18 EMROM存储单元



平时使用时的EPROM（平时使用时石英玻璃窗口需要封上）

EPROM编程器（编程时石英玻璃窗口需要封上）



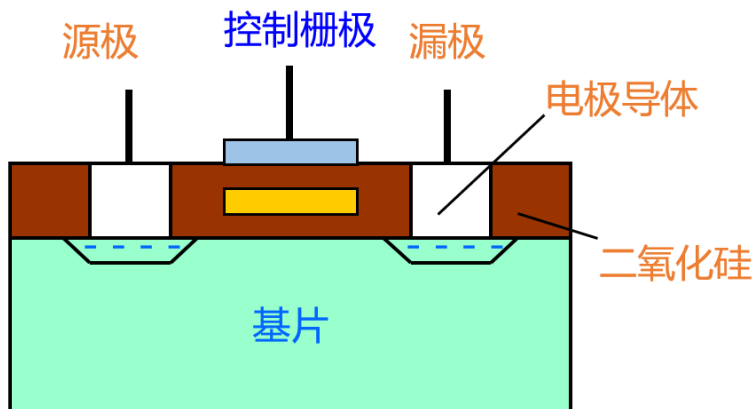
紫外线擦除时的EPROM（擦除时石英玻璃窗口需要露出）

EPROM紫外线擦除器（擦除时石英玻璃窗口需要露出）

4.2.3 只读存储器

— 4、电可擦除可编程只读存储器（EEPROM, Electrically Erasable Programmable ROM）

- EEPROM（E²PROM, E2PROM）是将EPROM的浮置栅改为控制栅极，新购置的EEPROM其内容全部为“1”
- EEPROM的编程（将部分存储单元的内容由“1”变为“0”）、读出方式与EPROM相同
- EEPROM的擦除：不需要在紫外线下照射，而是将控制栅极加上高电平，就可以使该存储单元的内容由“0”变为“1”
- 因此，EEPROM的擦除可以精准地删除（将“0”变为“1”）某一存储单元，而不是一次性的擦除芯片上的所有数据（使所有存储单元的内容都变为“1”）



EEPROM存储单元



EEPROM芯片：2816

4.2.3 只读存储器

– 5、闪存（Flash ROM）

- 闪存是一种快速擦写、非易失性存储器，可以在线进行擦除和重写，其逻辑结构与EEPROM相似
- 闪存的工作方式：读工作方式、编程工作方式、擦除工作方式、功耗下降方式
- 闪存类型：
 - ① **NOR Flash**: Intel于1988年首先开发出NOR Flash技术（采用或非门），彻底改变了原先由EPROM和EEPROM一统天下的局面
 - ② **NAND Flash**: 1989年，东芝公司发布了NAND Flash结构（采用与非门），强调降低每比特的成本，更高的性能，并且象磁盘一样可以通过接口轻松升级
- 闪存的应用：U盘、SSD（Solid State Disk, Solid State Drive, 固态硬盘）、SD卡（Secure Digital Memory Card）、Micro SD卡等



U盘



SSD（固态硬盘）



SD卡



Micro SD卡

NAND Flash和NOR Flash的比较

1989年东芝公司

1988年Intel公司

	NAND Flash	NOR Flash
芯片容量	<32GBit	<1GBit
访问方式	顺序读写	随机读写
接口方式	任意I/O口	特定完整存储器接口
读写性能	读取快（顺序读） 写入快 擦除快（可按块擦除）	读取快（RAM方式） 写入慢 写入慢
使用寿命	百万次	十万次
价格	低廉	高昂

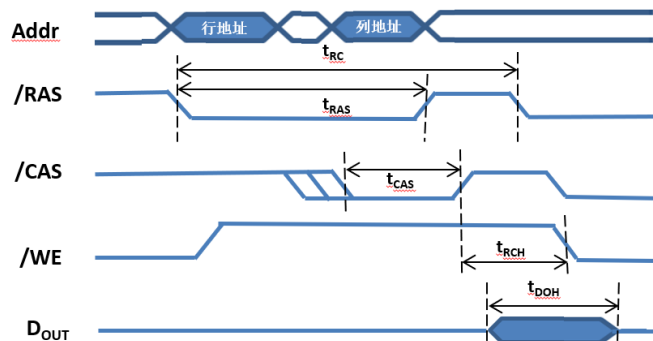
容量大
价格低
顺序读写

容量小
价格高
随机读写

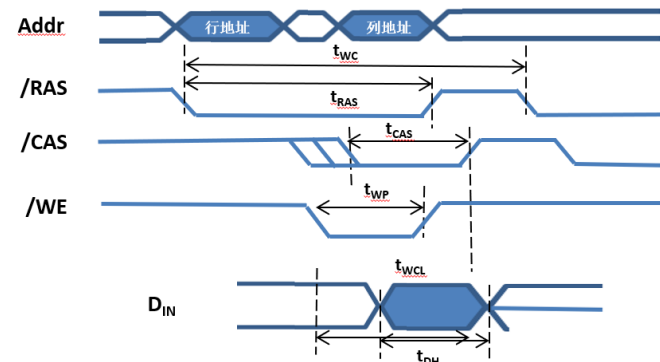
4.2.4 DRAM的发展

– 1、传统DRAM

- 例如，对Intel 2116芯片（DRAM）进行读出操作，首先给出行地址，然后给出/RAS信号锁存行地址；接着给出列地址，并用/CAS信号锁存列地址；从而选择被访问的存储单元；经过一段时间的延迟后才能读出数据
- 每一次数据访问都需要经历完整的行地址、列地址阶段，因此传统DRAM的工作方式效率比较低下



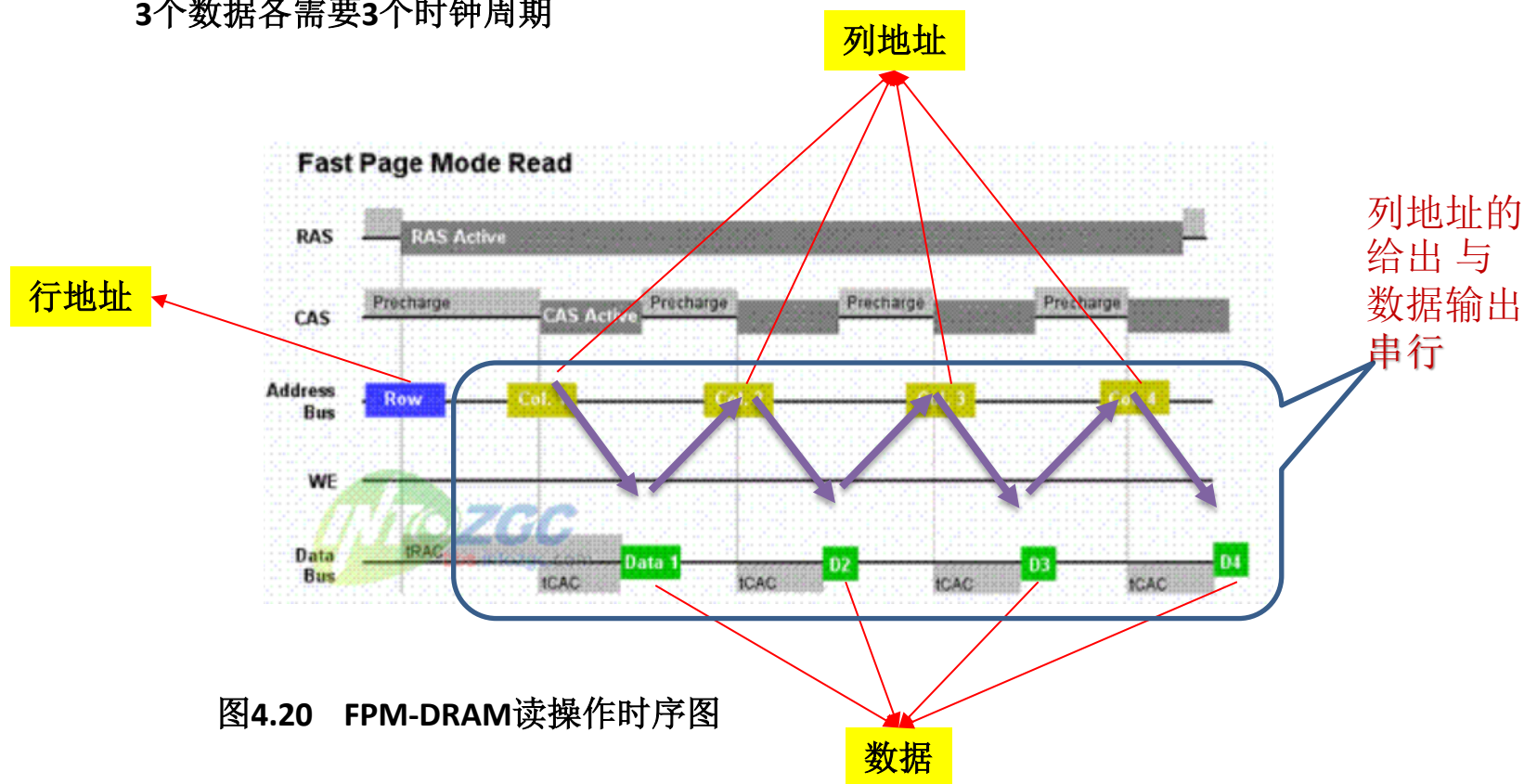
DRAM的读周期



DRAM的写周期

— 2、FPM-DRAM

- **FPM-DRAM: Fast Page Mode DRAM, 快页模式DRAM**；这里的页是指DRAM中某一行的所有存储单元
- 读写动态存储器同一行的数据时，其行地址第一次读写时锁定后保持不变，以后读写该行多列中的数据时，仅锁存列地址即可，省去了锁存行地址的时间，加快了主存储器的读写速度。
- 如果要访问的4个数据位于同一行，只需要给出1个行地址，然后分别给出4个列地址，就可以得到当前页（行）中的4个数据，从而提高存储器的访问速度
- 访问4个数据的FPM-DRAM的典型时序是：**6-3-3-3**，即第1个数据需要6个时钟周期，后面3个数据各需要3个时钟周期



— 3、EDO-DRAM

- EDO-DRAM: Extend Data Out DRAM, **扩展数据输出DRAM**
- 在快速页式工作技术上,增加了**数据输出**部分的数据锁存线路,延长输出数据的有效保持时间,从而地址信号改变了,仍然能取得正确的读出数据,可以进一步缩短地址送入时间,更加快了主存储器的读写速度。
- EDO-DRAM的原理与FPM-DRAM相似,区别在于EDO-DRAM的数据阶段和列地址阶段是重叠的(如数据1阶段和列地址2阶段是重叠的),从而可以进一步提高存储器的访问速度
- 访问5个数据的EDO-DRAM的典型时序是: **5-2-2-2-2**,即第1个数据需要5个时钟周期,后面4个数据各需要2个时钟周期

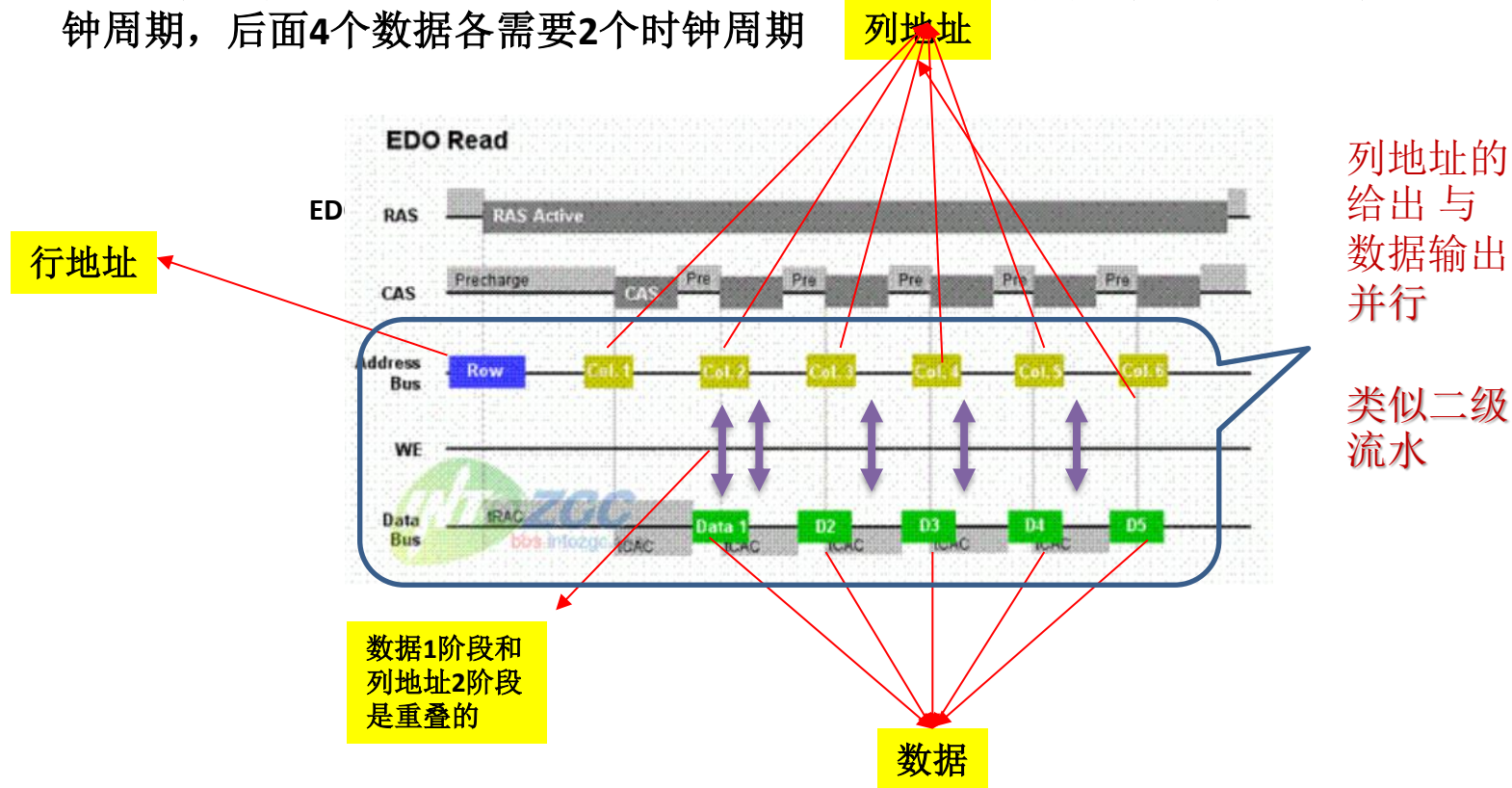


图4.21 EDO-DRAM读操作时序图

— 4、BEDO-DRAM

- BEDO-DRAM: Burst Extend Data Out DRAM, **突发式扩展数据输出DRAM**;
BEDO-DRAM是对EDO-DRAM的进一步优化, BEDO-DRAM内置了列地址计数器
- 这样连续访问的列地址(例如要读取同一行连续几个列的数据)不由外部数据总线给出, 而由内部**列地址计数器**自动给出, 节省了外部总线传输列地址的时间, 从而进一步提高访问存储器的速度
- 访问5个数据的BEDO-DRAM的典型时序是: **5-1-1-1-1**, 即第1个数据需要5个时钟周期, 后面4个数据各需要1个时钟周期

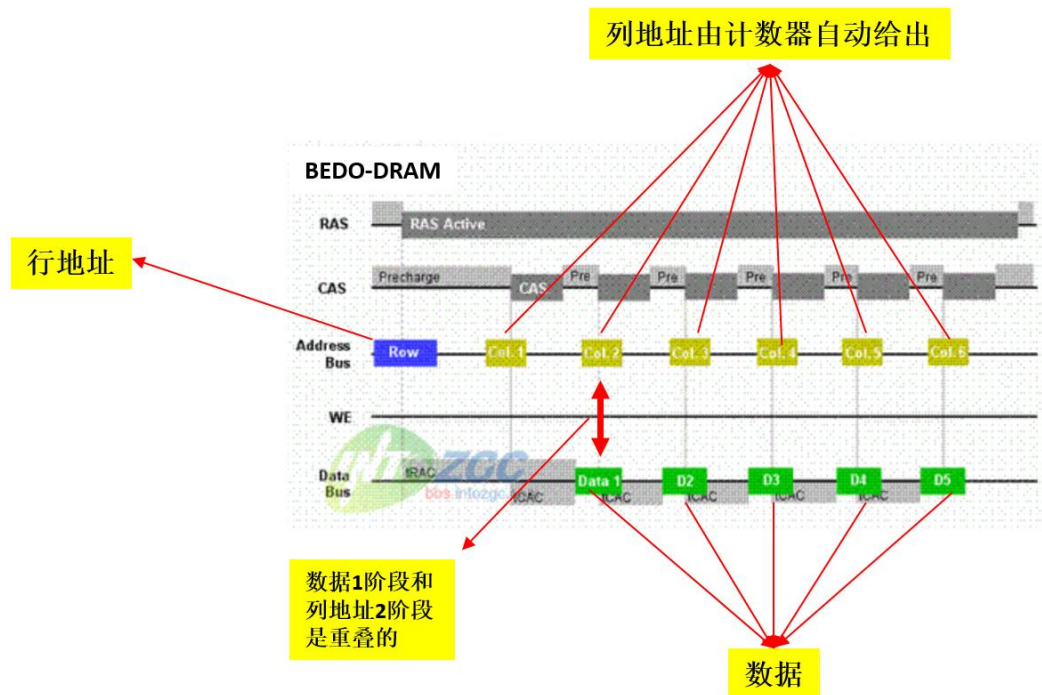


图4.22 BEDO-DRAM读操作时序图

– 5、同步DRAM（SDRAM）

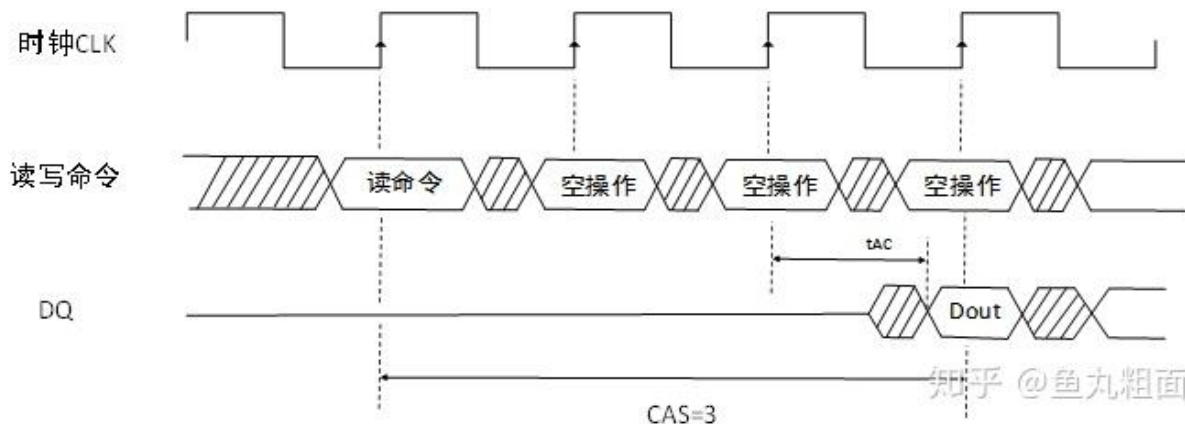
- SDRAM: Synchronous DRAM, **同步DRAM**
- **SDRAM_百度百科** <https://baike.baidu.com/item/SDRAM>
- FPM-DRAM、EDO-DRAM、BEDO-DRAM都属于异步DRAM；异步DRAM芯片本身没有时钟信号，其时序控制、地址寻址、数据访问都是由**内存控制器**通过异步方式控制实现的
- 而SDRAM芯片**自带时钟控制信号**，能与系统总线频率同步，第1个列数据就绪后，每经历1个时钟周期，就可以得到1个数据，从而能够进一步提高访问存储器的速度
- 目前主流的SDRAM访问一次数据的时间在10ns左右，其工作频率为1/10ns，即100MHz



5、同步DRAM（SDRAM）

– SDRAM的读操作

- **行激活**：在向SDRAM发送读或写命令之前必须打开该Bank中的一行，通过ACTIVE命令来确定要激活的Bank和行。要想对一个L-Bank中的阵列进行寻址，首先要确定行（Row），然后确定列。片选信号与L-Bank选择信号与行有效同时进行。
- **列读写**：当列地址选通后，就需要对SDRAM进行读写，而给SDRAM读命令还是写这就由WE#信号来决定。当WE#信号拉低时，SDRAM接收到的是写命令；当WE#拉高，SDRAM接收读命令。列寻址信号与读写命令是同时发出的。在发送列读写命令时必须与行有效命令有一个时间间隔，这个时间间隔被定义为**tRCD**，即RAS to CAS Delay（RAS至CAS延迟）。比如tRCD=3，就代表延迟周期为3个时钟周期，即在行有效命令发出后，经过3个时钟周期的延时，才能发送列读写命令，SDRAM进入数据读写操作。
- **读操作**：从CAS与读取命令发出到第一个数据输出的这段时间，被定义为CL（CAS Latency，CAS潜伏期）



参：SDRAM读写时序

— 6、DDR SDRAM

- **DDR工作原理 - 知乎** <https://zhuanlan.zhihu.com/p/576522124>
- **DDR SDRAM: Double Data Rate, 双倍数据速率同步DRAM**

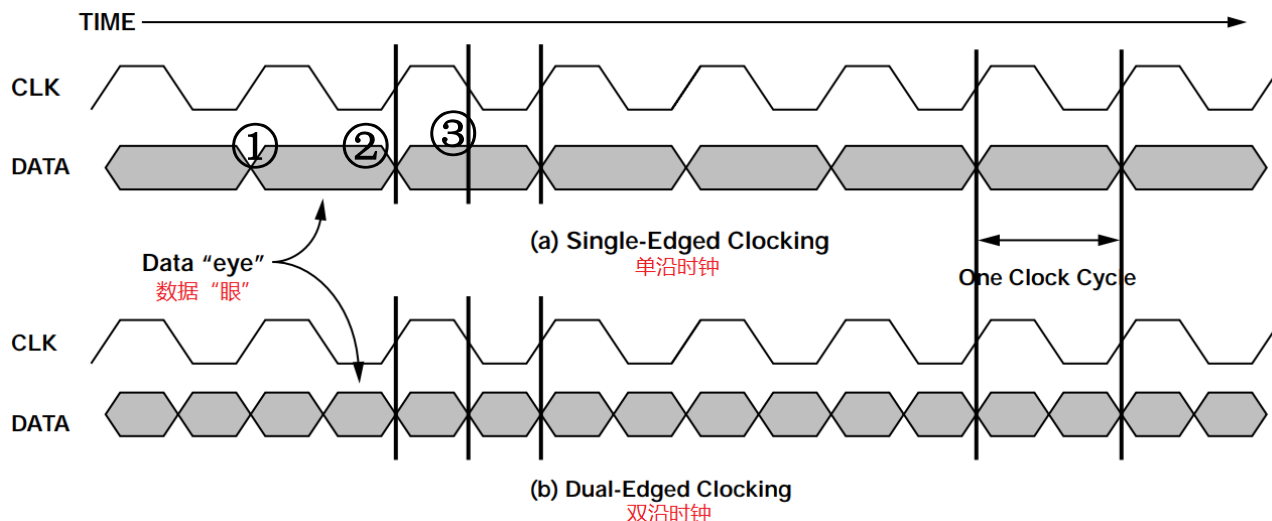
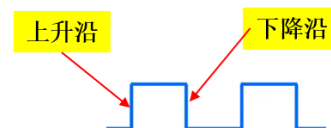


FIGURE 7.20: Running the bus clock at the data rate. The top diagram (a) illustrates a single-edged clocking scheme wherein the clock is twice the frequency of the data transmission. The bottom diagram (b) illustrates a dual-edged clocking scheme in which the data transmission rate is equal to the clock frequency.

图7.20：总线的时钟以数据率频率运行。上图(a)展示了单沿时钟方案，其中时钟是数据传输频率的两倍。上图(b)展示了双沿时钟方案，其中时钟和数据传输频率相等。

在一个使用单沿时钟发送数据的系统中，每个数据眼图会对应两个时钟沿，数据眼图的边缘由两个时钟的边沿确定，然后第三个时钟沿会位于数据传输眼图的中间(如图7.20a)。因此时钟信号有两个直接的功能：将数据驱动到总线上以及在总线上读取数据。注意到在单沿时钟的方案中，数据每个时钟周期只传输一次。

在双沿方案中，数据要不然是边沿对齐的，要不然是中心对齐的(如图7.20b)。这意味着时钟可以驱动数据到总线上，或者也可以从总线上读取数据，但是无法像单沿时钟方案中一样，只用一个时钟实现这两个功能。如图所示的时钟是边沿对齐的。

4.2.4 DRAM的发展

— DDR SDRAM

- **DDR SDRAM**内部采用**2路**预取技术，能够在时钟周期的**上升沿和下降沿**分别进行数据传输，即每个时钟周期可以传输**2个**数据，其速度是**SDRAM的2倍**（**SDRAM**每个时钟周期传输**1个**数据）
- **DDR2 SDRAM**：采用**4路**预取技术，每个时钟周期可以传输**4个**数据；**DDR2**数据总线的频率是**DRAM**工作频率的**2倍**
- **DDR3 SDRAM**：采用**8路**预取技术，每个时钟周期可以传输**8个**数据；**DDR3**数据总线的频率是**DRAM**工作频率的**4倍**
- **DDR4 SDRAM**：采用**16路**预取技术，每个时钟周期可以传输**16个**数据；**DDR4**数据总线的频率是**DRAM**工作频率的**8倍**

表4.2 不同DDR内存的常见参数

内存规格	传输率	总线频率	DRAM工作频率
DDR2-800	6400MB/s	400MHz	200MHz
DDR3-800	6400MB/s	400MHz	100MHz
DDR3-1600	12.8GB/s	800MHz	200MHz
DDR4-2400	19.2GB/s	1200MHz	150MHz
DDR4-3200	25.6GB/s	1600MHz	200MHz

书上这里有误

DDR4-2400

其等效数据传输频率=2400MHz

数据位宽=8字节=8B

传输率（内存带宽）=2400MHzx8B=19.2GB/s

总线频率=等效传输频率/2=1200MHz

DRAM工作频率=等效传输频率/8=2400/8=300MHz

8:8倍速

DDR4 SDRAM



金士顿 (Kingston) FURY 16GB DDR4 3200 台式机内存条 Beast野兽系列 骇客神条

【新品DDR5内存，高速出击！】新品晒单抽20元E卡，0元抽新品内存，众多好礼等你来赢！[请点击查看>](#)

京东价 **¥ 579.00** 降价通知

促销 **满送** 满4000元即赠热销商品，赠完即止

增值业务 **高价回收，极速到账**

配送至 **福建厦门市思明区城区** **有货** 支持 **可配送海外** | 99元免基础运费 | 180天只换不修

京东物流 京准达 | 预约送货 | 部分收货 **✓**

由 **京东** 发货，供应商提供售后服务。23:10前下单，预计**明天(03月17日)**送达

DDR4-3200 16GB

品牌：金士顿 (Kingston)

商品名称：金士顿HX432C18FB/16-SP

总容量：16GB

外观特征：马甲条

商品编号：100005089420

平台：台式机内存

内存频率：3200/3333MHz

商品毛重：50.00g

内存数量：1条单条

商品产地：中国大陆，中国台湾

DDR代数：DDR4

DDR4-3200

其等效传输频率=3200MHz

数据位宽=8字节=8B

传输率（内存带宽）=3200MHzx8B=25.6MB/s

总线频率=等效传输频率/2=1600MHz

DRAM工作频率=等效传输频率/8=3200/8=400MHz

DDR5 SDARM



金士顿 (Kingston) FURY 16GB DDR5 5200 台式机内存条 Beast野兽系列 骇客神条

新世代DDR5全新内存, 全面支持intel-12代CPU, 支持XMP3.0规范, PNP自动超频, 强悍性能! 全新体验! 【同款4800选购】[查看>](#)

京东秒杀

距离结束 13 : 26 : 13

秒杀价 ¥ 1199.00 降价通知

累计评价
1000+

增值业务 [高价回收, 极速到账](#)

配 送 至 [福建厦门市湖里区金山街道](#) 有货 支持 [可配送海外](#) | [99元免基础运费](#) | [180天只换不修](#)

[京东物流](#) 次日达 | [预约送货](#) | [部分收货](#) [V](#)

由 [京东](#) 发货, 供应商提供售后服务. 18:00前下单, 预计明天(03月11日)送达

重 量 0.06kg

DDR5-5200 16GB

品牌: [金士顿 \(Kingston\)](#)

商品名称: 金士顿KF552C40BB-16

总容量: 16GB

外观特征: 马甲条

商品编号: 100015306105

平台: 台式机内存

内存频率: 5200MHz

商品毛重: 60.00g

内存数量: 1条单条

商品产地: 中国大陆, 中国台湾

DDR代数: DDR5

DDR5-5200

其等效传输频率=5200MHz

数据位宽=8字节=8B

传输率(内存带宽)=5200MHzx8B=41.6MB/s

总线频率=等效传输频率/2=2600MHz

DRAM工作频率=等效传输频率/8=5200/8=650MHz

支持16路预取, 8倍速

补充：

现代化的**DRAM**技术，更像是一门**接口技术**，而不像是存储技术。得益于摩尔定律，随着工艺尺寸的不断缩进，**DRAM**的容量越做越大，这个并不难，但是如果没有**DDR1**，**DDR2**和**DDR3**，**DDR4**等不断的倍频高速接口出现，**DRAM**的速度基本没有办法提高。

其原因在于，**DRAM**作为一个大的存储体，尤其是靠电容作为存储的手段，其内部操作速度很难做到很高。电容的容量做得太小，内存读写的确可以变快，后果是漏电也会很快，需要不停的刷新防止数据丢失，内存的性能会受到影响。电容的容量做得太大，数据确实可以保存的很久，但是写入和读取的延时就会大大增加，**DRAM**的性能又会大大降低。

基于此，**DRAM**的内部存储操作速度难以大幅度改进。举个例子：即使**DRAM**接口**pin**做到了**1066M**，**DRAM**的存储单元其实并没有工作在**1066M**这么高的时钟下面，相反只有**133M**。快的只是接口而已。就好像一个很粗的水管，里面的水流速其实并不快，但是只要我们把出口收窄，出口的水速就会大幅提高，远远高于水管内的水速。

在右图给出的示例中，SDRAM的接口速度是133M，DDR1接口的速度是266M，DDR2接口的速度是533M，DDR3的接口速度是1066M。每提高一代，接口性能都提高了一倍，但是注意看，这四代接口的核心频率都是133M，也就是说，DRAM存储单元的读写速度没有提高

核心技术：

双沿传输
预取

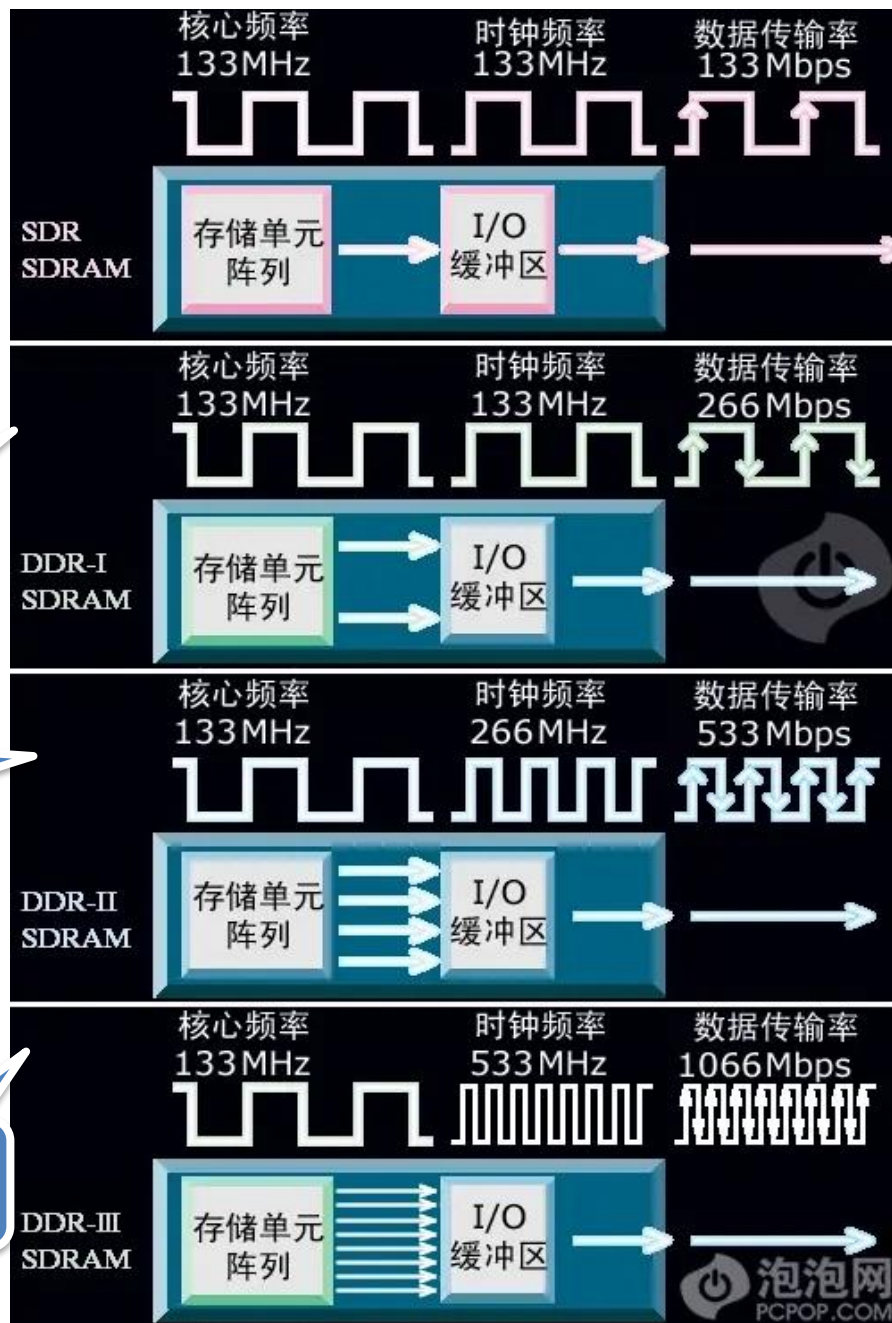
增加DRAM内部BUS宽度



二路预取

四路预取

八路预取



DRAM v.s SRAM

Static Random Access Memory

Dynamic Random Access Memory

类 型 特 点	SRAM（静态RAM）	DRAM（动态RAM）
存储信息	触发器	电容
破坏性读出	非	是
读出后需要重写？（再生）	不用	需要
运行速度	快	慢
集成度	低	高
发热量	大	小
存储成本	高	低
易失/非易失性存储器？	易失（断电后信息消失）	易失（断电后信息消失）
需要“刷新”？	不需要	需要
送行列地址	同时送	分两次送

cache

主存

参考资料

- 相变存储器 - 知乎<https://zhuanlan.zhihu.com/p/631577556>

PCM技术优势

- 1. **快速读写**: 相比传统的硬盘和闪存等存储器, **PCM**具有远高于闪存的读写速度, 这对于那些需要大量数据的任务来说是非常必要的。闪存的写入速度受限于电荷移动速度, 而**PCM**通过相变物质的固态、液态转变来保存数据, 并且能够实时切换状态。
 - 2. **非易失性**: 相变材料由于非易失性, 即使忘掉存储器是非常常见的, 相变储存器仍然会保留信息而不会遗失。这种特性使得**PCM**非常适合一些任务, 比如需要存储一些重要的数据备份。
 - 3. **高密度存储**: **PCM**可以在更小的空间内保存相对更大的数据, 并且其芯片大小比闪存芯片更小, 自然降低了制造成本, 从而为更大数据容量做出了贡献。
 - 4. **低功耗**: 当存储器不需要进行读取或写入操作时, 会使功率降至最小化。这种省电功能使得**PCM**能够在电池供电的设备上表现得更加优异。
- 破解内存墙, 除了“存算一体”还需要什么? - 知乎

4.3 主存的组织及与CPU 的连接

• 4.3.1 存储器与CPU的连接

- 单片存储芯片的容量有限，通常需要将多片存储芯片按照一定的方式组织起来，并与CPU连接，从而获得一个大容量的存储器
- 存储芯片与CPU的地址线、数据线及控制线连接时需要注意的问题：
 - ① 连接的地址线的数量与CPU要访问的主存容量有关
 - ② 连接的数据线的数量与计算机的字长有关
 - ③ SRAM芯片的控制线包括片选信号（/CS）和读写控制线（/WE）
 - ④ ROM芯片的控制线只有片选信号（/CS），没有读写控制线（/WE）
 - ⑤ DRAM芯片没有片选信号（/CS），进行容量扩展时，可以利用/RAS和/CAS信号控制芯片的选择

4.3.2 存储器的扩展

- 包括位扩展、字扩展、字位同时扩展
- 1、位扩展
 - 位扩展又称**字长扩展或数据总线扩展**；例如，可以利用32个256Kx1位的SRAM芯片，构成1个256Kx32位的存储器，其与CPU的连接如图4.24所示
 - 地址线：18根（ $2^{18}=256K$ ），同时连接到每个SRAM芯片的地址线A（18根）
 - 数据线：32根，分别连接到每个SRAM芯片的数据线D（1根）
 - 控制线：MREQ#（相当于/MREQ）连接到每个SRAM芯片的CS端（片选信号）；R/W#（相当于-R/W）连接到每个SRAM芯片的WE端（读写控制信号）

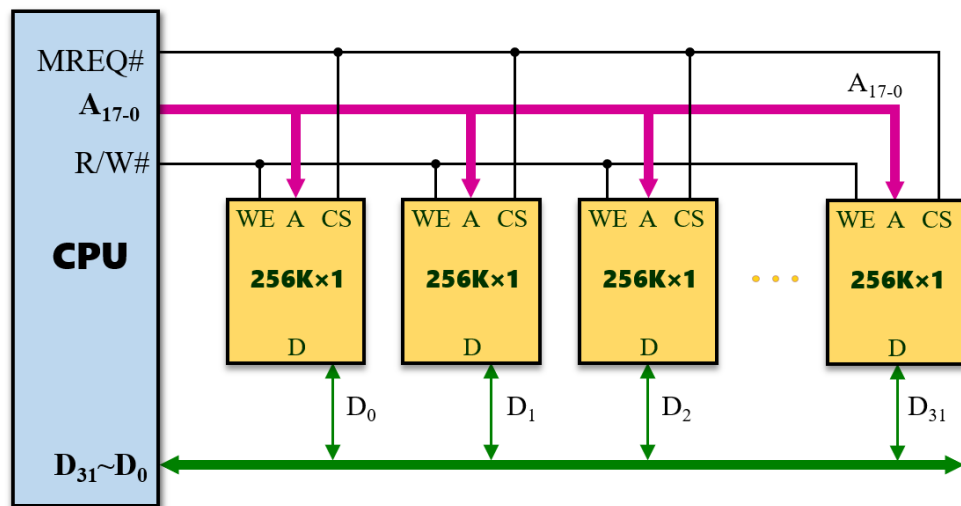


图4.24 存储器的位扩展

4.3.2 存储器的扩展

– 2、字扩展

- 字扩展又称**容量扩展或地址总线扩展**；例如，可以利用8个256Kx8位的SRAM芯片，构成1个2Mx8位的存储器，其与CPU的连接如图4.25所示
- 地址线：21根（ $2^{21}=2\text{M}$ ），其高位部分（ $A_{20} \sim A_{18}$ ）连接3-8译码器的输入，低位部分（ $A_{17} \sim A_0$ ）同时连接到每个SRAM芯片的地址线A（18根）
- 数据线：8根，同时连接到每个SRAM芯片的数据线D（8根）
- 控制线：MREQ#连接到3-8译码器的OE端；R/W#连接到每个存储器芯片的WE端
- 3-8译码器的输入： $A_{20} \sim A_{18}$ 和MREQ#
- 3-8译码器的输出： $Y_0 \sim Y_7$ ，分别连接每个SRAM的CS端

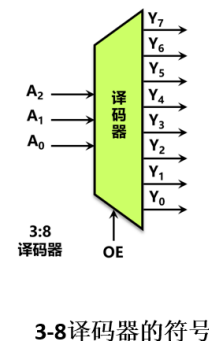
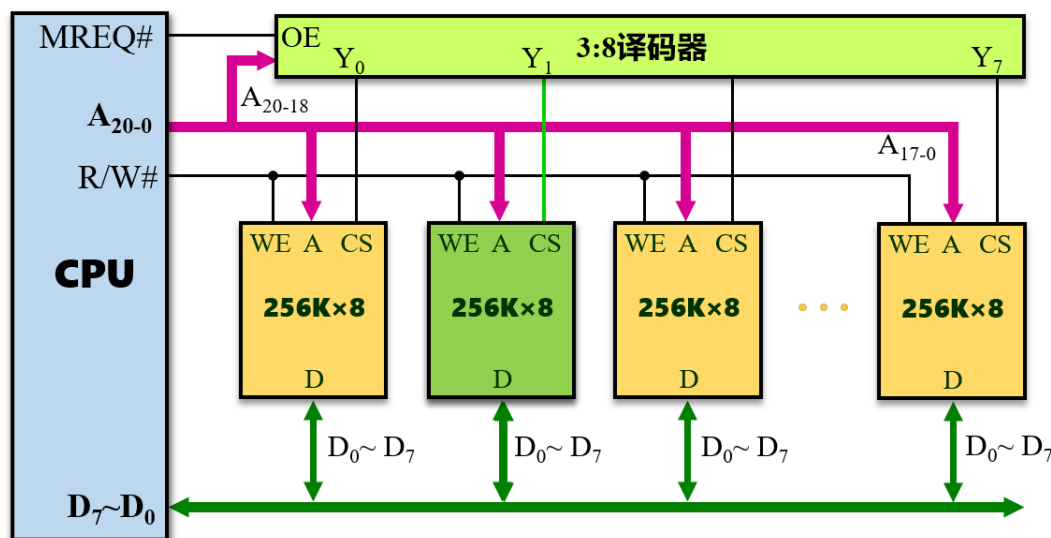


图4.25 存储器的字扩展

4.3.2 存储器的扩展

- 字扩展时各个SRAM芯片的地址范围：
 - 地址总线= $A_{20} \sim A_0$ (21根)
 - 地址范围= $0\ 0000\ 0000\ 0000\ 0000\ 0000 \sim 1\ 1111\ 1111\ 1111\ 1111\ 1111 = 000000H \sim 1FFFFFFH$
 - 第1个SRAM芯片的地址范围= $000000H \sim 03FFFFH$; 容量为256K
 - 第2个SRAM芯片的地址范围= $040000H \sim 07FFFFH$; 容量为256K
 - 第3个SRAM芯片的地址范围= $080000H \sim 0BFFFFH$; 容量为256K
 - 第4个SRAM芯片的地址范围= $0C0000H \sim 0FFFFFH$; 容量为256K
 - 第5个SRAM芯片的地址范围= $100000H \sim 13FFFFH$; 容量为256K
 - 第6个SRAM芯片的地址范围= $140000H \sim 17FFFFH$; 容量为256K
 - 第7个SRAM芯片的地址范围= $180000H \sim 1BFFFFH$; 容量为256K
 - 第8个SRAM芯片的地址范围= $1C0000H \sim 1FFFFFFH$; 容量为256K

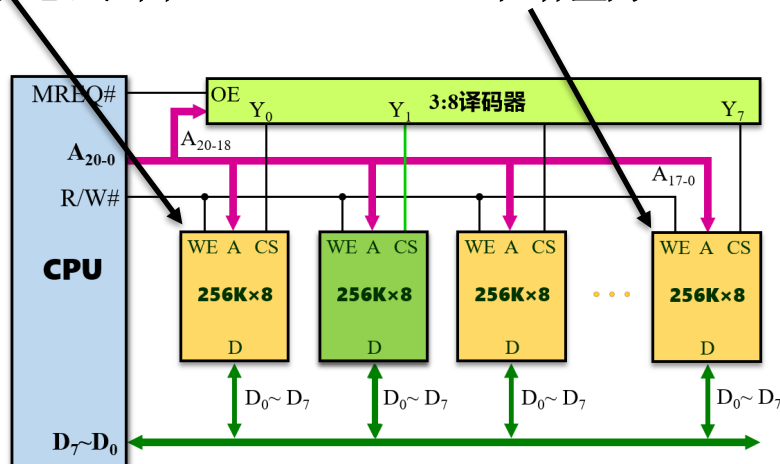


图4.25 存储器的字扩展

4.3.2 存储器的扩展

– 3、字位同时扩展

- 同时进行字扩展和位扩展；例如，可以利用32个256Kx8位的SRAM芯片，构成1个2Mx32位的存储器，其与CPU的连接如图4.26所示（ $32=(2M \times 32)/(256 \times 8)$ ）
- 地址线：21根（ $2^{21}=2M$ ），其高位部分（ $A_{20} \sim A_{18}$ ）连接到3-8译码器的输入，低位部分（ $A_{17} \sim A_0$ ）同时连接到每个SRAM芯片的地址线A（18根）
- 数据线：32根，分为4组（ $D_{31} \sim D_{24}$ ； $D_{23} \sim D_{16}$ ； $D_{15} \sim D_8$ ； $D_7 \sim D_0$ ），分别连接到各个SRAM芯片的数据线D（8根）
- 控制线：MREQ#连接到3-8译码器的OE端；R/W#连接到每个SRAM芯片的WE端
- 3-8译码器的输入： $A_{20} \sim A_{18}$ 和MREQ#
- 3-8译码器的输出： $Y_0 \sim Y_7$ ，分别连接每个SRAM的CS端

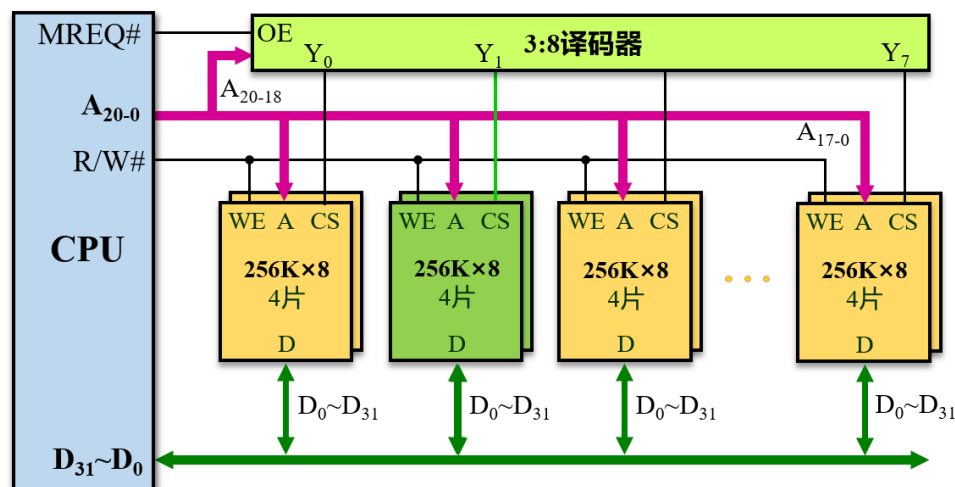


图4.26 存储器的字位扩展

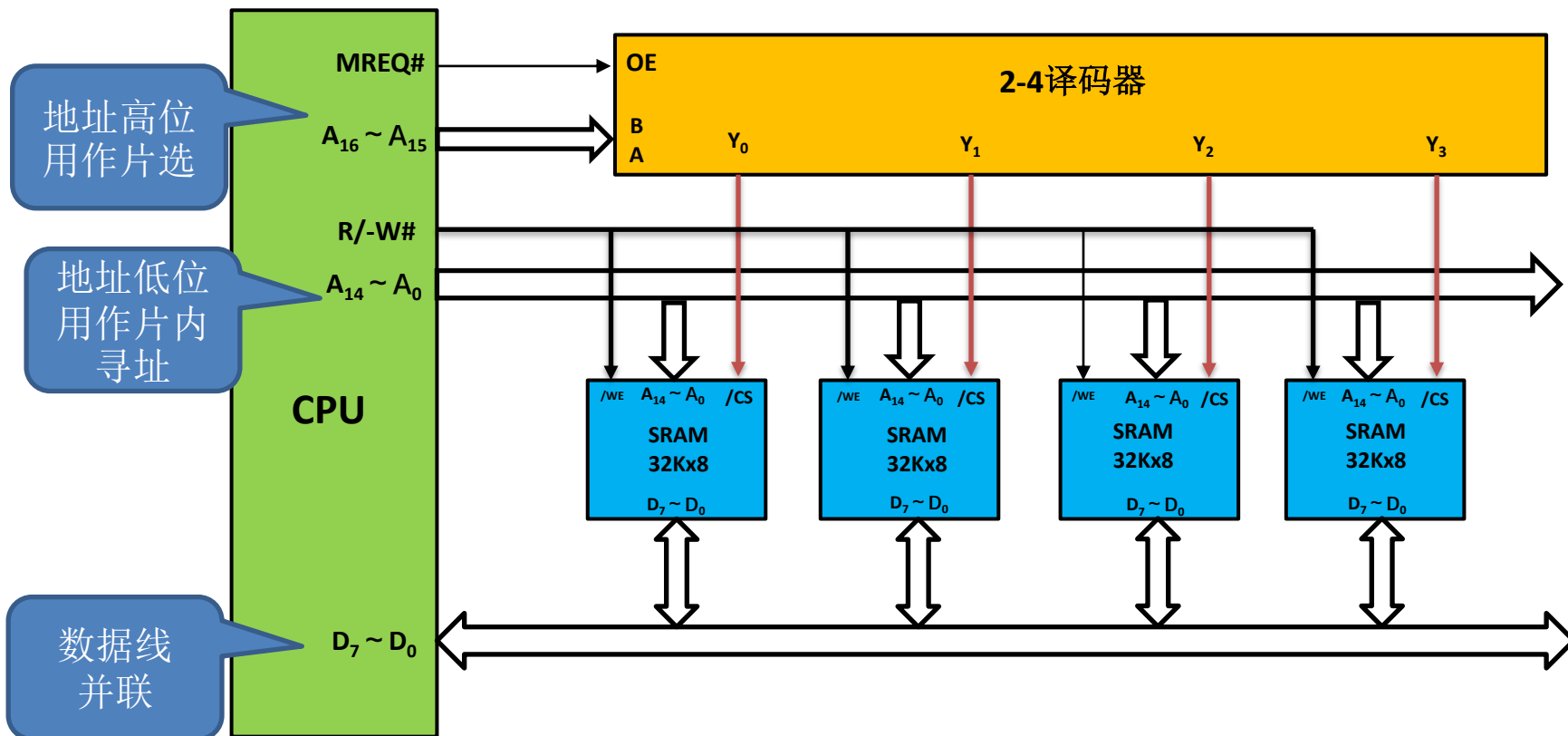
4.3.2 存储器的扩展

- **（补充）习题4.5** 用4个32Kx8位SRAM存储芯片，可设计出哪几种不同容量和字长的存储器？画出相应设计图，并完成与CPU的连接。
- 解：有三种：
 - **（1）字扩展：** 32K -> 128K
 - 4片32Kx8位 -> 128Kx8位
 - **（2）位扩展：** 8位 -> 32位
 - 4片32Kx8位 -> 32Kx32位
 - **（3）字位扩展：** 32K -> 64K， 8位 -> 16位
 - 4片32Kx8位 -> 64Kx16位

(1) 字扩展: 32K -> 128K

4片 32Kx8位 -> 128Kx8位

片内地址线, 数据线, 读写线并联, CS片选端利用译码器单独设计



32Kx8位

地址线=15根 ($A_{14} \sim A_0$)

数据线=8根 ($D_7 \sim D_0$)

128Kx8位

地址线=17根 ($A_{16} \sim A_0$)

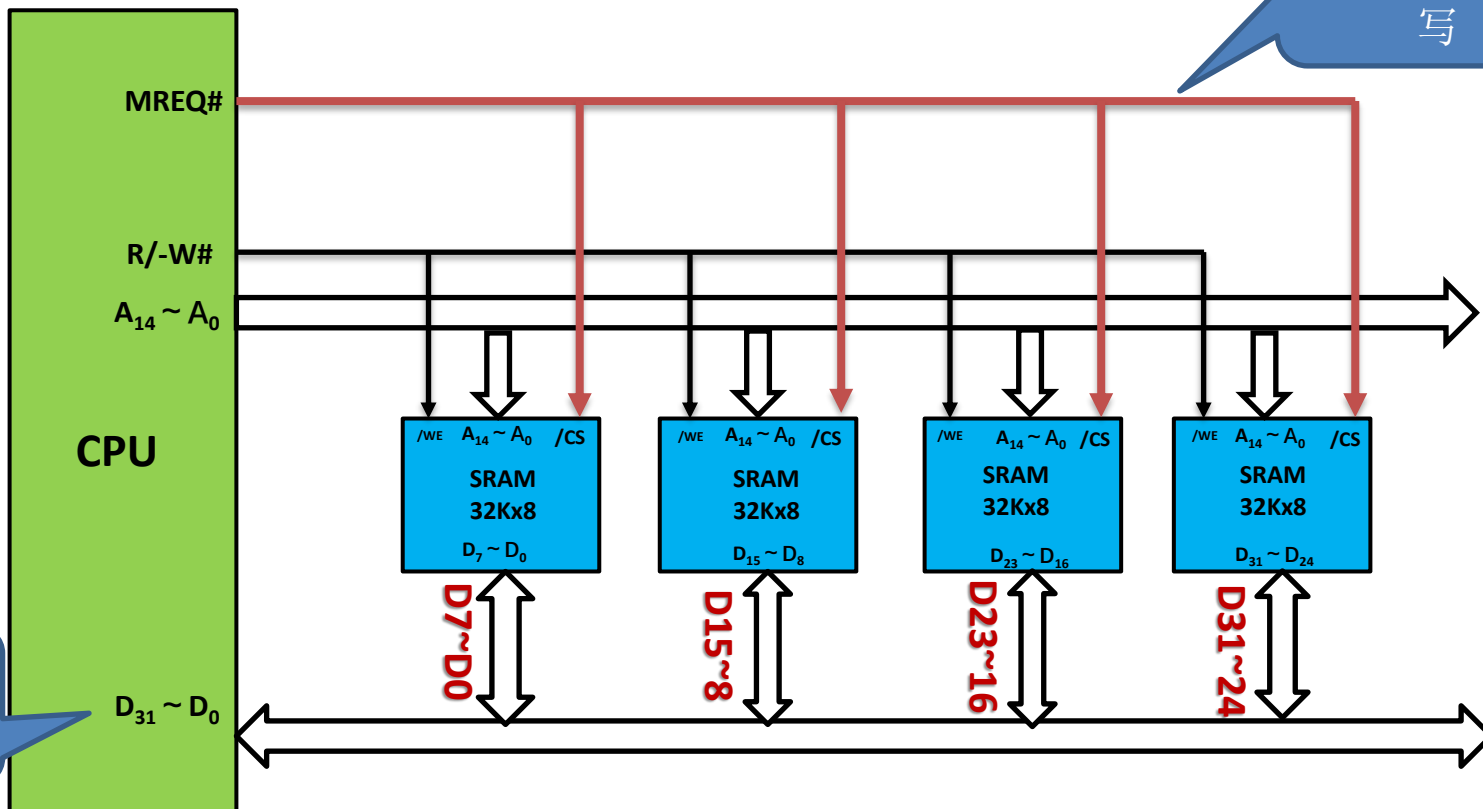
数据线=8根 ($D_7 \sim D_0$)

(2) 位扩展: 8位 -> 32位

4片32Kx8位 -> 32Kx32位

片内地址线, 读写线
并联, CS片选并联,
数据线单独设计

片选同时生效,
4片同时读
写



数据线
分成4组

32Kx8位

地址线=15根 (A₁₄ ~ A₀)

数据线=8根 (D₇ ~ D₀)

32Kx32位

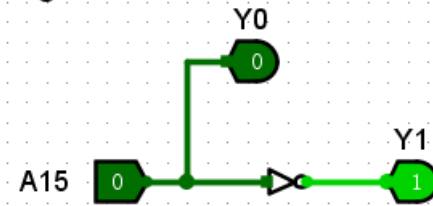
地址线=15根 (A₁₄ ~ A₀)

数据线=32根 (D₃₁ ~ D₀)

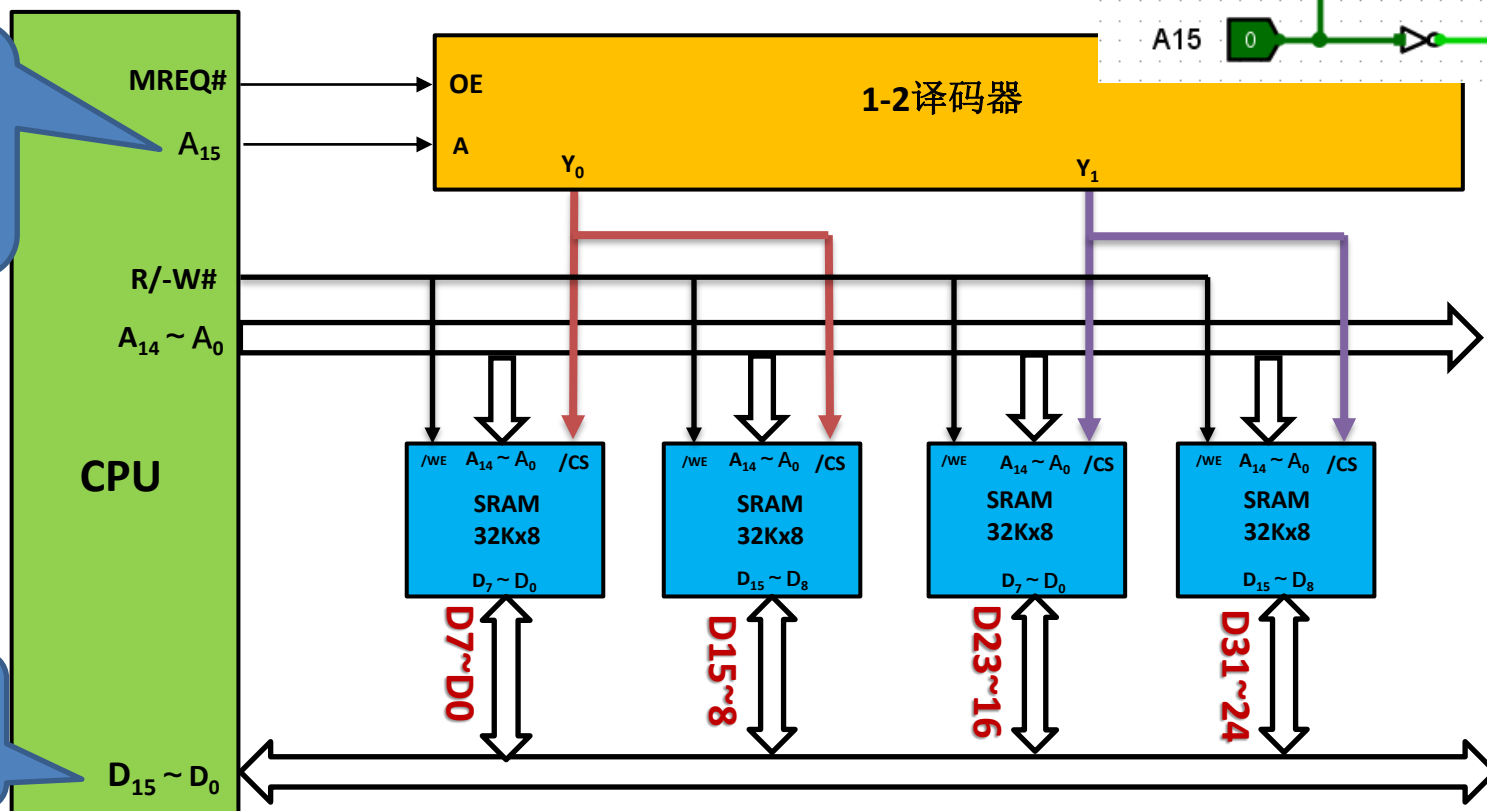
(3) 字位扩展: 32K -> 64K, 8位 -> 16位

4片32Kx8位 -> 64Kx16位

Logisim



A15=0, 选中前2片
A15=1, 选中后2片



数据线
分成4组

32Kx8位

地址线=15根 ($A_{14} \sim A_0$)

数据线=8根 ($D_7 \sim D_0$)

64Kx16位

地址线=16根 ($A_{15} \sim A_0$)

数据线=16根 ($D_{15} \sim D_0$)

- (补充) 习题4.7 某计算机字长为16位，请用16Kx8位SRAM芯片和32Kx16位ROM芯片，为该计算机设计一个主存储器，其容量为128Kx16位。要求18000H到1FFFFH为ROM区，其余为RAM区。画出存储器与CPU的连接。

$$1FFFFH - 18000H + 1 = 8000H = 1000000000000000B = 2^{16} = 32K$$

解：

- 18000H到1FFFFH为ROM区；因此，ROM区的容量=32Kx16位，需要1片32Kx16位ROM芯片
- RAM区的容量=128Kx16位-32Kx16位=96Kx16位，需要 $(96K \times 16) / (16K \times 8) = 12$ 片16Kx8位RAM芯片
- RAM采用字位同时扩展：16K → 96K（字扩展，6组片选），8位 → 16位（位扩展，2片1组）
- 32Kx16位的ROM芯片，地址线=15根（ $A_{14} \sim A_0$ ），数据线=16根（ $D_{15} \sim D_0$ ）
- 16Kx8位的RAM芯片，地址线=14根（ $A_{13} \sim A_0$ ），数据线=8根（ $D_7 \sim D_0$ ）
- 128Kx16位的地址空间为：00000H ~ 1FFFFH，其中00000H ~ 17FFFH为RAM空间，18000H ~ 1FFFFH为ROM空间

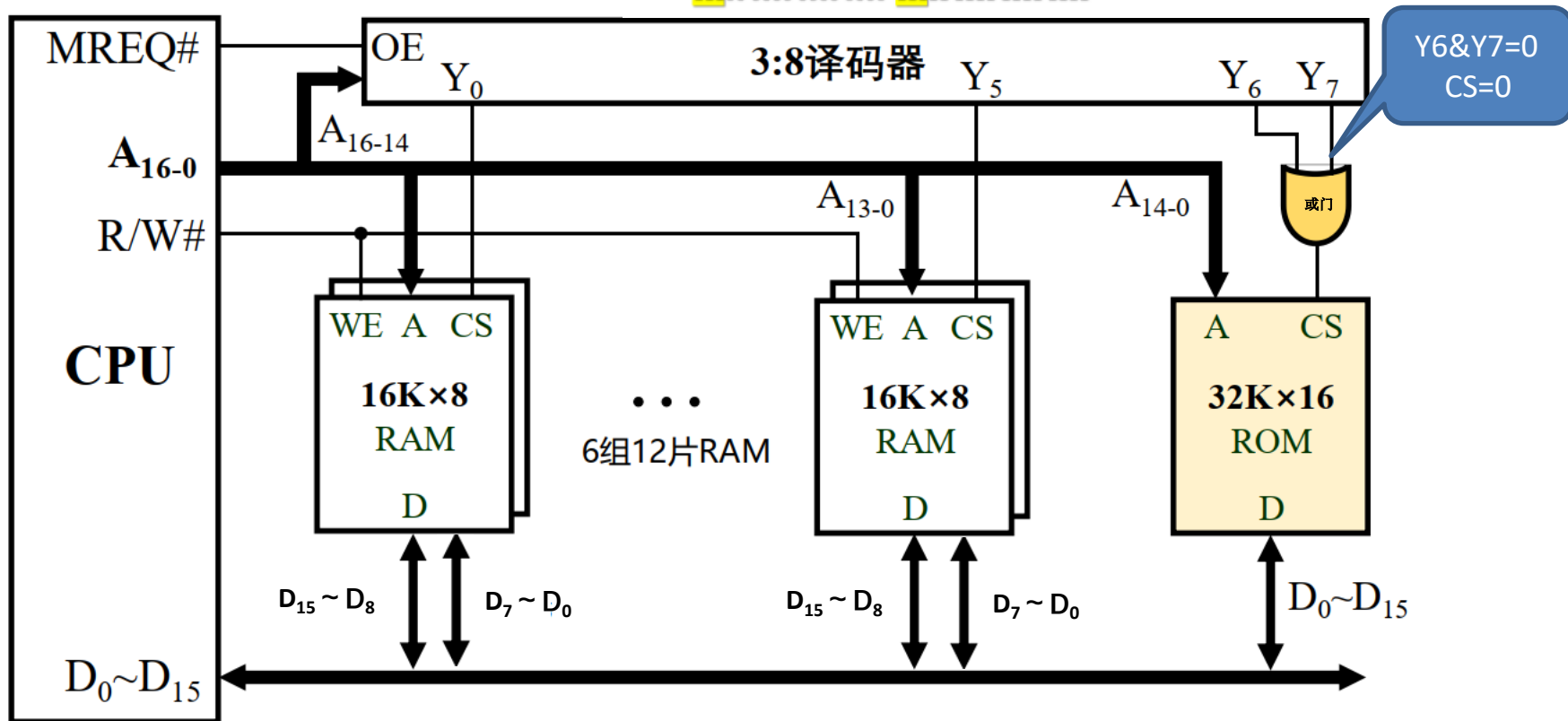
- 0000 0000 0000 0000 ~ 0011 1111 1111 1111 接第1、2片
- 0010 0000 0000 0000 ~ 0011 1111 1111 1111 接第3、4片RAM（第2组）
- 0100 0000 0000 0000 ~ 0111 1111 1111 1111 接第5、6片RAM（第3组）
- 0110 0000 0000 0000 ~ 0111 1111 1111 1111 接第7、8片RAM（第4组）
- 1000 0000 0000 0000 ~ 1011 1111 1111 1111 接第9、10片RAM（第5组）
- 1010 0000 0000 0000 ~ 1011 1111 1111 1111 接第11、12片RAM（第6组）
- 1100 0000 0000 0000 ~ 1111 1111 1111 1111 接1片ROM芯片（32K）

$$96K = 3 \times 2^{15} = 001100000000000000000000 = 18000H - 1 = 17FFFH$$

- 1100 0000 0000 0000 ~ 1101 1111 1111 1111
- 1110 0000 0000 0000 ~ 1111 1111 1111 1111

- 选用1个3-8译码器，将17根地址线的高3位（ $A_{16} \sim A_{14}$ ）接3-8译码器的输入（另一个输入为MREQ#）；译码器的输出为： $Y_7 \sim Y_0$
- 其中 $Y_5 \sim Y_0$ 分别接6组RAM芯片的片选， $Y_7 \sim Y_6$ 通过一个或门接ROM芯片的片选
- 17根地址线的低14位接12个RAM芯片，低15位接1个ROM芯片；16位数据线的高8位接6个RAM芯片、低8位接另外6个RAM，16位数据线直接接1个ROM芯片

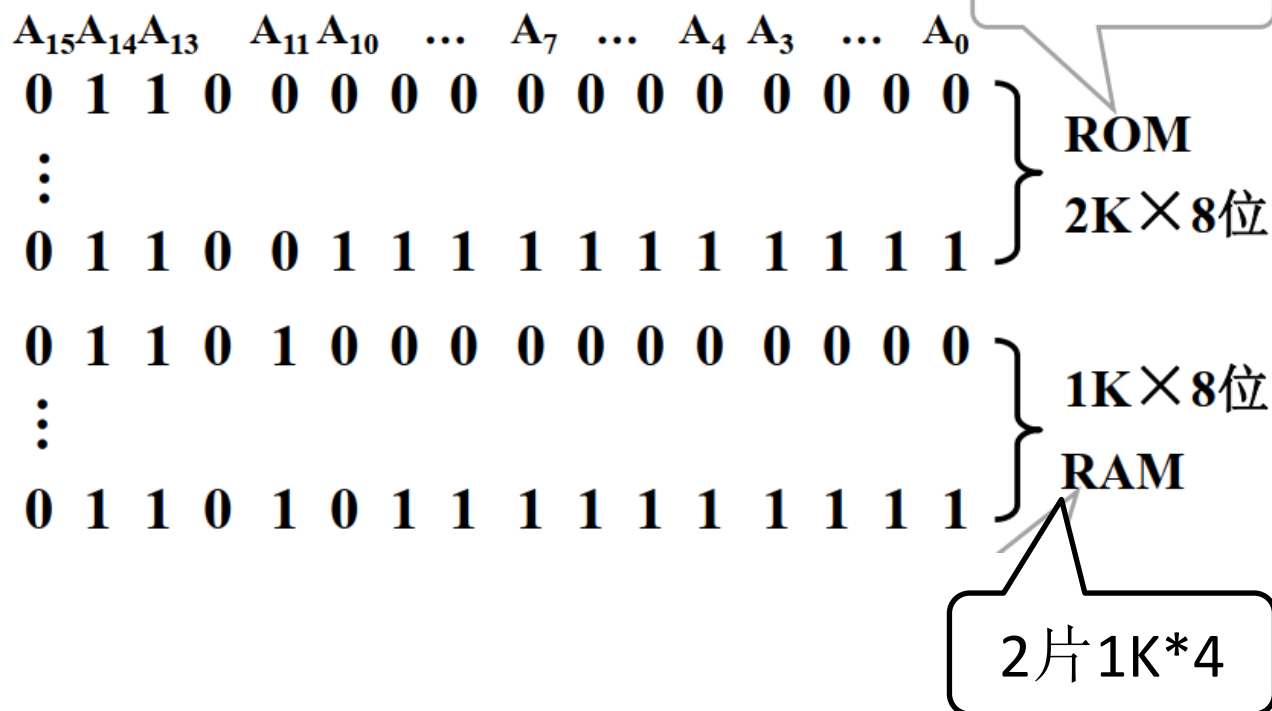
1 1000 0000 0000 0000 ~ 1 1111 1111 1111 1111 接1片ROM芯片（32K）
 • 11000 0000 0000 0000~11011 1111 1111 1111
 • 11100 0000 0000 0000~11111 1111 1111 1111



- 补充例题：CPU有16根地址线，8根数据线，用/MREQ作为访存控制信号，/WR读写控制信号。有下述芯片：1K × 4RAM, 4K × 8RAM, 8K × 8RAM, 2K × 8ROM, 4K × 8ROM，8K × 8ROM，请画出CPU与存储器的连接图，满足下述要求：
- 主存地址空间：6000H~67FFH系统程序区；6800H~6BFFH用户程序区

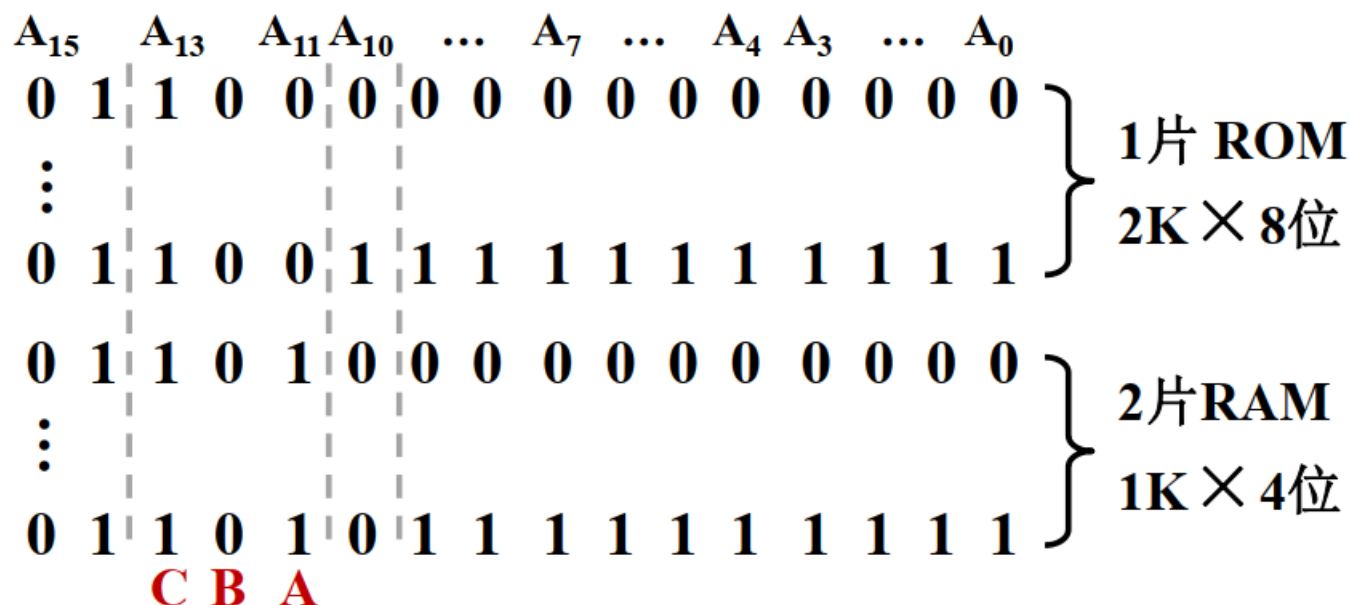
例4.1 解：

(1) 写出对应的二进制地址码



(3) 分配地址线

4.2



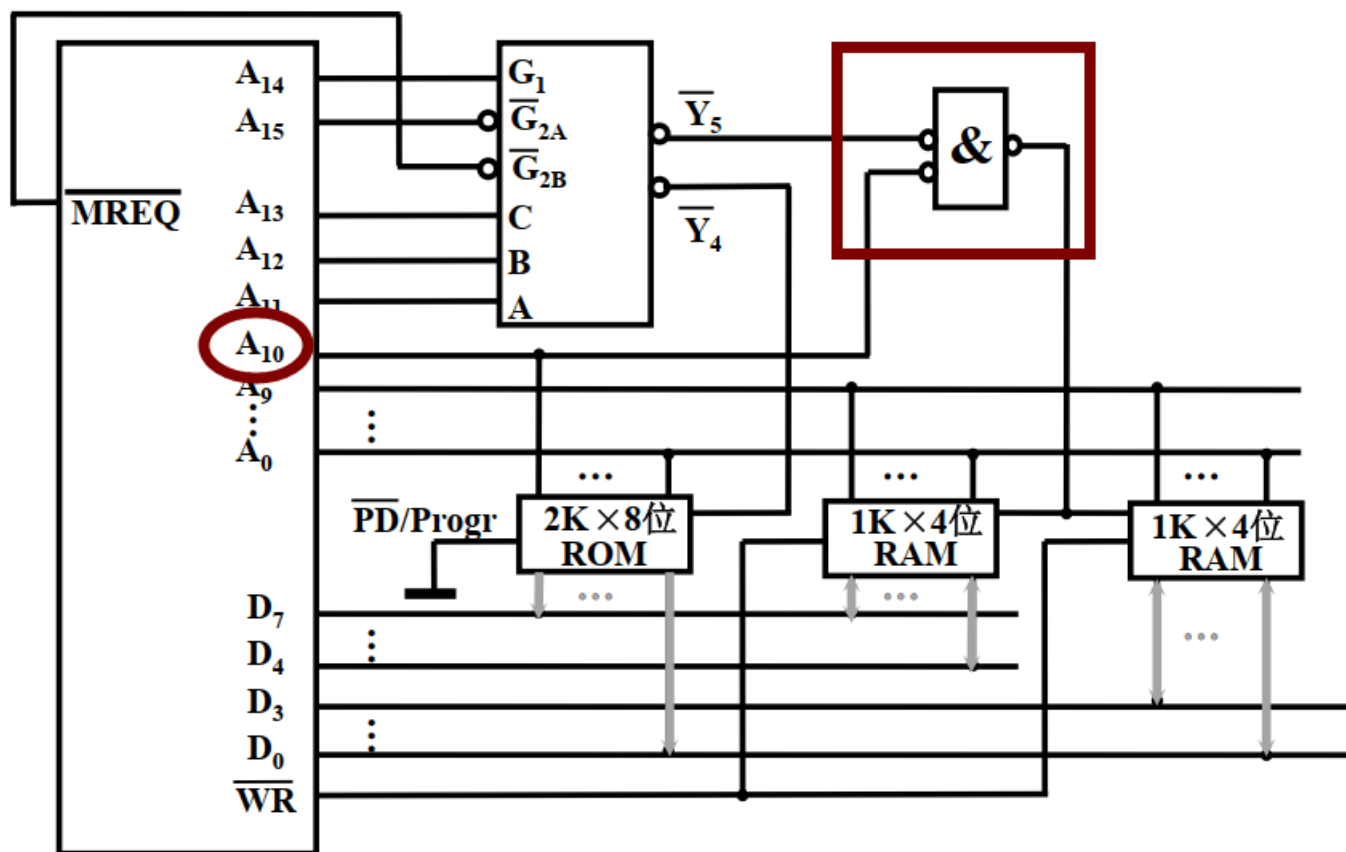
C B A

$A_{10} \sim A_0$ 接 2K × 8位 ROM 的地址线

$A_9 \sim A_0$ 接 1K × 4位 RAM 的地址线

例 4.1 CPU 与存储器的连接图

4.2



4.4 并行主存系统

- 通常CPU的速度很快，相比之下主存的速度不够快，因此主存的存取速度已经成为提升计算机系统性能的**瓶颈**
- 除了可以通过采用**高速器件**（如果FPM-DRAM、EDO-DRAM、BEDO-DRAM、SDRAM、DDR SDRAM等）来提高存储器访问速度外，还可以通过存储体的**并行**工作来提高存储器的访问速度

• 1、双端口存储器

- 有两个相互独立的端口（左L、右R）
- **MAR**: 存储器地址寄存器
- **MDR**: 存储器数据寄存器
- **Read**: 读控制信号
- **Write**: 写控制信号
- **/CS**: 片选信号
- **/Busy**: 忙信号

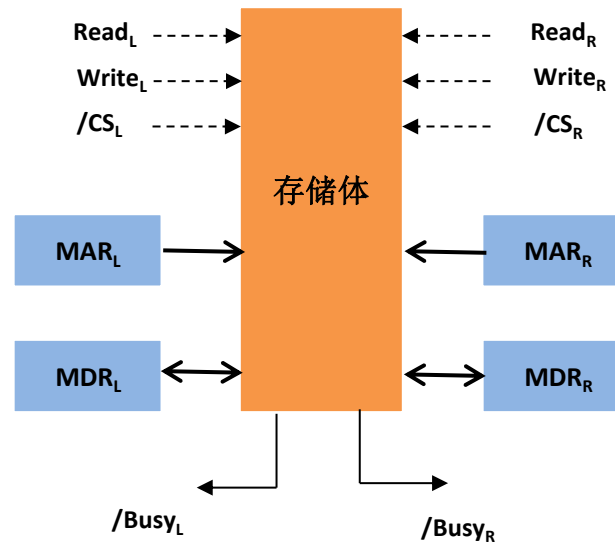


图4.27 双端口存储器结构示意图

4.4 并行主存系统

- (1) 并行读写
 - 当左右两个端口的地址不同时，两个端口使用各自的地址线、数据线和控制线，此时可以同时对存储体的两个不同的存储单元进行访问（读写）
- (2) 冲突处理
 - 当左右两个端口的地址相同时，将发生冲突（因为存储体不允许同时对同一个存储单元进行读写操作）
 - 此时由判断逻辑决定是哪一端先访问存储体，如左边的端口先访问存储体，然后将右边端口的 $/\text{Busy}_R$ 置为低电平（ $/\text{Busy}_R=0$ ，表示右边的端口不能访问存储体）；等左边的端口访问结束后，再将右边端口的 $/\text{Busy}_R$ 置为高电平（ $/\text{Busy}_R=1$ ，表示右边的端口可以访问存储体）

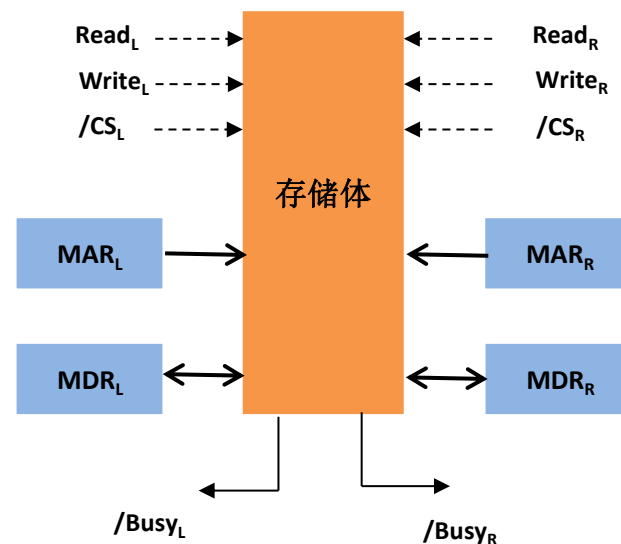


图4.27 双端口存储器结构示意图

4.4 并行主存系统

2、单体多字存储器

- 单体多字存储器的结构与存储器**位扩展方式**完全相同，此时多个存储模块（图4.24有32个存储模块，右图有4个存储模块）共享地址总线，可以实现同时访问多个存储模块
- 单体多字系统的特点是存储器中**只有一个存储体**，每个存储单元**存储m个字**，总线宽度也为m个字。一次并行读出m个字，地址必须顺序排列并处于同一存储单元。
- **缺点：**指令和数据在主存内必须是连续存放的，一旦遇到转移指令，或操作数不能连续存放，这种方法的效果就不明显

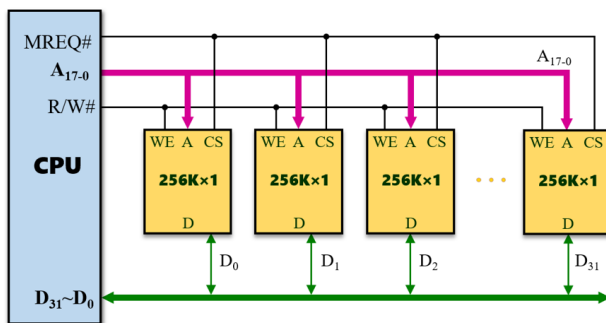
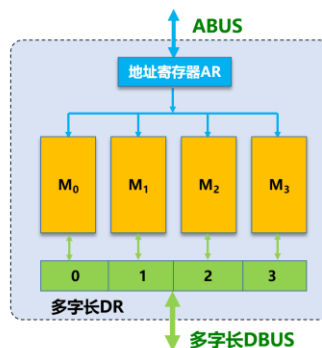


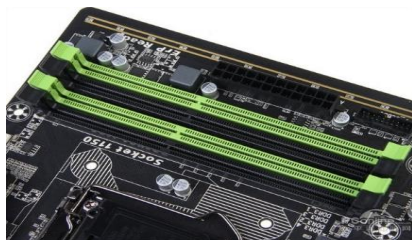
图4.24 存储器位扩展

存储器的位扩展（单体32字）



单体多字存储器的结构
单体4字

- **多通道内存技术：**就是采用单体多字技术，常见的有双通道、三通道、四通道技术



电脑主板上的**双通道**内存插槽



4.4 并行主存系统

— 联动模式的双通道内存技术:

- 相当于存储器位扩展中有2个存储模块，两个存储模块的地址线（A）、片选线（CS）、数据线（D）共享
- 此时要求两个存储模块（两根内存条）的容量、频率、时序必须完全一致，才能实现同步工作

— 非联动模式的双通道内存技术:

- 此时，两个存储模块采用独立的地址线（A1、A2）、片选线（CS1、CS2）和控制线（图中没有画出），数据总线也是独立的（D1、D2）
- 这种方式两个存储模块（两根内存条）可以独立工作，也可以并发工作；非联动方式的两根内存条只需要频率相同，容量、时序特征并不要求完全一致

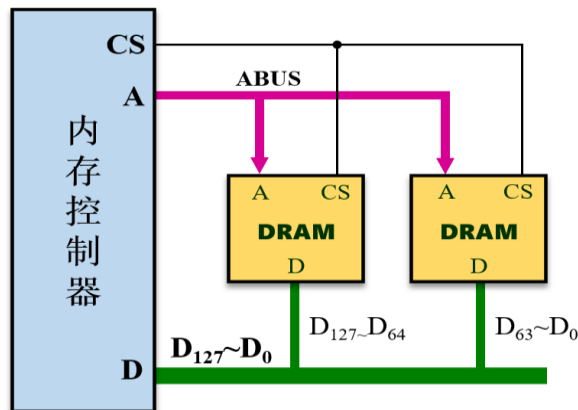


图4.28 (a) 联动模式

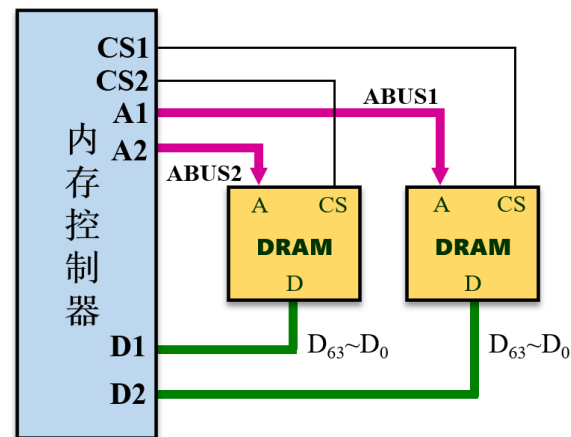


图4.28 (b) 非联动模式

4.4 并行主存系统

• 3、多体交叉存储器

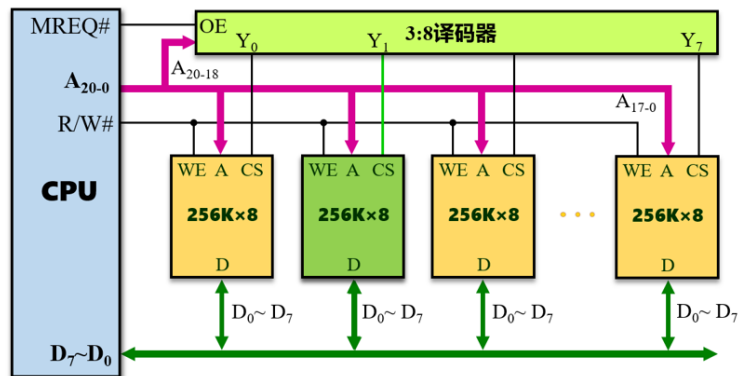
– (1) 高位多体交叉

- 高位多体交叉存储器的结构与存储器**字扩展**完全相同，也称为**顺序编址**模式；高位地址（ $A_{20} \sim A_{18}$ ）经过译码器用于选择不同的存储体，低位地址（ $A_{17} \sim A_0$ ）同时送各个存储体

- 高位多体交叉存储器的**特点**：

- ① 相邻的地址往往在同一个存储体内（如地址0、1、2、3都在存储体 M_0 中）
- ② 不同存储体中的地址不相邻（存储体 M_0 、 M_1 、 M_2 、 M_3 的第一单元的地址分别是0、8、16、24，不相邻）

- 高位多体交叉存储器的**缺点**：由于程序具有局部性和连续性的特点，程序的指令和数据往往分布在同一个存储体中，从而导致一个存储体被频繁访问，其它存储体基本处于空闲状态，无法实现多个存储体并行工作



存储器的字扩展（高位交叉）

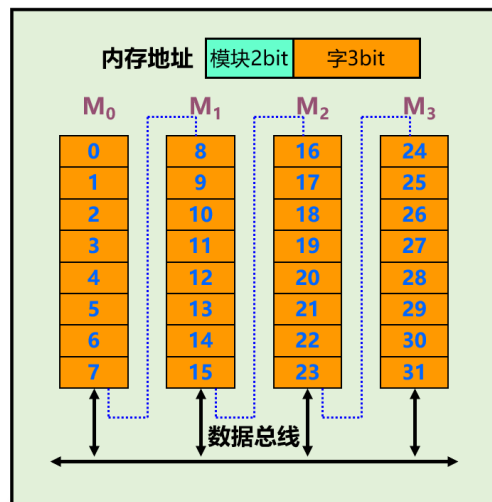
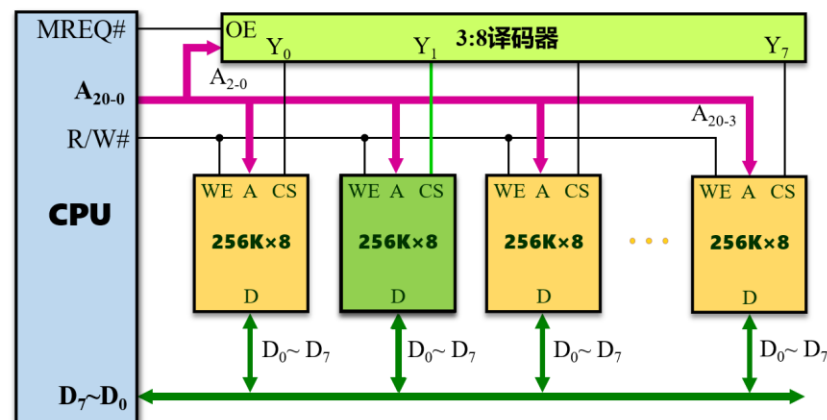


图4.29 (a) 高位交叉方式（顺序编址）

4.4 并行主存系统

— (2) 低位多体交叉

- 低位多体交叉存储器的结构与存储器字扩展也基本相同，只是低位地址（右下角图中的 $A_2 \sim A_0$ ）经过译码器用于选择不同的存储体，而高位地址（ $A_{20} \sim A_3$ ）同时送各个存储体；低位多体交叉也称为交叉编址模式
- 低位多体交叉存储器的特点：
 - ① 相邻的地址往往在不同的存储体内（如地址0、1、2、3分别处在存储体 M_0 、 M_1 、 M_2 、 M_3 中）
 - ② 同一存储体中的地址不相邻（存储体 M_0 的地址分别是0、4、18、12、……，不相邻）
- 低位多体交叉存储器的优点：由于程序具有局部性和连续性的特点，如果将程序的指令和数据分布在不同的存储体中，可以实现多个存储体并行工作



存储器的字扩展（低位交叉）

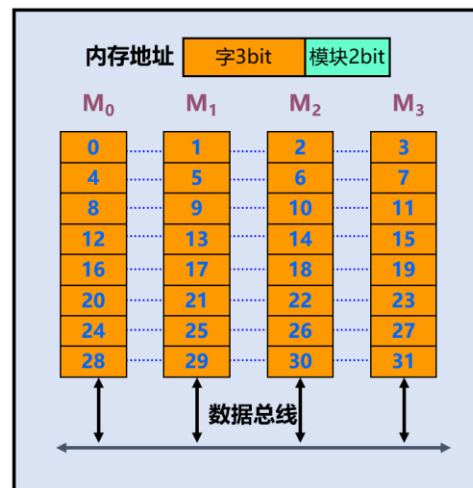


图4.29 (b) 低位交叉方式（交叉编址）

4.4 并行主存系统

- 例4.1：设计计算机字长为64位，存储器容量为128MW（**W表示字**=64位，因此存储器的容量=128x2²⁰x64位），采用模数为8的存储器交叉方式进行组织（即**m=8**），存储周期**T=200ns**，数据总线宽度为64位，总线传输周期**τ=25ns**（**T=mτ**，200ns=8x25ns），若连续读8个字，分别计算顺序访问方式和交叉访问方式下存储器的带宽。
- 解：
 - 顺序方式下，连续读8个字所需的时间=8T=8x200ns=1600ns
 - 顺序方式下的存储器带宽=(64x8)/1600ns=3.2x10⁸bps=320Mbps
 - 交叉方式下，连续读8个字所需的时间 = T + (8-1)τ = 200ns + 7x25ns = 375ns
 - 交叉方式下的存储器带宽=(64x8)/375ns=1.37x10⁹bps=1370Mbps
 - 交叉方式比顺序方式快：1370/320 = 4.27倍

4.5 高速缓冲存储器

• 4.5.1 cache工作原理

- SRAM的特点是速度快、容量小、价格高，DRAM的特点是速度慢、容量大、价格低，因此计算机的主存（内存，内存条）通常采用DRAM，而不是采用SRAM
- 为了提高CPU访问主存的速度，通常在CPU与主存之间增加一个隐藏的小容量快速SRAM，称为cache（高速缓冲存储器）
- 因为CPU执行的程序具有较强的局部性，可以将主存中经常访问或即将访问的数据的副本调度到cache中，使得大部分数据访问可以在cache中进行，从而提升系统的性能

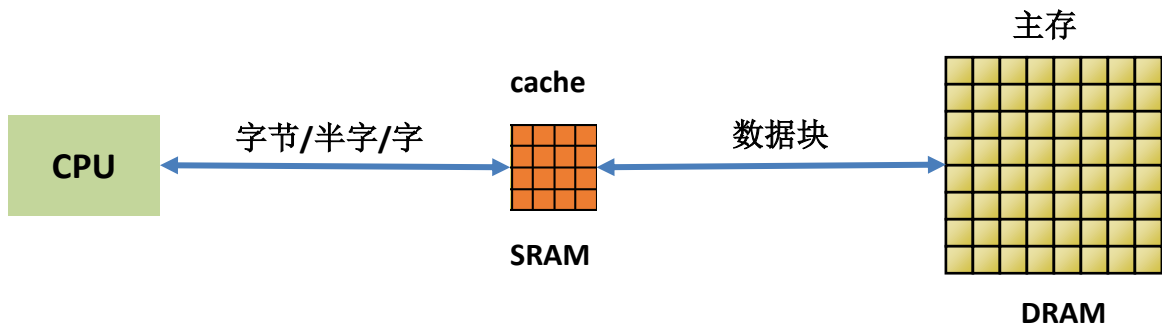


图4.31 cache基本原理

4.5.2 程序局部性

- **程序局部性**：是指在一段时间内，整个程序的执行仅限于程序中的某一部分（时间），而执行程序所需的指令和数据也仅局限于某个存储区域内（空间）
- **时间局部性**：是指当程序访问一个存储位置时，该位置在**未来可能会被多次访问**，如程序的循环结构和调用过程就很好地体现了时间局部性
- **空间局部性**：是指一旦程序访问了某个存储单元，则其**附近的存储单元**也即将被访问；如计算机中的指令代码、数组、结构体元素，通常在主存中顺序存放，对其数据访问就具有很强的空间局部性

- **例4.2**：分析下面C语言代码的程序局部性

```
int sumvec()
{
    int i,sum=0;
    for (i=0; i<1000; i++)
        sum += V[i];
    return sum;
}
```

4.5.2 程序局部性

— 解：

- **for**循环体中的指令在主存中顺序存放，具有**空间局部性**；**for**循环体将被执行**1000**次，因此还具有**时间局部性**
- 变量**i**、**sum**是单个变量，不存在空间局部性；但是这两个变量在每次循环执行时都会用到，因此具有**时间局部性**
- 数组**V[i]**中包含**1000**个数据元素，这些数据在内存中是顺序存放，因此具有**空间局部性**

```
int sumvec()
{
    int i,sum=0;
    for (i=0; i<1000; i++)
        sum += V[i];
    return sum;
}
```

4.5.3 cache的基本概念

- **命中**：如果CPU要访问的数据在cache中，则称为命中（Hit）；将命中时的数据访问时间称为**命中访问时间**，通常为cache访问时间 t_c
- **数据缺失**：如果CPU要访问的数据不在cache中，则称为数据缺失（Miss，也称为不命中），此时要将缺失数据从主存调入cache中才能访问数据；将数据缺失时的访问时间称为**缺失补偿**（Miss Penalty），通常为主存访问时间 t_m
- 通常将cache和主存分成若干个固定大小的**数据块**（Block）
- **主存地址** = 主存块地址 + 块内偏移（offset）
- **cache地址** = cache块地址 + 块内偏移（offset）
- 主存和cache的块内偏移是一样大的，即主存块和cache块的大小是相同的

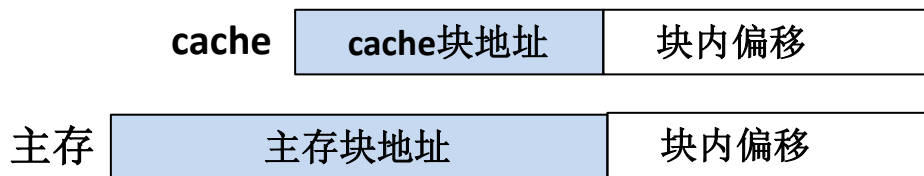


图4.32 cache和主存的地址格式

4.5.3 cache的基本概念

- **命中率**：假设 N_c 为命中cache的次数， N_m 为从主存中访问信息的次数（即不命中的次数），则命中率 h （Hit Ratio）：
 - $h = N_c / (N_c + N_m)$
- **缺失率**（Miss Ratio）= $1-h$
- cache/主存系统的**平均访问时间** t_a ：
 - $t_a = ht_c + (1-h)t_m$
 - 其中， t_c 为命中访问时间； t_m 为缺失访问时间；命中率 h 越高， t_a 越接近 t_c ， $h=1$ 时， $t_a=t_c$
- **访问效率** e ：
 - $e = t_c / t_a = t_c / (ht_c + (1-h)t_m) = 1 / (h + (1-h)r)$
 - 其中 $r = t_m / t_c$ ，表示主存与cache访问时间的倍数
- 命中率与程序局部性、cache容量、数据块（Block）大小、cache的组织方式有关

Cache优化策略

B.3 Six Basic Cache Optimizations

- First Optimization: Larger Block Size to Reduce Miss Rate
- Second Optimization: Larger Caches to Reduce Miss Rate
- Third Optimization: Higher Associativity to Reduce Miss Rate
- Fourth Optimization: Multilevel Caches to Reduce Miss Penalty
- Fifth Optimization: Giving Priority to Read Misses over Writes to Reduce Miss Penalty
- Sixth Optimization: Avoiding Address Translation During Indexing of the Cache to Reduce Hit Time

2.3. Ten Advanced Optimizations of Cache Performance

- First Optimization: Small and Simple First-Level Caches to Reduce Hit Time and Power
- Second Optimization: Way Prediction to Reduce Hit Time
- Third Optimization: Pipelined Access and Multibanked Caches to Increase Bandwidth
- Fourth Optimization: Nonblocking Caches to Increase Cache Bandwidth
- Fifth Optimization: Critical Word First and Early Restart to Reduce Miss Penalty
- Sixth Optimization: Merging Write Buffer to Reduce Miss Penalty
- Seventh Optimization: Compiler Optimizations to Reduce Miss Rate
- Eighth Optimization: Hardware Prefetching of Instructions and Data to Reduce Miss Penalty or Miss Rate
- Ninth Optimization: Compiler-Controlled Prefetching to Reduce Miss Penalty or Miss Rate
- Tenth Optimization: Using HBM to Extend the Memory Hierarchy

Sixth Edition

John L. Hennessy | David A. Patterson

COMPUTER ARCHITECTURE

A Quantitative Approach



4.5.4 cache读、写流程与关键技术

– 1、cache读流程

- CPU给出主存地址RA（Read Address，包括块地址和块内偏移），以RA的块地址为关键字在cache中进行查找，如果找到表示**命中**，此时通过访问cache就可获取地址为RA的数据
- 如果找不到则表示**数据缺失**，此时需要访问主存，将地址为RA的主存块调入cache中，并发送地址为RA的数据到CPU中
- 此时若cache已经满了，还需要进行**替换**，并更新查找表

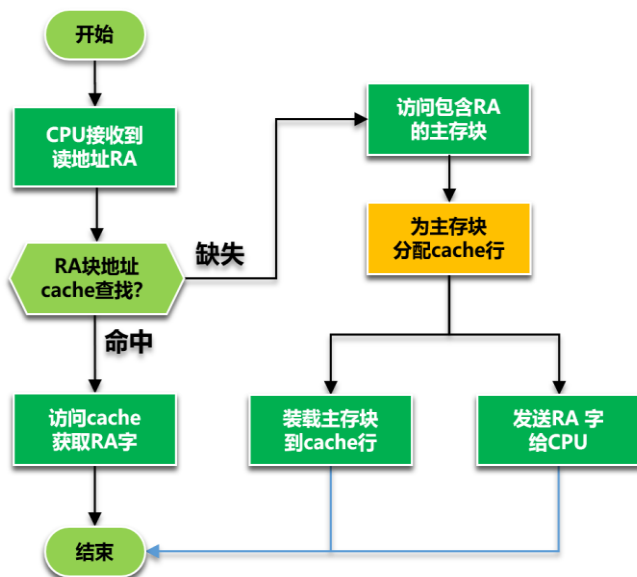


图4.33 cache读操作的基本流程



cache的替换过程

– 2、cache写流程

- CPU给出主存地址WA（Write Address，包括块地址和块内偏移），以WA块地址为关键字在cache中进行查找，如果找到表示**命中**；此时将地址为WA的数据写入cache中，并根据写策略决定要不要同时写入主存（如果是**写回策略**，则不需要写入主存；如果是**写直达策略**，则需要同时写入主存）

写缺失：要写入的数据块尚未调入cache

写分配：先将数据块调入cache，执行写操作

如果找不到则表示**写缺失**，此时若采用**写分配法**，则将地址为WA的主存块调入cache中，再进行数据写入cache的操作；如果不采用写分配法，则直接将数据写到主存中。

此时若cache已经满了，还需要进行**替换**，并更新查找表

- **脏数据**：新写入cache中的数据与主存中的原始数据不一致时，新写入cache中的这部分数据称为脏数据（Dirty Data）

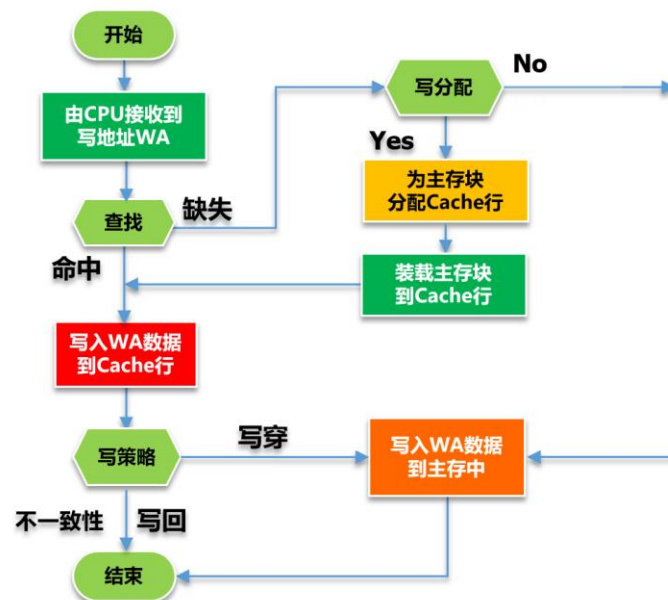
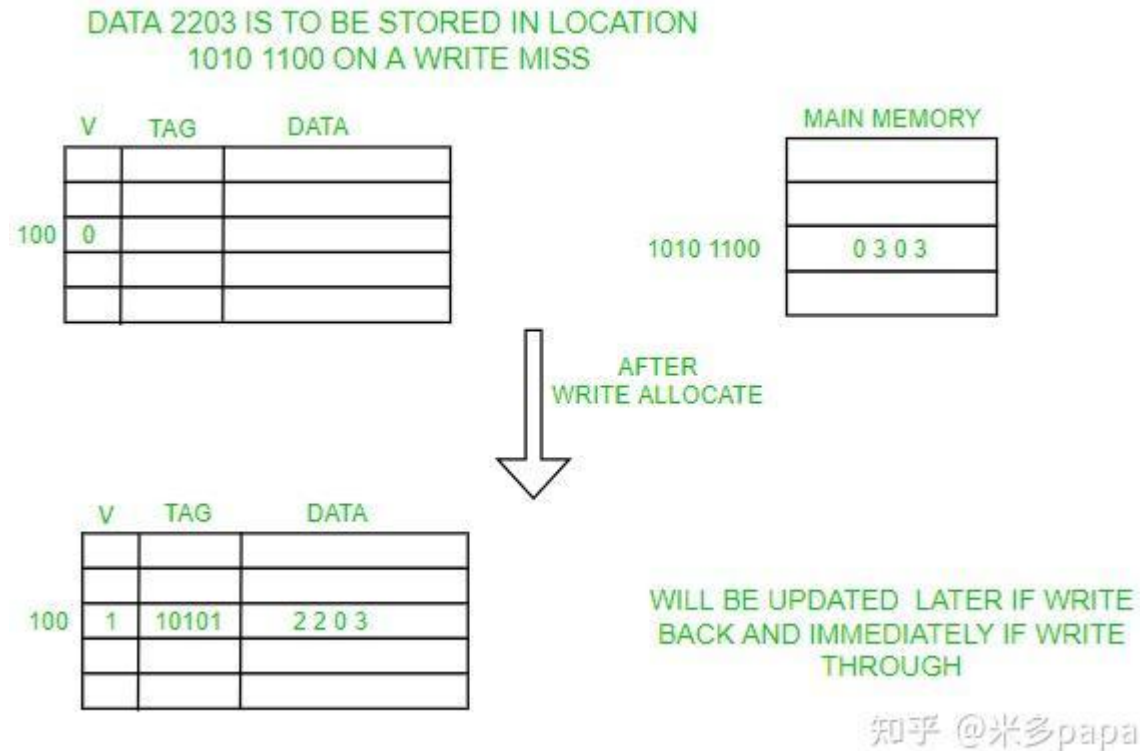


图4.34 cache写操作的基本流程



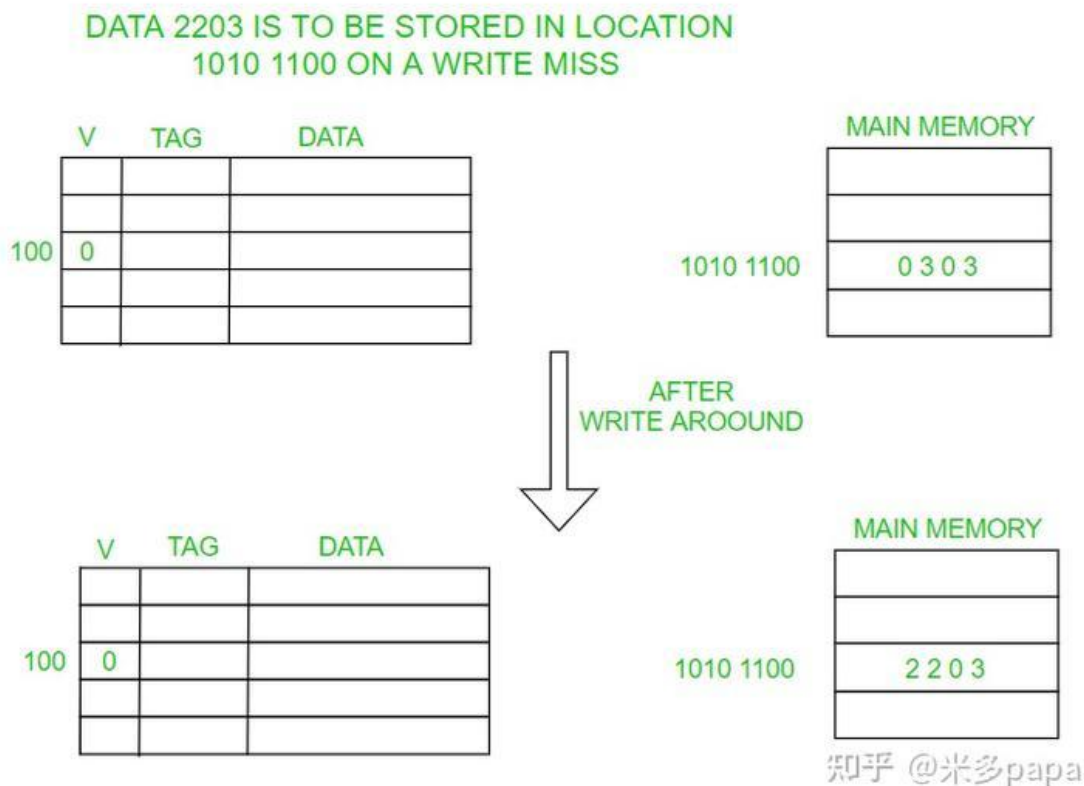
cache的替换过程

Write allocate (写分配)



写分配（**Write Allocation**）也被称为写时取（**fetch on write**）。即：在发生写缺失时，先把这一块读到cache，再在cache中写。在这种方法中，写入未命中类似于读取未命中（适用于**write back**）。

Write no allocate（无写分配）



无写分配（**No Write Allocation**）也称为 **Write-no-allocate** 或 **write around**。即：写cache失效时，直接写进内存。在这种方法中，仅在读取未命中时将数据加载到缓存中。这里数据直接写入或者更新到主存而不干扰缓存。当数据无需立即再次使用时，最好使用这种方法（对应**write through**）。

4.5.4 cache读、写流程与关键技术

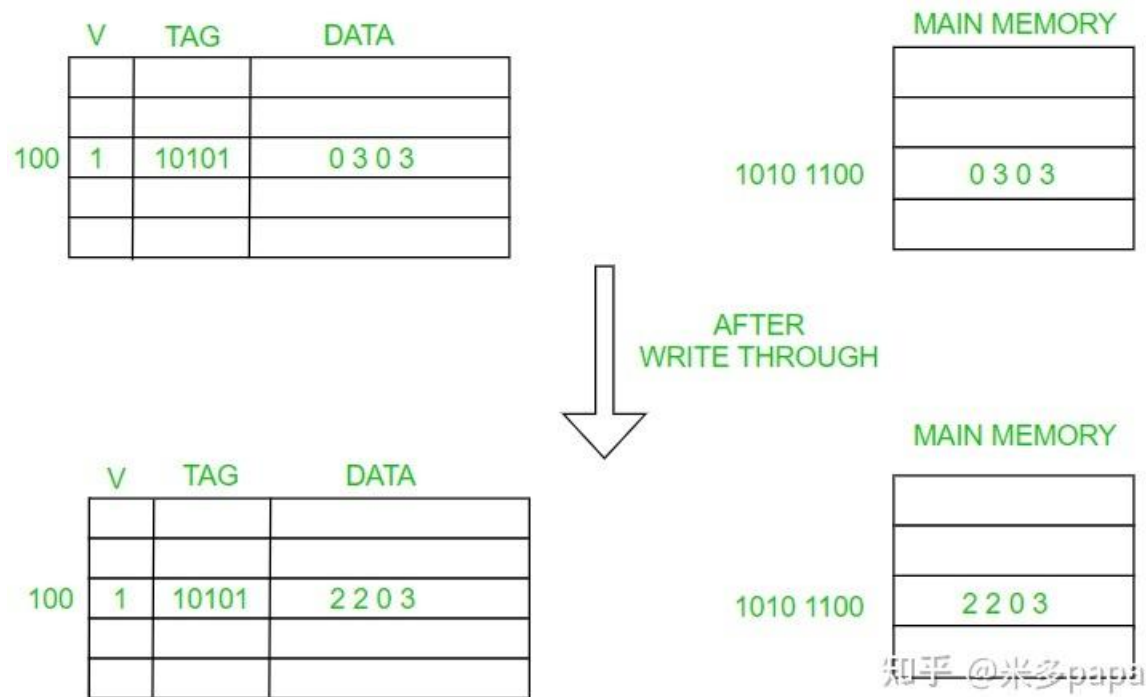
- 因为cache是主存一部分内容的**副本**，在写入cache时，需要保证cache与主存数据的一**致性**，即cache的内容改变了，主存相应的内容也要改变
- Cache的写入策略主要有：写回法、写穿法
- 1、写回法（Write Back, WB）
 - 采用写回法，当写入cache时，只修改cache的内容，而不立即修改主存的内容；只有**当该cache行被替换出去**时，才将脏数据（cache被修改后的数据称为脏数据）写回主存
 - 写回法可以使cache在大部分时间（没有发生替换的时间）的写入速度等于访问cache的速度，只有在发生替换时，cache的写入速度才变为访问主存的速度（此时要执行写入主存的操作）
 - 采用写回法时，cache的每一行必须配置一个**修改位，也称为脏位（Dirty Bit）**；若该行被修改，则脏位=1，否则为0；当该行被替换出去时，如果脏位=1，则需要将该行的内容写入主存

4.5.4 cache读、写流程与关键技术

– 2、写穿法（Write Through，WT）

- 写穿法也称为直写法，当写入cache时，**同时**对cache的行和主存对应的数据块进行修改
- 写穿法的优点：cache的每行不需要设置一个修改位（脏位），并且发生替换时，直接将该cache行丢弃，不需要写回主存
- 写穿法的缺点：cache的写入时间将变为访问主存的时间，从而大大降低了写cache的速度
- 写穿法较好地维护了cache与主存的一致性，Intel 80486处理器的片内cache采用的就是写穿法

Write through（写直达）

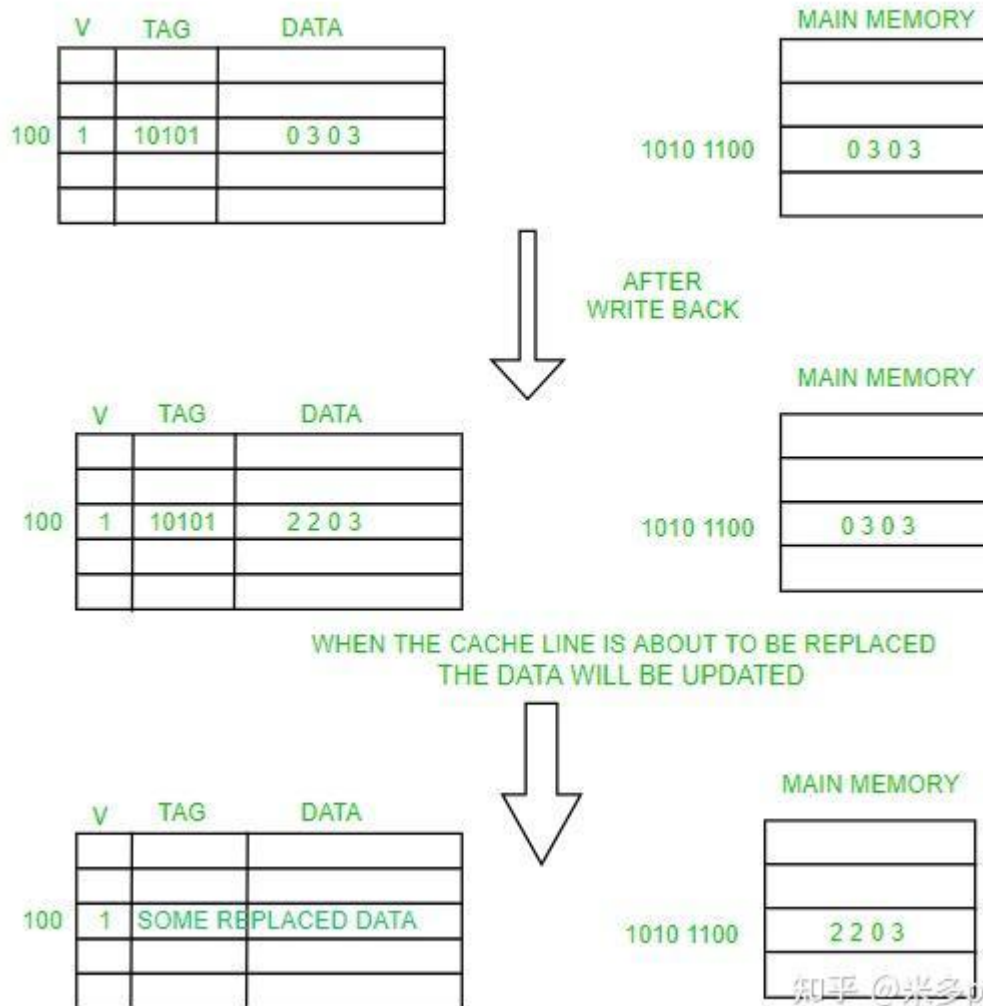


在透写（**Write Through**）场景中，数据同时更新到缓存和内存（**simultaneously updated to cache and memory**）。这个过程更简单、更可靠。这用于没有频繁写入缓存的情况（写入操作的次数较少）。

它有助于数据恢复（在断电或系统故障的情况下）。因为我们必须写入两个位置（内存和缓存），数据写入将经历延迟。虽然它解决了不一致的问题，但它的问题是在写操作中使用缓存的优势，因为使用缓存的全部目的是避免对主内存的多次访问。

Write back (写回)

回写 (**Write Back**) 也被称为延迟写入 (**Write Behind / Write Deferred**)。也就是说,最初数据只在缓存中更新,稍后再更新到内存中。对内存的写入动作会被推迟,直到修改的内容在缓存中即将被另一个缓存块替换。



4.5.4 cache读、写流程与关键技术

— 3、cache实现关键技术

4.5.5 相联存储器

- (1) **数据查找**：即如何快速查找数据是否在cache中？如果查找到，则为命中，否则为缺失；通常采用**相联存储器**以提高数据查找的速度

4.5.6 地址映射

- (2) **地址映射**：主存中的数据块应如何放置到cache中？是任意放置，还是按照一定的规则放置？不同的**地址映射策略**会对cache的性能及硬件成本带来影响

4.5.7 替换算法

- (3) **替换策略**：在将主存数据块放置到cache中时，如果cache满了，则需要将cache中的某个数据块替换或淘汰掉，不同的替换策略会对数据命中率产生影响

4.5.8 写入策略

- (4) **写入策略**：在cache的写入操作时，如何保证cache与主存数据的一致性，通常有写回和写直达两种策略，**写回策略**可以提高写操作的速度，但会带来数据不一致的问题；**写直达策略**能保证数据一致性，但影响写入操作的速度

4.5.5 相联存储器

- 普通的存储器是按地址进行访问，而相联存储器则是按内容进行访问
- 相联存储器（CAM）：Content Addressable Memory
- 相联存储器用于存放cache的查找表，其内部存储的基本数据是键值对（key，value）
- 相联存储器的输入不是地址，而是检索关键字key，输出是key对应的value值
- 例如要查找某人的工资数，此时key为“姓名”或“工号”，value为“工资数”，在相联存储器中输入key=“张帅”，就可以得到value=“4000”；或者输入key=“021”，就可以得到value=“2000”

按内容进行访问的存储器

(key,value)

物理地址	工号	姓名	出生年月	工资数
N	001	张帅	1976/7	4000
N+1	021	李猛	1978/9	2000
N+2	023	郝牛	1977/6	3000

4.5.5 相联存储器

— 相联存储器的基本原理（图4.35）：

- 假设相联存储器（cache的查找表）中有4个键值对（key，value），每个键值对还有一个有效位valid；valid=1表示该键值对是有效的，valid=0表示该键值对是无效的
- 假设检索关键字key=234，此时会将该检索关键字与相联存储器中的4个键值对的key进行比较，比较相等且valid=1的那个键值对（图中的第2个键值对）为找到的键值对，将其value值输出（value=777）

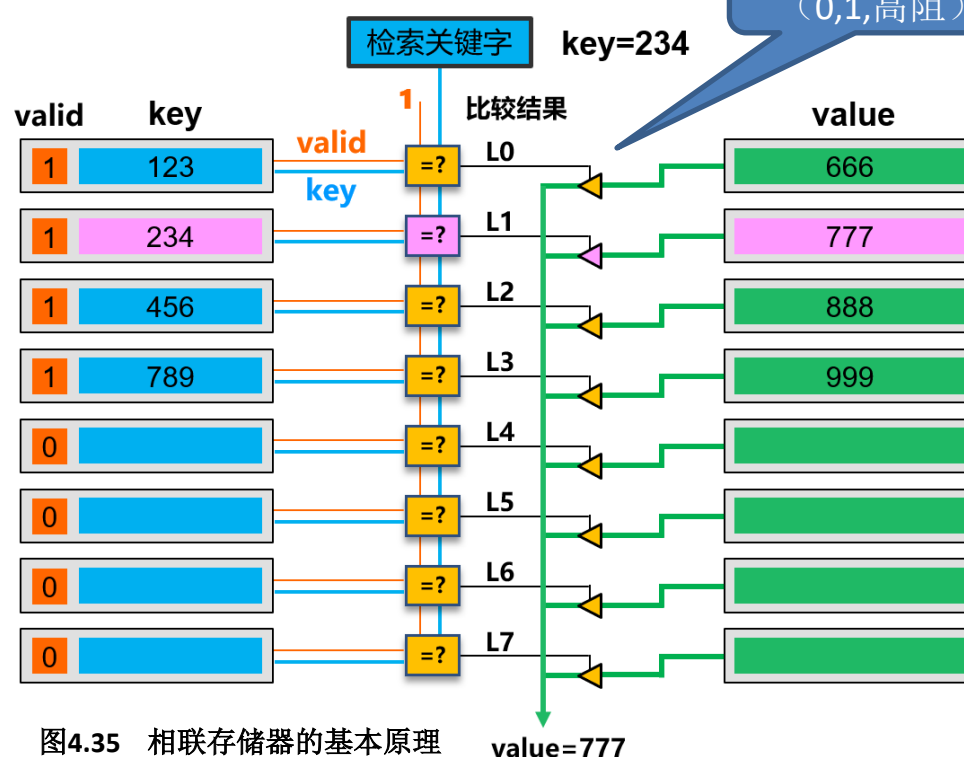


图4.35 相联存储器的基本原理

4.5.6 地址映射

- 地址映射是指如何将主存地址空间映射到cache的地址空间，即主存的某一块将载入到cache的哪一块中？
- 地址映射有3种方法：
 - ① 全相联映射（**Full Associative Mapping**）：主存的某一块可以映射到cache的任意块中
 - ② 直接相联映射（**Direct Mapping**）：主存的某一块只能映射到cache的固定块中
 - ③ 组相联映射（**Set Associative Mapping**）：主存的某一块只能映射到cache的固定组的任意块中

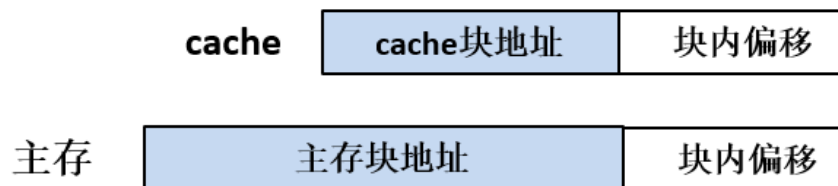


图4.32 cache和主存的地址格式

4.5.6 地址映射

– (1) 全相联映射方式:

- 假设：主存块地址=10位，cache块地址=5位；则主存有1024块，cache有32块（行）
- 主存的某一块，可以映射到cache的任一块；例如主存的第0块，可以映射到cache的第0块、第1块、...、第31块

全相联映射

主存 (块)	cache (块)
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	0
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	1
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	2
.....	...
.....	...
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	30
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	31

4.5.6 地址映射

– (2) 直接相联映射方式:

- 假设: 主存块地址=10位, cache块地址=5位; 则主存有1024个块, cache有32块(行)
- 主存的某一块, 只能映射到cache的固定一块; 例如主存的第0块, 只能映射到cache的第0块; 主存的第33块, 只能映射到cache的第1块
- 地址映射公式: $i(\text{cache块}) = j(\text{主存块}) \bmod n(\text{cache块数/行数})$

直接相联映射

主存 (块)	cache (块)
0、32、64、96、128、160、192、224、.....、960、992	0
1、33、65、97、129、161、193、225、.....、961、993	1
2、34、66、98、130、162、194、226、.....、962、994	2
.....	...
.....	...
30、62、94、126、158、190、222、254、.....、990、1022	30
31、63、95、127、159、191、223、255、.....、991、1023	31

4.5.6 地址映射

— (3) 组相联映射方式:

- 假设: 主存块地址=10位, cache块地址=5位, cache采用2路组相联映射方式; 则主存有1024个块, cache分为16组、每组2块(行)、共32块(行)
- 主存的某一块, 只能映射到cache的**固定组**(在该组中可以映射到其中的**任意一块**); 例如主存的第0块, 只能映射到cache的第0组(可以映射到第0组的第0块或第1块); 主存的第33块, 只能映射到cache的第1组(可以映射到第1组的第2块或第3块)

2路组相联映射

主存 (块)	cache	
	组	块
0、16、32、48、64、80、96、.....、992、1008	0	0、1
1、17、33、49、65、81、97、.....、993、1009	1	2、3
2、18、34、50、66、82、98、.....、994、1010	2	4、5
.....	...	...
.....	...	...
14、30、46、62、78、94、110、.....、1006、1022	14	28、29
15、31、47、63、79、95、111、.....、1007、1023	15	30、31

4.5.6 地址映射

– 1、全相联映射

• (1) 全相联映射基本原理

- 全相联映射：主存的某一块可以映射到cache的任意块中
- 假设主存有 mn 块 ($B_0 \sim B_{mn-1}$)，cache有 n 块 (行0~行 $n-1$)；cache的每一行包括：数据块副本（即主存某一块的副本）、主存块地址、有效位
- 地址映射过程：将主存地址中的主存块地址（tag）与 n 个cache行的主存块地址进行比较（ n 路并发比较，即1个与 n 个进行比较），比较相等且有效位=1的那一行cache为命中，否则为缺失（没有比较相等的对应块，或者是比较相等但有效位=0）

因为全相联是映射到cache任意块，没有cache块信息，因而需要并行比较。

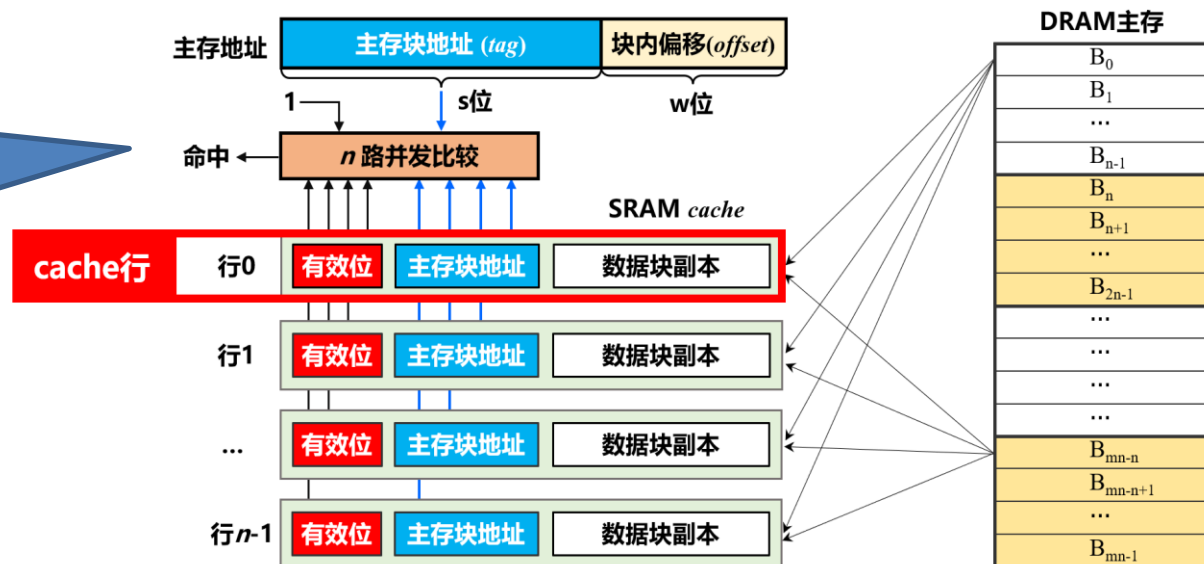


图4.36 全相联映射的逻辑示意图

• (2) 全相联映射硬件实现

没有cache块号，因为全相联映像
主存任意块可进入cache任意块

– 假设：主存地址（9位）= 主存块地址tag（7位）+ 块内偏移（2位）

– 即主存有128块（ $2^7=128$ ），cache只有8块（8行）

– 因为块内偏移为2位，因此cache的每一行数据块副本缓冲区中有4个字（ $2^2=4$ ）

Offset 确定具
体输出4个字中
的哪一个

选中cache的L4行
中的w 1 (01)



图4.37 全相联映射的硬件逻辑实现框图（该图只有cache，没有主存）

- 现要求访问主存地址（十六进制）分别为1F、20、24、48、54、55、103的7个字，请给出全相联映射的访问过程
- 访问主存地址为1F的字时，因为cache初始态为空，访问缺失，需要将主存地址为1F所在的块（4个字，地址分别为：1C、1D、1E、1F）载入到cache的第0行（也可以载入到cache的其它行），并将第0行cache的tag置为主存的tag（7）、valid置为1
- 访问主存地址为20、24、48、54、103的字的过程，与访问主存地址为1F的字的过程类似，都属于载入
- 访问主存地址为55的字时，因为主存块地址tag=15，此时多路并发比较相等，且有效位valid=1，表示命中

表4.4 全相联映射的访问过程

访问序列	1	2	3	4	5	6	7
主存地址 (十六进制)	1F	20	24	48	54	55	103
主存地址 (二进制)	00001111	00010000	00010010	00100100	00101010	00101011	10000000
主存地址 (tag,offset) (十六进制)	(7, 3)	(8, 0)	(9, 0)	(12, 0)	(15, 0)	(15, 1)	(40, 3)
cache行	0	1	2	3	4	4	5
访问情况	载入	载入	载入	载入	载入	命中	载入

- 全相联映射方式的特点:

- ① 主存数据块可以映射到cache的任意一行（块），因此cache的**利用率高**
- ② 只要cache中还有空行就不会引起冲突，因此cache的**冲突率低**
- ③ 查找时需要并发比较查找表中所有项，每一个cache行对应一个比较电路，**硬件成本较高**，只适合小容量cache使用
- ④ cache满时，载入新数据块需要利用替换算法进行替换，**替换策略和算法较为复杂**

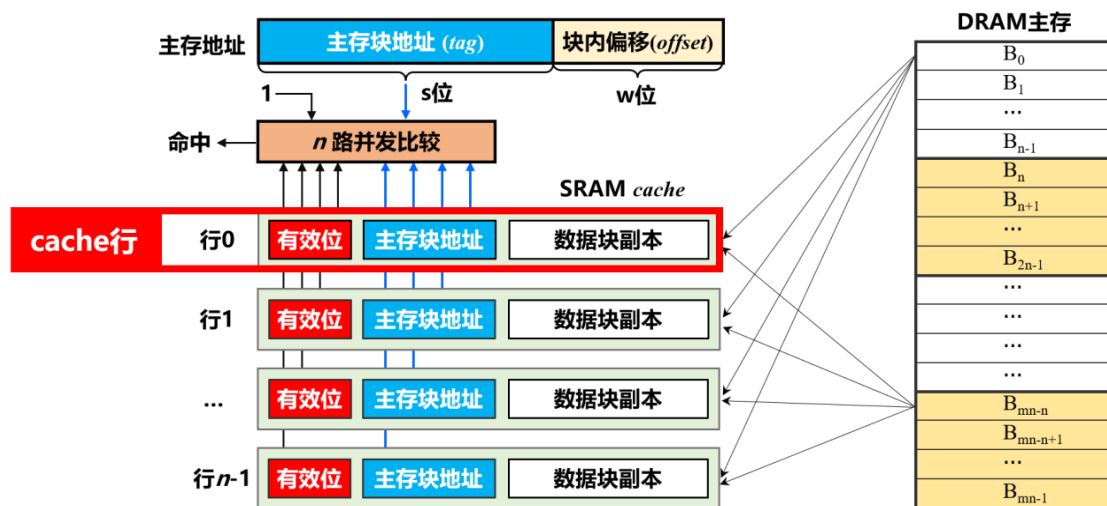


图4.36 全相联映射的逻辑示意图

– 2、直接相联映射

• (1) 直接相联映射基本原理

– 直接相联映射：主存的某一块只能映射到cache的固定行（块）中

– 假设主存块号为 j ，cache行号为 i ，cache共有 n 行，则有： $i = j \bmod n$

其实就是 n 块

行索引，实际是以cache块地址为索引，进行匹配。

主存字块标记

cache块地址

块内地址

– 主存地址 = 区地址 (tag) + 行索引 (index) + 块内偏移 (offset)

– 地址映射过程：根据主存地址中的行索引 (index) 找到cache的对应行，将主存地址中的区地址 (tag) 与该cache行的区地址 (tag) 进行比较，比较相等且有效位=1则为命中，否则为缺失（比较不相等，或者是比较相等但有效位=0）

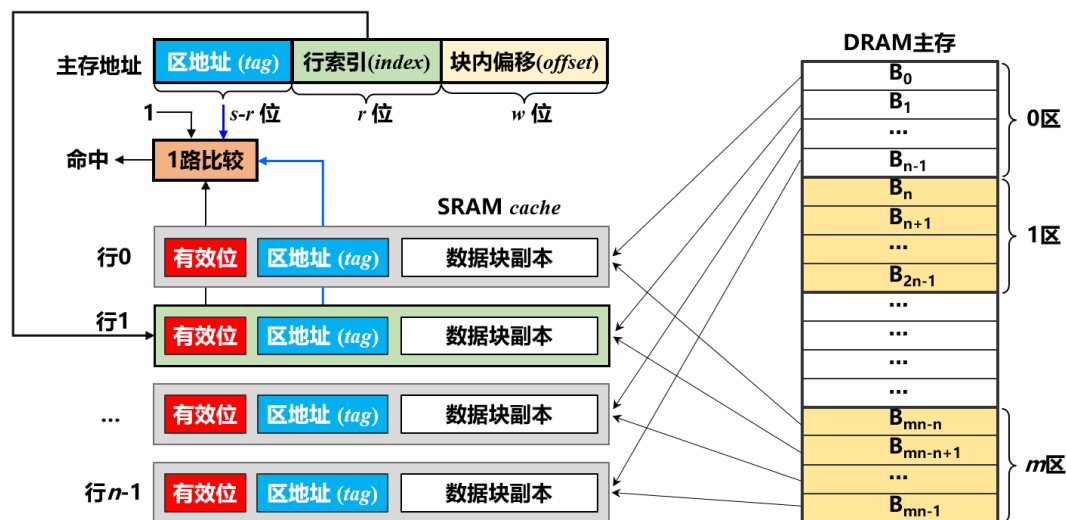


图4.38 直接相联映射的逻辑示意图

- (2) 直接相联映射硬件实现

- 假设：主存地址（9位）= 区地址tag（4位）+ 行索引index（3位）+ 块内偏移offset（2位）
- 即主存有128块（ $2^7=128$ ），分为16个区（ $2^4=16$ ），每个区8块（ $2^3=8$ ）；cache只有8块（8行）
- 块内偏移为2位，因此cache的每一行数据块副本缓冲区中有4个字（ $2^2=4$ ）

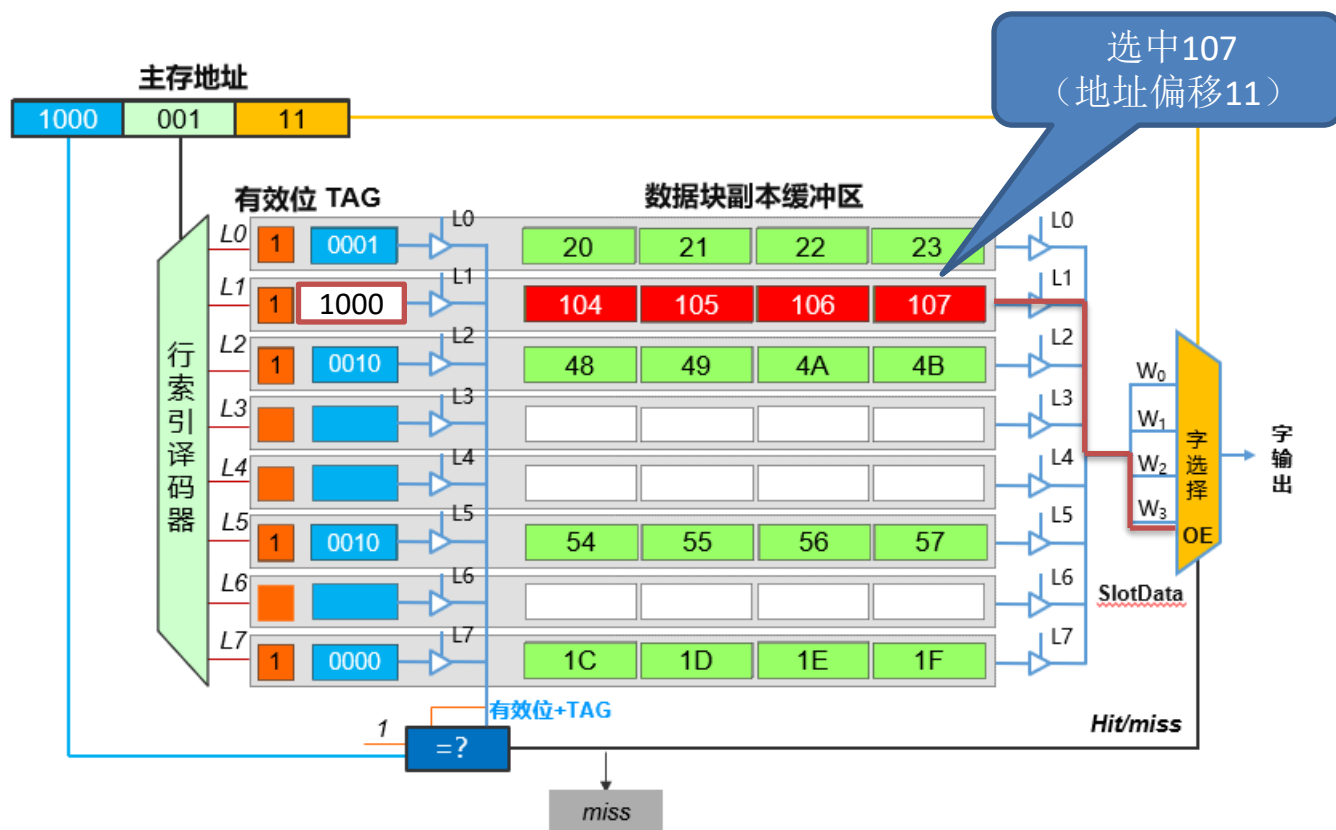


图4.39 直接相联映射的硬件逻辑实现框图

- 现要求访问主存地址（十六进制）分别为1F、20、24、48、54、55、103的7个字，请给出直接相联映射的访问过程
- 访问主存地址为1F的字时，因为cache初始态为空，访问缺失，需要将主存地址为1F所在的块（4个字，地址分别为：1C、1D、1E、1F）**载入**到cache的第7行（1F对应的行索引=7），并将该行cache的tag置为主存的tag（0000）、valid置为1；访问主存地址为20、24、48、54的字的访问过程，与访问主存地址为1F的字的访问过程类似，都属于**载入**
- 访问主存地址为55的字时，根据其行索引（index=5）找到cache的第5行，并比较主存的区地址（tag=0010）与该行cache的区地址（0010），结果相同，且有效位valid=1，表示**命中**
- 访问主存地址为103的字时，根据其行索引（index=0）找到cache的第0行，并比较主存区地址（tag=1000）与该行cache的区地址（0001），结果不相同，但有效位valid=1，表示该cache行有数据（**冲突**），需要进行**替换**

表4.5 直接相联映射的访问过程

访问序列	1	2	3	4	5	6	7
主存地址 (十六进制)	1F	20	24	48	54	55	103
主存地址 (二进制)	0000 111 11	0001 000 00	0001 001 00	0010 010 00	0010 101 00	0010 101 01	1000 000 11
主存地址 (tag,index,offset) (十六进制)	(0, 7 , 3)	(1, 0 , 0)	(1, 1 , 0)	(2, 2 , 0)	(2, 5 , 0)	(2, 5 , 1)	(8 , 0 , 3)
cache行	7	0	1	2	5	5	0
访问情况	载入	载入	载入	载入	载入	命中	替换

- 直接相联映射方式的特点：

- ① 由于主存数据块只能映射到cache的特定行（块），因此cache的**利用率低、命中率低**
- ② 行索引（**index**）相同的所有主存块都将映射到cache的同一行，即使cache未滿，也会发生数据冲突，因此cache的**冲突率高**
- ③ 查找时只需要根据主存地址的行索引（**index**）访问对应的cache行，并比较主存地址的区地址（**tag**）与该cache行的区地址（**tag**），因此只需要一个比较器（1路比较），**硬件成本低**，适合于大容量cache使用
- ④ 无须使用复杂的替换算法，**直接替换**冲突的数据块即可

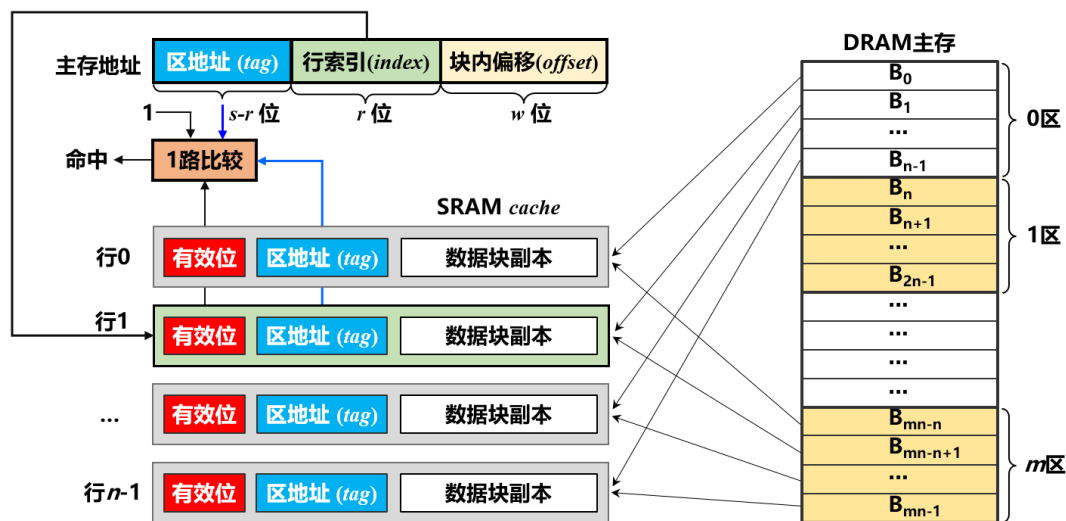


图4.38 直接相联映射的逻辑示意图

- 例4.3 某计算机字长32位，采用直接相联映射方式；主存容量4MB，按字节编址；cache的数据块副本缓冲区容量为4KB，cache块长度为8个字。
- ① 画出直接映射方式下主存地址划分情况，并说明每个字段位数。
- ② 设cache初始状态为空，若CPU顺序访问0-99号单元（字），从中读出100个字（假设主存一次读一个字），并重复此顺序10次，请计算cache的命中率。
- ③ 如果cache的存取时间是20ns，主存的访问时间是200ns，请问平均访问时间是多少？
- ④ cache-主存系统访问效率是多少？

• 解：

– ①

- 直接相联映射方式：主存地址 = 区地址（tag）+ 行索引（index）+ 块内偏移（offset）；
cache地址 = 块地址 + 块内偏移（offset）
- 主存容量=4MB，因此主存地址=22位（ $2^{22}=4\text{M}$ ）；cache的数据块副本缓冲区容量=4KB，因此cache地址=12位（ $2^{12}=4\text{K}$ ）
- cache块长度为8个字=8x32位=32B（字长=32位），因此offset=5位（ $2^5=32$ ），cache块地址=12-5=7位
- 因为cache的块地址与主存的行索引相等，因此：index=7位
- 主存的区地址=22-7-5=10位

内存按字节编址

$$4\text{MB}/4\text{KB}=1024=2^{10}$$

$$4\text{KB}/(8 \times 4\text{B})=2^7$$

按字节寻址，8个字相当于32个字节，所以 $32=2^5$

- 因此，主存地址（22位）= 区地址（10位）+ 行索引（7位）+ 块内偏移（5位）

- 例4.3 某计算机字长32位，采用**直接相联映射**方式；主存容量4MB，按字节编址；cache的数据块副本缓冲区容量为4KB，cache块长度为8个字。
 - ① 画出直接映射方式下主存地址划分情况，并说明每个字段位数。
 - ② 设cache初始状态为空，若CPU顺序访问0-99号单元（字），从中读出100个字（假设主存一次读一个字），并重复此顺序10次，请计算cache的命中率。
 - ③ 如果cache的存取时间是20ns，主存的访问时间是200ns，请问平均访问时间是多少？
 - ④ cache-主存系统访问效率是多少？

• 解（续）：

– ②

- cache的每块有8个字（单元）；一开始cache是空的，CPU访问0号单元将**不命中**，并主存的将0~7号单元装入cache中，因此CPU访问1~7号单元将命中
- 之后，CPU访问8号单元**不命中**，访问9~15号单元命中；CPU访问16号单元**不命中**，访问17~23号单元命中；CPU访问24号单元**不命中**，访问25~31号单元命中；CPU访问32号单元**不命中**，访问33~39号单元命中；CPU访问40号单元**不命中**，访问41~47号单元命中；CPU访问48号单元**不命中**，访问49~55号单元命中；CPU访问56号单元**不命中**，访问57~63号单元命中；CPU访问64号单元**不命中**，访问65~71号单元命中；CPU访问72号单元**不命中**，访问73~79号单元命中；CPU访问80号单元**不命中**，访问81~87号单元命中；CPU访问88号单元**不命中**，访问89~95号单元命中；CPU访问96号单元**不命中**，访问97~99号单元命中
- 接下去，CPU重复访问9次时，全部命中
- 因此，1000次访问，有13次不命中，其余987次都命中，命中率=987/1000=98.7%

cache数据块副本缓冲区

	第0字	第1字	第2字	第3字	第4字	第5字	第6字	第7字
第0行	0	1	2	3	4	5	6	7
第1行	8	9	10	11	12	13	14	15
第2行	16	17	18	19	20	21	22	23
第3行	24	25	26	27	28	29	30	31
第4行	32	33	34	35	36	37	38	39
第5行	40	41	42	43	44	45	46	47
第6行	48	49	50	51	52	53	54	55
第7行	56	57	58	59	60	61	62	63
第8行	64	65	66	67	68	69	70	71
第9行	72	73	74	75	76	77	78	79
第10行	80	81	82	83	84	85	86	87
第11行	88	89	90	91	92	93	94	95
第12行	96	97	98	99				
...								
...								
...								
第127行								

- 例4.3 某计算机字长32位，采用直接相联映射方式；主存容量4MB，按字节编址；cache的数据块副本缓冲区容量为4KB，cache块长度为8个字。
 - ① 画出直接映射方式下主存地址划分情况，并说明每个字段位数。
 - ② 设cache初始状态为空，若CPU顺序访问0-99号单元（字），从中读出100个字（假设主存一次读一个字），并重复此顺序10次，请计算cache的命中率。
 - ③ 如果cache的存取时间是20ns，主存的访问时间是200ns，请问平均访问时间是多少？
 - ④ cache-主存系统访问效率是多少？

• 解（续）：

– ③

• 平均访问时间： $t_a = ht_c + (1-h)t_m$

• 其中， $t_c = 20\text{ns}$ ， $t_m = 200\text{ns}$ ， $h = 0.987$

• 因此， $t_a = 0.987 \times 20\text{ns} + (1-0.987) \times 200\text{ns} = 22.34\text{ns}$

– ④

• 访问效率： $e = t_c / t_a = 20\text{ns} / 22.34\text{ns} = 89.5\%$

- 例4.4 主存地址空间大小为256MB，按字节编址。指令cache和数据cache分离，均有8行，每个cache行的数据块大小均为64B。数据cache采用直接相联映射方式。现有两个功能相同的程序A和B，其伪代码如下所示。假定int型数据为32位补码，程序编译时i、j、sum均分配在寄存器中，数组a[][]按行优先方式存放，首地址为320（十进制）。请回答如下问题，要求说明理由或给出计算过程：
 - ① 若不考虑用于cache一致性维护和替换算法的控制位，则数据cache的总容量是多少？
 - ② 数组元素a[0][31]、a[1][1]所在主存块对应的cache行（块）分别是多少？假设cache行（块）号从零开始。
 - ③ 程序A和B的数据访问命中率各是多少？那个程序的执行时间更短？

程序A:

```
int a[256][256];
int sum_array1()
{
    int i,j,sum=0;
    for (i = 0; i < 256; i++)
        for (j = 0; j < 256; j++)
            sum += a[i][j];
    return sum;
}
```

利用空间
局部性hit

程序B:

```
int a[256][256];
int sum_array2()
{
    int i,j,sum=0;
    for (j = 0; j < 256; j++)
        for (i = 0; i < 256; i++)
            sum += a[i][j];
    return sum;
}
```

i,j 行列互换
导致miss

- **例4.4** 主存地址空间大小为**256MB**，按**字节**编址。指令**cache**和数据**cache**分离，均有**8**行，每个**cache**行的数据块大小均为**64B**。数据**cache**采用直接相联映射方式。现有两个功能相同的程序**A**和**B**，其伪代码如下所示。假定**int**型数据为**32**位补码，程序编译时**i**、**j**、**sum**均分配在寄存器中，数组**a**按行优先方式存放，首地址为**320**（十进制）。请回答如下问题，要求说明理由或给出计算过程：

- ① 若不考虑用于**cache**一致性维护和替换算法的控制位，则数据**cache**的总容量是多少？
- ② 数组元素**a[0][31]**、**a[1][1]**所在主存块对应的**cache**行分别是多少？假设**cache**行号从零开始。
- ③ 程序**A**和**B**的数据访问命中率各是多少？那个程序的执行时间更短？

• 解：

– ①

- 直接相联映射：主存地址 = 区地址tag + 行索引index + 块内偏移offset
- 主存容量=256MB，主存地址=28位（ $2^{28}=256M$ ）
- **cache**有8行，因此index=3（ $2^3=8$ ）
- 每个**cache**行的数据块大小=64B，故offset=6（ $2^6=64$ ）
- 因此，tag=28-3-6=19
- 每个**cache**行包括：valid（1位）+ tag（19位）+ 数据块大小（64B）=20+64x8=532位
=(532/8)字节 =66.5B
- 数据**cache**总容量=cache行数 x 行大小=8x66.5B=532B

- 例4.4 主存地址空间大小为256MB，按字节编址。指令cache和数据cache分离，均有8行，每个cache行的数据块大小均为64B。数据cache采用直接相联映射方式。现有两个功能相同的程序A和B，其伪代码如下所示。假定int型数据为32位补码，程序编译时i、j、sum均分配在寄存器中，数组a按行优先方式存放，首地址为320（十进制）。请回答如下问题，要求说明理由或给出计算过程：

- ① 若不考虑用于cache一致性维护和替换算法的控制位，则数据cache的总容量是多少？
- ② 数组元素a[0][31]、a[1][1]所在主存块对应的cache行分别是多少？假设cache行号从零开始。
- ③ 程序A和B的数据访问命中率各是多少？那个程序的执行时间更短？

（续）：

– ②

- 数组按行优先方式存放，首地址为320（十进制），每个数组元素占32位（int型数据，32位=4个字节），则二维数组a在内存中的分布如下：

内存地址	320	324	328	332	336		444		1340	1344	1348
行优先	a[0][0]	a[0][1]	a[0][2]	a[0][3]	a[0][4]		a[0][31]		a[0][255]	a[1][0]	a[1][1]

- a[0][31]所在的内存地址=320 + 31x4 = 444；对应的主存块号=[444/64]=6
- a[1][1]所在的内存地址=320 + 256x4 + 4 = 1348；对应的主存块号=[1348/64]=21

– []表示取整

块大小
为64B

- 因为采用直接相联映射方式，主存块号与cache行号的对应关系为：
 - cache行号 = 主存块号 mod n
 - n=8（cache有8行）
- a[0][31]所在主存块对应的cache行是：6 mod 8 = 6
- a[1][1]所在主存块对应的cache行是：21 mod 8 = 5

- 例4.4 主存地址空间大小为256MB，按字节编址。指令cache和数据cache分离，均有8行，每个cache行的数据块大小均为64B。数据cache采用直接相联映射方式。现有两个功能相同的程序A和B，其伪代码如下所示。假定int型数据为32位补码，程序编译时i、j、sum均分配在寄存器中，数组a按行优先方式存放，首地址为320（十进制）。请回答如下问题，要求说明理由或给出计算过程：

- ① 若不考虑用于cache一致性维护和替换算法的控制位，则数据cache的总容量是多少？
- ② 数组元素a[0][31]、a[1][1]所在主存块对应的cache行分别是多少？假设cache行号从零开始。
- ③ 程序A和B的数据访问命中率各是多少？那个程序的执行时间更短？

解（续）：

— ③

- 每个cache行的数据块大小为64B，其可以存放 $64B/32\text{位}=64\times 8/32 = 16$ 个int型数据
- 程序A中数据的访问顺序与存储顺序相同（行优先方式），第1次访问的数组元素是a[0][0]，缺失，需要载入数据块（载入16个int型数据：a[0][0] ~ a[0][15]），后续15次访问均命中；程序A的所有数据访问都符合这一规律；因此命中率=15/16= 93.75%
- 程序B按照数组的列执行外层循环，其数据访问顺序与存储顺序不同；第1次访问的数组元素是a[0][0]，缺失，需要载入数据块（载入16个int型数据：a[0][0] ~ a[0][15]）；第2次访问的数组元素是a[1][0]，仍然是缺失；程序B的所有数据访问都符合这一规律，均为缺失；因此命中率=0

存储顺序（行优先）	a[0][0]	a[0][1]	a[0][2]	a[0][3]	a[0][4]		a[0][31]		a[0][255]	a[1][0]	a[1][1]
访问顺序（行优先）	a[0][0]	a[0][1]	a[0][2]	a[0][3]	a[0][4]		a[0][31]		a[0][256]	a[1][0]	a[1][1]
存储顺序（行优先）	a[0][0]	a[0][1]	a[0][2]	a[0][3]	a[0][4]		a[0][31]		a[0][255]	a[1][0]	a[1][1]
访问顺序（列优先）	a[0][0]	a[1][0]	a[2][0]	a[3][0]	a[4][0]		a[31][0]		a[256][0]	a[0][1]	a[1][1]

4.5.6 地址映射

– 3、组相联映射

- (1) 组相联映射基本原理

- 全相联映射命中率高，但是查找电路硬件成本也高；直接相联映射查找电路硬件成本低，但是命中率也低
- 组相联映射是直接相联映射和全相联映射的折中，既能提高命中率，又能降低查找电路的硬件成本
- 组相联映射将cache分成若干个固定大小的组，每个组有k行，称为k-路组相联
- 主存数据块首先采用直接相联映射方式映射到cache的某一组，然后按照全相联映射方式映射到该组内的任意一行
- 假设主存块号为j，cache组号为i，cache共有n组，则有： $i = j \bmod n$
- 主存地址 = 标记 (tag) + 组索引 (index) + 块内偏移 (offset)

4.5.6 地址映射

- 地址映射过程：根据主存地址中的组索引（index）找到cache的对应组，将主存地址中的标记（tag）与该cache组的所有行的标记（tag）进行比较，比较相等且有效位=1则为命中，否则为缺失（没有一个比较相等，或者是比较相等但有效位=0）

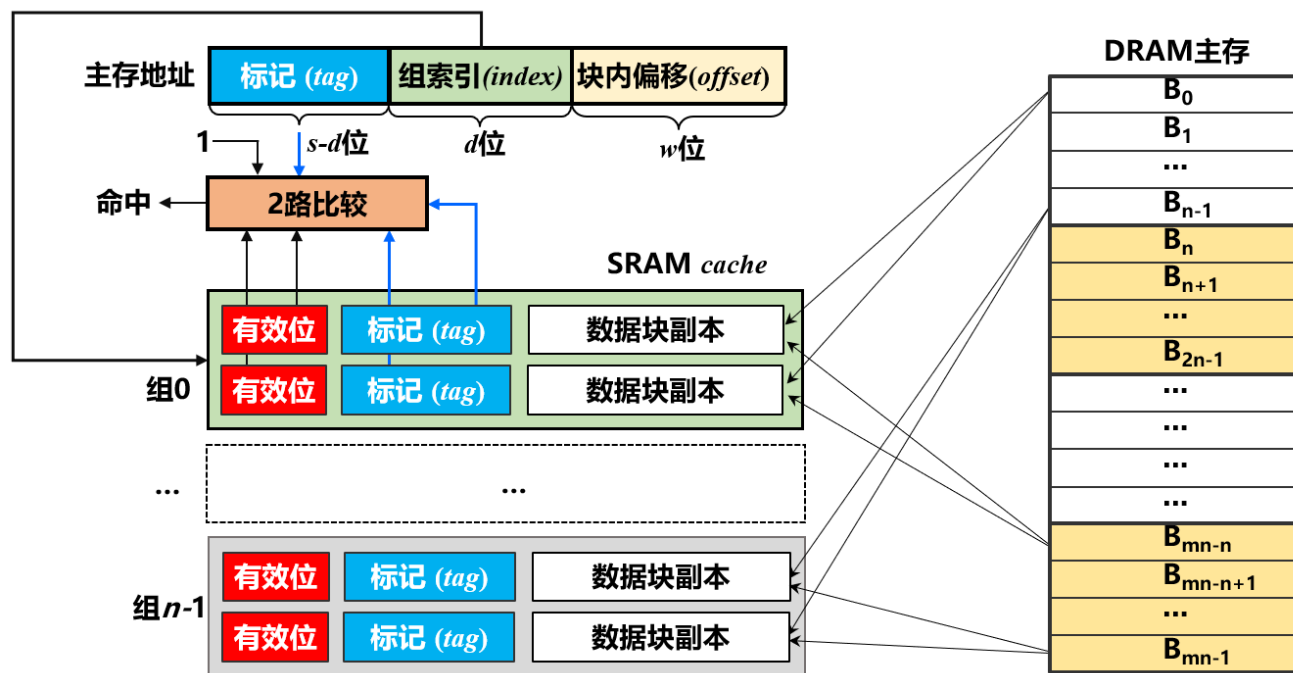


图4.41 2路组相联映射逻辑示意图

主存地址 = 标记(tag) + 组索引(index) + 块内偏移(offset)

cache每一行包括：有效位、标记(tag)、数据块副本

- (2) 组相联映射硬件实现

- 假设：主存地址（9位）= 标记tag（5位）+ 组索引index（2位）+ 块内偏移offset（2位）
- 组索引为2位，因此cache有4组（ $2^2=4$ ）；2路组相联映射，cache每组有2行，cache共有8行；块内偏移为2位，因此cache的每一行数据块副本缓冲区中有4个字（ $2^2=4$ ）

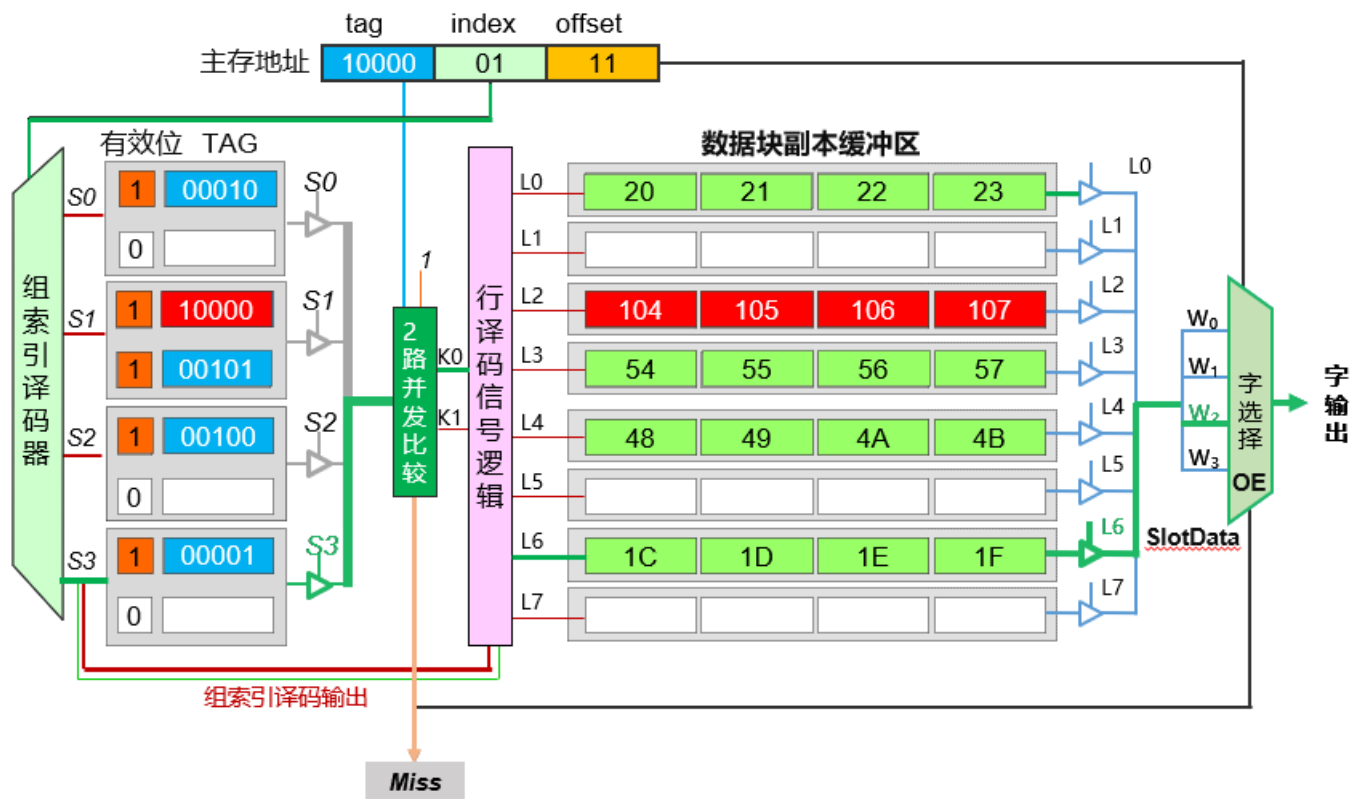


图4.42 2路组相联映射的硬件逻辑实现框图（该图只有cache，没有主存）

- 现要求访问主存地址（十六进制）分别为**1F**、**20**、**24**、**48**、**54**、**55**、**103**的7个字，请给出2路组相联映射的访问过程
- 访问主存地址为**1F**的字时，因为**cache**初始态为空，访问缺失，需要将主存地址为**1F**所在的块（4个字，地址分别为：**1C**、**1D**、**1E**、**1F**）**载入**到**cache**的第3组（**1F**对应的组索引=3）的任意一行（比如第6行），并将该行**cache**的**tag**置为主存的**tag**（**00001**）、**valid**置为**1**；访问主存地址为**20**、**24**、**48**、**54**、**103**的字的访问过程，与访问主存地址为**1F**的字的访问过程类似，都属于**载入**
- 访问主存地址为**55**的字时，根据其组索引（**55**对应的组索引=1）找到**cache**的第1组，并比较主存的**tag**（**00101**）与该组2行**cache**的**tag**，结果与第3行相同，且有效位**valid**=1，表示**命中**

表4.7 组相联映射的访问过程

访问序列	1	2	3	4	5	6	7
主存地址 (十六进制)	1F	20	24	48	54	55	103
主存地址 (二进制)	00001 11 11	00010 00 00	00010 01 00	00100 10 00	00101 01 00	00101 01 01	10000 00 11
主存地址 (tag,index,offset) (十六进制)	(1, 3, 3)	(2, 0, 0)	(2, 1, 0)	(4, 2, 0)	(5, 1, 0)	(5, 1, 1)	(10, 0, 3)
cache组/行	3/6	0/0	1/2	2/4	1/3	1/3	0/1
访问情况	载入	载入	载入	载入	载入	命中	载入

- 比较表4.7（组相联映射）和表4.5（直接相联映射），组相联映射没有发生替换，其**cache利用率**比直接相联映射**有所提高**
- 比较图4.41（组相联映射）和图4.36（全相联映射），组相联映射只需要**2路**并发比较，而全相联映射需要**8路**并发比较，组相联映射的**硬件成本更低**
- 当**k=1**时，即**cache**的每组只有**1行**（**1-路组相联**），此时组相联映射演变为直接相联映射
- 当**n=1**时，即**cache**只有**1组**，此时组相联映射演变为全相联映射

访问序列	1	2	3	4	5	6	7
主存地址 (十六进制)	1F	20	24	48	54	55	103
主存地址 (二进制)	00001 11 11	00010 00 00	00010 01 00	00100 10 00	00101 01 00	00101 01 01	10000 00 11
主存地址 (tag, index, offset) (十六进制)	(1, 3, 3)	(2, 0, 0)	(2, 1, 0)	(4, 2, 0)	(5, 1, 0)	(5, 1, 1)	(10, 0, 3)
Cache组/行	3/6	0/0	1/2	2/4	1/3	1/3	0/1
访问情况	载入	载入	载入	载入	载入	命中	载入

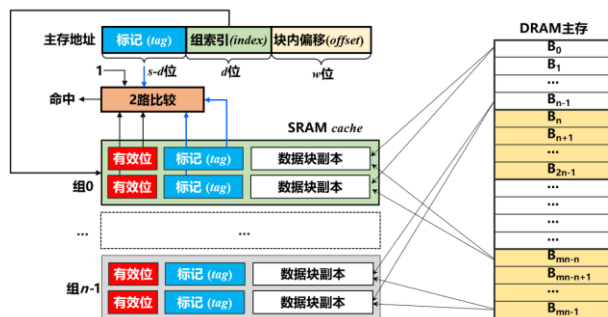


图4.41

访问序列	1	2	3	4	5	6	7
主存地址 (十六进制)	1F	20	24	48	54	55	103
主存地址 (二进制)	0000 111 11	0001 000 00	0001 001 00	0010 010 00	0010 101 00	0010 101 01	1000 000 11
主存地址 (tag, index, offset) (十六进制)	(0, 7, 3)	(1, 0, 0)	(1, 1, 0)	(2, 2, 0)	(2, 5, 0)	(2, 5, 1)	(8, 0, 3)
cache行	0	1	2	3	4	4	5
访问情况	载入	载入	载入	载入	载入	命中	替换

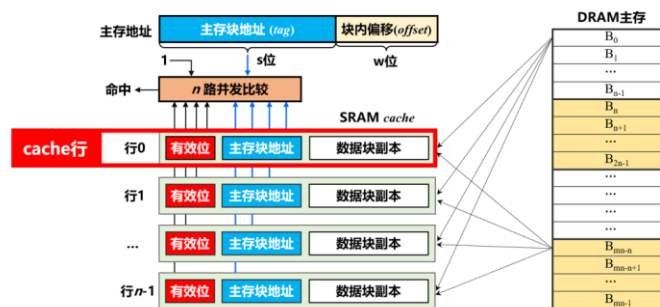


图4.36

– 4、不同映射方式比较

- 主存地址：

- ① 全相联映射：主存地址 = 主存块地址（ s 位） + 块内偏移（ w 位）

- ② 直接相联映射：主存地址 = 区地址（ $s-r$ 位） + 行索引（ r 位） + 块内偏移（ w 位）

- ③ 组相联映射：主存地址 = 标记（ $s-d$ 位） + 组索引（ d 位） + 块内偏移（ w 位）

- 并且： $s > r > d$ ；当 $d=0$ ，组相联映射演变为全相联映射；当 $d=r$ ，组相联映射演变为直接相联映射

- 直接相联映射：查找容易（主存的块对应cache的唯一块），淘汰简单，但命中率较低

- 全相联映射：查找时并发比较的硬件成本最高，淘汰算法复杂，但命中率较高

- 组相联映射：兼具直接相联映射和全相联映射的优势，既能提高命中率，又能降低查找电路的硬件成本



全相联映射

主存地址 = 主存块地址（**10位**）+ 块内偏移（**w位**）

cache地址 = cache块地址（**5位**）+ 块内偏移（**w位**）

全相联映射

主存 (块)	cache (块)
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	0
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	1
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	2
.....	...
.....	...
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	30
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	31

直接相联映射

主存地址 = 区地址（**5位**）+ 行索引（**5位**）+ 块内偏移（**w位**）

cache地址 = cache块地址（**5位**）+ 块内偏移（**w位**）

直接相联映射

主存 (块)	cache (块)
0、32、64、96、128、160、192、224、.....、960、992	0
1、33、65、97、129、161、193、225、.....、961、993	1
2、34、66、98、130、162、194、226、.....、962、994	2
.....	...
.....	...
30、62、94、126、158、190、222、254、.....、990、1022	30
31、63、95、127、159、191、223、255、.....、991、1023	31

2路组相联映射

主存地址 = 标记（**6位**）+ 组索引（**4位**）+ 块内偏移（**w位**）

cache地址 = cache块地址（**5位**）+ 块内偏移（**w位**）

2路组相联映射

主存 (块)	cache	
	组	块
0、16、32、48、64、80、96、.....、992、1008	0	0、1
1、17、33、49、65、81、97、.....、993、1009	1	2、3
2、18、34、50、66、82、98、.....、994、1010	2	4、5
.....	...	...
.....	...	...
14、30、46、62、78、94、110、.....、1006、1022	14	28、29
15、31、47、63、79、95、111、.....、1007、1023	15	30、31

4.5.7 替换算法

- 替换算法也称为淘汰算法
- 如果cache中已经装满数据，当新的数据块要载入时，必须从cache中替换（淘汰）掉一个数据块
- 如果淘汰掉的数据块中存在脏数据（**Dirty Data**），还需要将淘汰的数据同步到主存中，脏数据，即：cache的数据与主存数据不一致
- 常用的替换算法有4种：
 - ① 先进先出算法 **FIFO**
 - ② 最不经常使用算法 **LFU**
 - ③ 近期最少使用算法 **LRU**
 - ④ 随机替换算法

4.5.7 替换算法

– 1、先进先出算法

- 先进先出：First In First Out, FIFO
- FIFO算法按照数据块进入cache的先后决定替换的顺序，在需要进行替换时，选择最先被载入cache的行（数据块）进行替换
- FIFO算法需要记录每个数据块载入cache的时间戳或时间计数，以方便替换时比较先后顺序
- FIFO算法的优点：系统开销较小
- FIFO算法的缺点：没有考虑程序访问的局部性，可能会把一些需要经常使用的块（如循环程序块）也作为最早进入cache的块而替换掉，从而导致cache的命中率不高

4.5.7 替换算法

– 2、最不经常使用算法

- 最不经常使用：Least Frequently Used, LFU
- LFU算法将访问次数最少的cache行（数据块）淘汰掉
- cache的每行设置一个计数器，每访问该行一次，计数器加1；当需要替换时，对所有可淘汰行的计数值进行比较，将计数值最小的行淘汰掉
- LFU算法的不足：计数器记录的是cache工作后的历史访问统计情况，并不能严格反映近期访问情况
- 例如，cache有A和B两行，A行在前期多次被访问，而后期未被访问，但是其累计访问次数值很多；B行在前期不常用，而后期正被频繁访问，但是其累计访问次数值小于A行，因而被LFU算法淘汰

4.5.7 替换算法

— 3、近期最少使用算法

- 近期最少使用：Least Recently Used, LRU
- LRU算法将近期内最久未被访问过的cache行淘汰掉
- cache的每行设置一个计数器，当访问某cache行时，该cache行的计数器清0，其它cache行的计数器加1；因此计数器是用于统计未被访问的次数
- 当需要替换时，比较所有可淘汰行的计数值，将计数值最大的行淘汰掉（即近期最久未被访问的行）
- LRU算法保护了刚载入cache的新数据，符合cache的工作原理（程序局部性），使cache具有较高的命中率
- LRU算法的缺点：需要快速比较多个cache行的计数器值，硬件实现比较复杂

4.5.7 替换算法

— 4、随机替换算法

- 随机替换就是在需要进行替换时，从所有可淘汰行中，随机选取一行进行替换
- 优点是实现最容易，而且替换速度也比前面三种算法快
- 缺点是随机淘汰的数据，很可能马上又要使用，从而降低cache的命中率和工作效率
- 模拟研究表明，随机替换算法的性能只是比LFU和LRU稍差一些
- 在虚拟存储器的TLB表中，为了提高替换速度，就采用了随机替换算法
- TLB: Translation Lookaside Buffer，地址转换后援缓冲器，简称“快表”，为CPU的一种缓存，由存储器管理单元用于改进虚拟地址到物理地址的转换速度

4.5.7 替换算法

- 替换算法与cache的组织方式紧密相关
 - ① 直接相联映射方式：一个主存块对应唯一的cache行，因此不需要替换算法
 - ② 全相联映射方式：一个主存块可以放置到cache的任意一行，因此需要有替换算法
 - ③ 组相联映射方式：一个主存块可以放置到唯一的cache组中的任意一行，也需要有替换算法，只是这时是在一个cache组中替换某一行

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（1）

- **LFU算法**：cache的每行设置一个计数器，每访问该行一次，计数器加1；当需要替换时，对所有可淘汰行的计数值进行比较，将计数值最小的行淘汰掉

— **LFU算法的访问过程**：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ⁰	1 ⁰	1 ⁰	1 ¹	1 ¹	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²
cache 第1行		2 ⁰	2 ⁰	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	5 ⁰	6 ⁰	7 ⁰	5 ⁰	6 ⁰	7 ⁰	5 ⁰
cache 第2行			3 ⁰	3 ⁰	3 ⁰	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹
cache 第3行								4 ⁰	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（1）

- LFU算法：cache的每行设置一个计数器，每访问该行一次，计数器加1；当需要替换时，对所有可淘汰行的计数值进行比较，将计数值最小的行淘汰掉

- LFU算法的访问过程：

前3次分别访问数据块1、2、3，
因为cache是空的，因此需要载入

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ⁰	1 ⁰	1 ⁰	1 ¹	1 ¹	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²
cache 第1行		2 ⁰	2 ⁰	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	5 ⁰	6 ⁰	7 ⁰	5 ⁰	6 ⁰	7 ⁰	5 ⁰
cache 第2行			3 ⁰	3 ⁰	3 ⁰	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹
cache 第3行								4 ⁰	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（1）

- LFU算法：cache的每行设置一个计数器，每访问该行一次，计数器加1；当需要替换时，对所有可淘汰行的计数值进行比较，将计数值最小的行淘汰掉

- LFU算法的访问过程：

第4次访问数据块2，cache中有数据块2，为命中，同时将对数据块2的访问次数加1（2¹）

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ⁰	1 ⁰	1 ⁰	1 ¹	1 ¹	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²
cache 第1行		2 ⁰	2 ⁰	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	5 ⁰	6 ⁰	7 ⁰	5 ⁰	6 ⁰	7 ⁰	5 ⁰
cache 第2行			3 ⁰	3 ⁰	3 ⁰	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹
cache 第3行								4 ⁰	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（1）

- LFU算法：cache的每行设置一个计数器，每访问该行一次，计数器加1；当需要替换时，对所有可淘汰行的计数值进行比较，将计数值最小的行淘汰掉

- LFU算法的访问过程：

第5次访问数据块1，cache中有数据块1，为命中，同时将数据块1的访问次数加1（1¹）

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ⁰	1 ⁰	1 ⁰	1 ¹	1 ¹	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²
cache 第1行		2 ⁰	2 ⁰	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	5 ⁰	6 ⁰	7 ⁰	5 ⁰	6 ⁰	7 ⁰	5 ⁰
cache 第2行			3 ⁰	3 ⁰	3 ⁰	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹
cache 第3行								4 ⁰	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（1）

- LFU算法：cache的每行设置一个计数器，每访问该行一次，计数器加1；当需要替换时，对所有可淘汰行的计数值进行比较，将计数值最小的行淘汰掉

- LFU算法的访问过程：

第6次访问数据块3，cache中有数据块3，为命中，同时将数据块3的访问次数加1（3¹）

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ⁰	1 ⁰	1 ⁰	1 ¹	1 ¹	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²
cache 第1行		2 ⁰	2 ⁰	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	5 ⁰	6 ⁰	7 ⁰	5 ⁰	6 ⁰	7 ⁰	5 ⁰
cache 第2行			3 ⁰	3 ⁰	3 ⁰	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹
cache 第3行								4 ⁰	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（1）

- LFU算法：cache的每行设置一个计数器，每次访问时，对访问的数据块计数器加1。替换时，对所有可淘汰行的计数值进行比较，将计数值最小的数据块淘汰。
- LFU算法的访问过程：

第7次访问数据块1，cache中有数据块1，为命中，同时将数据块1的访问次数再加1（ 1^2 ）

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1^0	1^0	1^0	1^0	1^1	1^1	1^2	1^2	1^2	1^2	1^2	1^2	1^2	1^2	1^2	1^2
cache 第1行		2^0	2^0	2^1	2^1	2^1	2^1	2^1	2^1	5^0	6^0	7^0	5^0	6^0	7^0	5^0
cache 第2行			3^0	3^0	3^0	3^1	3^1	3^1	3^1	3^1	3^1	3^1	3^1	3^1	3^1	3^1
cache 第3行								4^0	4^1	4^1	4^1	4^1	4^1	4^1	4^1	4^1
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（1）

- LFU算法：cache的每行设置一个计数器，每访问该行一次，计数器加1；当需要替换时，对所有可淘汰行的计数值进行比较，将计数值最小的行淘汰掉

- LFU算法的访问过程：

第8次访问数据块4，cache中没有，不命中，需要载入

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ⁰	1 ⁰	1 ⁰	1 ¹	1 ¹	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²
cache 第1行		2 ⁰	2 ⁰	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	5 ⁰	6 ⁰	7 ⁰	5 ⁰	6 ⁰	7 ⁰	5 ⁰
cache 第2行			3 ⁰	3 ⁰	3 ⁰	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹
cache 第3行								4 ⁰	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（1）

- LFU算法：cache的每行设置一个计数器，每访问该行一次，计数器加1；当需要替换时，对所有可淘汰行的计数值进行比较，将计数值最小的行淘汰。

- LFU算法的访问过程：

第9次访问数据块4，cache中有数据块4，为命中，同时将数据块4的访问次数加1（4¹）

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ⁰	1 ⁰	1 ⁰	1 ¹	1 ¹	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²
cache 第1行		2 ⁰	2 ⁰	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	5 ⁰	6 ⁰	7 ⁰	5 ⁰	6 ⁰	7 ⁰	5 ⁰
cache 第2行			3 ⁰	3 ⁰	3 ⁰	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹
cache 第3行								4 ⁰	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

- 例4.5：假设cache有4行，某程序要访问主存（1~7），共访问16次，访问次序为：1、2、3、4、5、6、7、5。请分析LFU和LRU算法的访问过程。

解：（1）

- LFU算法：cache的每行设置一个计数器，每访问时，对所有可淘汰行的计数值进行比较，将计数值最小的那一行淘汰。
- LFU算法的访问过程：

第10次要访问数据块5，cache中没有，需要载入，但是cache中已经满了，按照LFU算法淘汰计数值最小的那一行，第1行、第2行、第3行的计数值相同（ 2^1 、 3^1 、 4^1 ），都是最小，此时可采用随机替换算法或FIFO算法确定从3行中替换那一行，这里采用FIFO算法，因此替换第1行（数据块2最先载入）

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1^0	1^0	1^0	1^0	1^1	1^1	1^2	1^2	1^2	1^2	1^2	1^2	1^2	1^2	1^2	1^2
cache 第1行		2^0	2^0	2^1	2^1	2^1	2^1	2^1	2^1	5^0	6^0	7^0	5^0	6^0	7^0	5^0
cache 第2行			3^0	3^0	3^0	3^1	3^1	3^1	3^1	3^1	3^1	3^1	3^1	3^1	3^1	3^1
cache 第3行								4^0	4^1	4^1	4^1	4^1	4^1	4^1	4^1	4^1
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程。

解：（1）

- LFU算法：cache的每行设置一个计数器，每访问时，对所有可淘汰行的计数值进行比较，将

第11次要访问数据块6，cache中没有，需要载入，但是cache中已经满了，按照LFU算法淘汰计数值最小的那一行，第1行（5⁰）的计数值最小，因此替换第1行

- LFU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ⁰	1 ⁰	1 ⁰	1 ¹	1 ¹	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²
cache 第1行		2 ⁰	2 ⁰	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	5 ⁰	6 ⁰	7 ⁰	5 ⁰	6 ⁰	7 ⁰	5 ⁰
cache 第2行			3 ⁰	3 ⁰	3 ⁰	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹
cache 第3行								4 ⁰	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程。

解：（1）

- LFU算法：cache的每行设置一个计数器，每访问时，对所有可淘汰行的计数值进行比较，将

第12次要访问数据块7，cache中没有，需要载入，但是cache中已经满了，按照LFU算法淘汰计数值最小的那一行，第1行（6⁰）的计数值最小，因此替换第1行

- LFU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ⁰	1 ⁰	1 ⁰	1 ¹	1 ¹	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²
cache 第1行		2 ⁰	2 ⁰	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	5 ⁰	6 ⁰	7 ⁰	5 ⁰	6 ⁰	7 ⁰	5 ⁰
cache 第2行			3 ⁰	3 ⁰	3 ⁰	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹
cache 第3行								4 ⁰	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程。

解：（1）

- LFU算法：cache的每行设置一个计数器，每访问时，对所有可淘汰行的计数值进行比较，将

第13次要访问数据块5，cache中没有，需要载入，但是cache中已经满了，按照LFU算法淘汰计数值最小的那一行，第1行（7⁰）的计数值最小，因此替换第1行

- LFU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ⁰	1 ⁰	1 ⁰	1 ¹	1 ¹	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²
cache 第1行		2 ⁰	2 ⁰	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	5 ⁰	6 ⁰	7 ⁰	5 ⁰	6 ⁰	7 ⁰	5 ⁰
cache 第2行			3 ⁰	3 ⁰	3 ⁰	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹
cache 第3行								4 ⁰	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问：

第14次要访问数据块6，cache中没有，需要载入，但是cache中已经满了，按照LFU算法淘汰计数值最小的那一行，第1行（5⁰）的计数值最小，因此替换第1行

- 解：（1）
 - LFU算法：cache的每行设置一个计数器，每访问时，对所有可淘汰行的计数值进行比较，将
 - LFU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ⁰	1 ⁰	1 ⁰	1 ¹	1 ¹	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²
cache 第1行		2 ⁰	2 ⁰	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	5 ⁰	6 ⁰	7 ⁰	5 ⁰	6 ⁰	7 ⁰	5 ⁰
cache 第2行			3 ⁰	3 ⁰	3 ⁰	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹
cache 第3行								4 ⁰	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程。

解：（1）

- LFU算法：cache的每行设置一个计数器，每访问时，对所有可淘汰行的计数值进行比较，将

第15次要访问数据块7，cache中没有，需要载入，但是cache中已经满了，按照LFU算法淘汰计数值最小的那一行，第1行（6⁰）的计数值最小，因此替换第1行

- LFU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ⁰	1 ⁰	1 ⁰	1 ¹	1 ¹	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²
cache 第1行		2 ⁰	2 ⁰	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	5 ⁰	6 ⁰	7 ⁰	5 ⁰	6 ⁰	7 ⁰	5 ⁰
cache 第2行			3 ⁰	3 ⁰	3 ⁰	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹
cache 第3行								4 ⁰	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程。

解：（1）

- LFU算法：cache的每行设置一个计数器，每访问时，对所有可淘汰行的计数值进行比较，将

第16次要访问数据块5，cache中没有，需要载入，但是cache中已经满了，按照LFU算法淘汰计数值最小的那一行，第1行（7⁰）的计数值最小，因此替换第1行

- LFU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ⁰	1 ⁰	1 ⁰	1 ¹	1 ¹	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²
cache 第1行		2 ⁰	2 ⁰	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	5 ⁰	6 ⁰	7 ⁰	5 ⁰	6 ⁰	7 ⁰	5 ⁰
cache 第2行			3 ⁰	3 ⁰	3 ⁰	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹
cache 第3行								4 ⁰	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（1）

- LFU算法：cache的每行设置一个计数器，每访问该行一次，计数器加1；当需要替换时，对所有可淘汰行的计数值进行比较，将计数值最小的行淘汰掉
- LFU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ⁰	1 ⁰	1 ⁰	1 ¹	1 ¹	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²	1 ²
cache 第1行		2 ⁰	2 ⁰	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	2 ¹	5 ⁰	6 ⁰	7 ⁰	5 ⁰	6 ⁰	7 ⁰	5 ⁰
cache 第2行			3 ⁰	3 ⁰	3 ⁰	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹	3 ¹
cache 第3行								4 ⁰	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹	4 ¹
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	替换	替换	替换	替换

— LFU算法的命中率=5/16=31.25%

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- **LRU算法**：cache的每行设置一个计数器，当访问某cache行时，该cache行的计数器清0，其它cache行的计数器加1；因此计数器是用于统计未被访问的次数；当需要替换时，比较所有可淘汰行的计数值，将计数值最大的行淘汰掉
- **LRU算法的访问过程**：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ¹	1 ²	1 ³	1 ⁰	1 ¹	1 ⁰	1 ¹	1 ²	1 ³	1 ⁴	7 ⁰	7 ¹	7 ²	7 ⁰	7 ¹
cache 第1行		2 ⁰	2 ¹	2 ⁰	2 ¹	2 ²	2 ³	2 ⁴	2 ⁵	5 ⁰	5 ¹	5 ²	5 ⁰	5 ¹	5 ²	5 ⁰
cache 第2行			3 ⁰	3 ¹	3 ²	3 ⁰	3 ¹	3 ²	3 ³	3 ⁴	6 ⁰	6 ¹	6 ²	6 ⁰	6 ¹	6 ²
cache 第3行								4 ⁰	4 ⁰	4 ¹	4 ²	4 ³	4 ⁴	4 ⁵	4 ⁶	4 ⁷
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法：cache的每行设置一个计数器，当访问某cache行时，该cache行的计数器清0，其它cache行的计数器加1；因此计数器是用于统计未被访问的次数；当需要替换时，比较所有可淘汰行的计数值，将计数值最大的行淘汰掉

- LRU算法的访问过程：

第1次访问数据块1，因为cache中是空的，需要载入，并将数据块1的计数值清0（1⁰）

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ¹	1 ²	1 ³	1 ⁰	1 ¹	1 ⁰	1 ¹	1 ²	1 ³	1 ⁴	7 ⁰	7 ¹	7 ²	7 ⁰	7 ¹
cache 第1行		2 ⁰	2 ¹	2 ⁰	2 ¹	2 ²	2 ³	2 ⁴	2 ⁵	5 ⁰	5 ¹	5 ²	5 ⁰	5 ¹	5 ²	5 ⁰
cache 第2行			3 ⁰	3 ¹	3 ²	3 ⁰	3 ¹	3 ²	3 ³	3 ⁴	6 ⁰	6 ¹	6 ²	6 ⁰	6 ¹	6 ²
cache 第3行								4 ⁰	4 ⁰	4 ¹	4 ²	4 ³	4 ⁴	4 ⁵	4 ⁶	4 ⁷
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法： cache的每行设置一个计数器，当访问某cache行时，该cache行的计数器清0，其它cache行的计数器加1；因此计数器是用于统计未被访问的次数；当需要替换时，比较所有可

第2次访问数据块2，因为cache中是空的，需要载入，将数据块2的计数值清0（ 2^0 ），并将数据块1的计数值加1（ 1^1 ）

- LRU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1^0	1^1	1^2	1^3	1^0	1^1	1^0	1^1	1^2	1^3	1^4	7^0	7^1	7^2	7^0	7^1
cache 第1行		2^0	2^1	2^0	2^1	2^2	2^3	2^4	2^5	5^0	5^1	5^2	5^0	5^1	5^2	5^0
cache 第2行			3^0	3^1	3^2	3^0	3^1	3^2	3^3	3^4	6^0	6^1	6^2	6^0	6^1	6^2
cache 第3行								4^0	4^0	4^1	4^2	4^3	4^4	4^5	4^6	4^7
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法： cache的每行设置一个计数器，当访问某cache行时，该cache行的计数器清0，其它cache行的计数器加1；因此计数器是用于统计未被访问的次数；当需要替换时，比较所有可

第3次访问数据块3，因为cache中是空的，需要载入，将数据块3的计数值清0（ 3^0 ），并将数据块1、数据块2的计数值加1（ 1^2 、 2^1 ）

- LRU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1^0	1^1	1^2	1^3	1^0	1^1	1^0	1^1	1^2	1^3	1^4	7^0	7^1	7^2	7^0	7^1
cache 第1行		2^0	2^1	2^0	2^1	2^2	2^3	2^4	2^5	5^0	5^1	5^2	5^0	5^1	5^2	5^0
cache 第2行			3^0	3^1	3^2	3^0	3^1	3^2	3^3	3^4	6^0	6^1	6^2	6^0	6^1	6^2
cache 第3行								4^0	4^0	4^1	4^2	4^3	4^4	4^5	4^6	4^7
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法： cache的每行设置一个计数器，当访问某cache行时，该cache行的计数器清0，其它cache行的计数器加1；因此计数器是用于统计未被访问的次数；当需要替换时，比较所有可

第4次访问数据块2，cache中有，为命中，将数据块2的计数值清0（ 2^0 ），并将数据块1、数据块3的计数值加1（ 1^3 、 3^1 ）

- LRU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1^0	1^1	1^2	1^3	1^0	1^1	1^0	1^1	1^2	1^3	1^4	7^0	7^1	7^2	7^0	7^1
cache 第1行		2^0	2^1	2^0	2^1	2^2	2^3	2^4	2^5	5^0	5^1	5^2	5^0	5^1	5^2	5^0
cache 第2行			3^0	3^1	3^2	3^0	3^1	3^2	3^3	3^4	6^0	6^1	6^2	6^0	6^1	6^2
cache 第3行								4^0	4^0	4^1	4^2	4^3	4^4	4^5	4^6	4^7
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法：cache的每行设置一个计数器，当访问某cache行时，该cache行的计数器清0，其它cache行的计数器加1；因此计数器是用于统计未被访问的次数；当需要替换时，比较所有可

- LRU算法的访问过程：

第5次访问数据块1，cache中有，为命中，将数据块1的计数值清0（ 1^0 ），并将数据块2、数据块3的计数值加1（ 2^1 、 3^2 ）

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1^0	1^1	1^2	1^3	1^0	1^1	1^0	1^1	1^2	1^3	1^4	7^0	7^1	7^2	7^0	7^1
cache 第1行		2^0	2^1	2^0	2^1	2^2	2^3	2^4	2^5	5^0	5^1	5^2	5^0	5^1	5^2	5^0
cache 第2行			3^0	3^1	3^2	3^0	3^1	3^2	3^3	3^4	6^0	6^1	6^2	6^0	6^1	6^2
cache 第3行								4^0	4^0	4^1	4^2	4^3	4^4	4^5	4^6	4^7
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法：cache的每行设置一个计数器，当访问某一行时，该行的计数器清0，其它cache行的计数器加1。需要替换时，比较所有可替换行的计数器，将计数器最大的行替换。
 - 第6次访问数据块3，cache中有，为命中，将数据块3的计数值清0（3⁰），并将数据块1、数据块2的计数值加1（1¹、2²）
- LRU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ¹	1 ²	1 ³	1 ⁰	1 ¹	1 ⁰	1 ¹	1 ²	1 ³	1 ⁴	7 ⁰	7 ¹	7 ²	7 ⁰	7 ¹
cache 第1行		2 ⁰	2 ¹	2 ⁰	2 ¹	2 ²	2 ³	2 ⁴	2 ⁵	5 ⁰	5 ¹	5 ²	5 ⁰	5 ¹	5 ²	5 ⁰
cache 第2行			3 ⁰	3 ¹	3 ²	3 ⁰	3 ¹	3 ²	3 ³	3 ⁴	6 ⁰	6 ¹	6 ²	6 ⁰	6 ¹	6 ²
cache 第3行								4 ⁰	4 ⁰	4 ¹	4 ²	4 ³	4 ⁴	4 ⁵	4 ⁶	4 ⁷
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法：cache的每行设置一个计数器，当访问某一行时，该行的计数器清0，其它cache行的计数器加1。需要替换时，比较所有可替换行的计数器，将计数器最大的行替换。
 - 第7次访问数据块1，cache中有，为命中，将数据块1的计数值清0（1⁰），并将数据块2、数据块3的计数值加1（2³、3¹）
- LRU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ¹	1 ²	1 ³	1 ⁰	1 ¹	1 ⁰	1 ¹	1 ²	1 ³	1 ⁴	7 ⁰	7 ¹	7 ²	7 ⁰	7 ¹
cache 第1行		2 ⁰	2 ¹	2 ⁰	2 ¹	2 ²	2 ³	2 ⁴	2 ⁵	5 ⁰	5 ¹	5 ²	5 ⁰	5 ¹	5 ²	5 ⁰
cache 第2行			3 ⁰	3 ¹	3 ²	3 ⁰	3 ¹	3 ²	3 ³	3 ⁴	6 ⁰	6 ¹	6 ²	6 ⁰	6 ¹	6 ²
cache 第3行								4 ⁰	4 ⁰	4 ¹	4 ²	4 ³	4 ⁴	4 ⁵	4 ⁶	4 ⁷
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法：cache的每行设置一个计数器，当访问某一行时，该行的计数器清0，其它cache行的计数器加1。需要替换时，比较所有可替换行的计数器，将计数器最大的行替换。第8次访问数据块4，cache中没有，需要载入，将数据块4的计数值清0（4⁰），并将数据块1、数据块2、数据块3的计数值加1（1¹、2⁴、3²）
- LRU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ¹	1 ²	1 ³	1 ⁰	1 ¹	1 ⁰	1 ¹	1 ²	1 ³	1 ⁴	7 ⁰	7 ¹	7 ²	7 ⁰	7 ¹
cache 第1行		2 ⁰	2 ¹	2 ⁰	2 ¹	2 ²	2 ³	2 ⁴	2 ⁵	5 ⁰	5 ¹	5 ²	5 ⁰	5 ¹	5 ²	5 ⁰
cache 第2行			3 ⁰	3 ¹	3 ²	3 ⁰	3 ¹	3 ²	3 ³	3 ⁴	6 ⁰	6 ¹	6 ²	6 ⁰	6 ¹	6 ²
cache 第3行								4 ⁰	4 ⁰	4 ¹	4 ²	4 ³	4 ⁴	4 ⁵	4 ⁶	4 ⁷
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法：cache的每行设置一个计数器，当访问某一行时，该行的计数器清0，其它cache行的计数器加1。需要替换时，比较所有可替换行的计数器，将计数器最大的行替换。
 - 第9次访问数据块4，cache中有，为命中，将数据块4的计数值清0（ 4^0 ），并将数据块1、数据块2、数据块3的计数值加1（ 1^2 、 2^5 、 3^3 ）
- LRU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1^0	1^1	1^2	1^3	1^0	1^1	1^0	1^1	1^2	1^3	1^4	7^0	7^1	7^2	7^0	7^1
cache 第1行		2^0	2^1	2^0	2^1	2^2	2^3	2^4	2^5	5^0	5^1	5^2	5^0	5^1	5^2	5^0
cache 第2行			3^0	3^1	3^2	3^0	3^1	3^2	3^3	3^4	6^0	6^1	6^2	6^0	6^1	6^2
cache 第3行								4^0	4^0	4^1	4^2	4^3	4^4	4^5	4^6	4^7
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法：cache的每行计数器清0，其它cache行的计数器需要替换时，比较所有可

第10次访问数据块5，cache中没有，需要替换，根据LRU算法，替换4个数据块中近期访问最少的那个数据块（ 1^2 、 2^5 、 3^3 、 4^0 ），即数据块2（ 2^5 ），同时将数据块5的计数值清0（ 5^0 ），将数据块1、3、4的计数值加1（ 1^3 、 3^4 、 4^1 ）

- LRU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1^0	1^1	1^2	1^3	1^0	1^1	1^0	1^1	1^2	1^3	1^4	7^0	7^1	7^2	7^0	7^1
cache 第1行		2^0	2^1	2^0	2^1	2^2	2^3	2^4	2^5	5^0	5^1	5^2	5^0	5^1	5^2	5^0
cache 第2行			3^0	3^1	3^2	3^0	3^1	3^2	3^3	3^4	6^0	6^1	6^2	6^0	6^1	6^2
cache 第3行								4^0	4^0	4^1	4^2	4^3	4^4	4^5	4^6	4^7
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法：cache的每行计数器清0，其它cache行的计数器需要替换时，比较所有可

第11次访问数据块6，cache中没有，需要替换，根据LRU算法，替换4个数据块中近期访问最少的那个数据块（ 1^3 、 5^0 、 3^4 、 4^1 ），即数据块3（ 3^4 ），同时将数据块6的计数值清0（ 6^0 ），将数据块1、5、4的计数值加1（ 1^4 、 5^1 、 4^2 ）

- LRU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1^0	1^1	1^2	1^3	1^0	1^1	1^0	1^1	1^2	1^3	1^4	7^0	7^1	7^2	7^0	7^1
cache 第1行		2^0	2^1	2^0	2^1	2^2	2^3	2^4	2^5	5^0	5^1	5^2	5^0	5^1	5^2	5^0
cache 第2行			3^0	3^1	3^2	3^0	3^1	3^2	3^3	3^4	6^0	6^1	6^2	6^0	6^1	6^2
cache 第3行								4^0	4^0	4^1	4^2	4^3	4^4	4^5	4^6	4^7
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法：cache的每行计数器清0，其它cache行的计数器需要替换时，比较所有可

第12次访问数据块7，cache中没有，需要替换，根据LRU算法，替换4个数据块中近期访问最少的那个数据块（ 1^4 、 5^1 、 6^0 、 4^2 ），即数据块1（ 1^4 ），同时将数据块7的计数值清0（ 7^0 ），将数据块5、6、4的计数值加1（ 5^2 、 6^1 、 4^3 ）

- LRU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1^0	1^1	1^2	1^3	1^0	1^1	1^0	1^1	1^2	1^3	1^4	7^0	7^1	7^2	7^0	7^1
cache 第1行		2^0	2^1	2^0	2^1	2^2	2^3	2^4	2^5	5^0	5^1	5^2	5^0	5^1	5^2	5^0
cache 第2行			3^0	3^1	3^2	3^0	3^1	3^2	3^3	3^4	6^0	6^1	6^2	6^0	6^1	6^2
cache 第3行								4^0	4^0	4^1	4^2	4^3	4^4	4^5	4^6	4^7
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法：cache的每行计数器清0，其它cache行的计数器需要替换时，比较所有可替换数据块的计数值，将计数值最小的数据块替换。第13次访问数据块5，cache中有，为命中，同时将数据块5的计数值清0（ 5^0 ），将数据块7、6、4的计数值加1（ 7^1 、 6^2 、 4^4 ）。

LRU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1^0	1^1	1^2	1^3	1^0	1^1	1^0	1^1	1^2	1^3	1^4	7^0	7^1	7^2	7^0	7^1
cache 第1行		2^0	2^1	2^0	2^1	2^2	2^3	2^4	2^5	5^0	5^1	5^2	5^0	5^1	5^2	5^0
cache 第2行			3^0	3^1	3^2	3^0	3^1	3^2	3^3	3^4	6^0	6^1	6^2	6^0	6^1	6^2
cache 第3行								4^0	4^0	4^1	4^2	4^3	4^4	4^5	4^6	4^7
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法：cache的每行计数器清0，其它cache行的计数器需要替换时，比较所有可
第14次访问数据块6，cache中有，为命中，同时将数据块6的计数值清0（ 6^0 ），将数据块7、5、4的计数值加1（ 7^2 、 5^1 、 4^5 ）
- LRU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1^0	1^1	1^2	1^3	1^0	1^1	1^0	1^1	1^2	1^3	1^4	7^0	7^1	7^2	7^0	7^1
cache 第1行		2^0	2^1	2^0	2^1	2^2	2^3	2^4	2^5	5^0	5^1	5^2	5^0	5^1	5^2	5^0
cache 第2行			3^0	3^1	3^2	3^0	3^1	3^2	3^3	3^4	6^0	6^1	6^2	6^0	6^1	6^2
cache 第3行								4^0	4^0	4^1	4^2	4^3	4^4	4^5	4^6	4^7
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法：cache的每行计数器清0，其它cache行的计数器需要替换时，比较所有可
第15次访问数据块7，cache中有，为命中，同时将数据块7的计数值清0（ 7^0 ），将数据块5、6、4的计数值加1（ 5^2 、 6^1 、 4^6 ）
- LRU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1^0	1^1	1^2	1^3	1^0	1^1	1^0	1^1	1^2	1^3	1^4	7^0	7^1	7^2	7^0	7^1
cache 第1行		2^0	2^1	2^0	2^1	2^2	2^3	2^4	2^5	5^0	5^1	5^2	5^0	5^1	5^2	5^0
cache 第2行			3^0	3^1	3^2	3^0	3^1	3^2	3^3	3^4	6^0	6^1	6^2	6^0	6^1	6^2
cache 第3行								4^0	4^0	4^1	4^2	4^3	4^4	4^5	4^6	4^7
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

解：（2）

- LRU算法：cache的每行计数器清0，其它cache行的计数器需要替换时，比较所有可替换数据块的计数值，将计数值最小的数据块替换。第16次访问数据块5，cache中有，为命中，同时将数据块5的计数值清0（ 5^0 ），将数据块7、6、4的计数值加1（ 7^1 、 6^2 、 4^7 ）。

LRU算法的访问过程：

地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1^0	1^1	1^2	1^3	1^0	1^1	1^0	1^1	1^2	1^3	1^4	7^0	7^1	7^2	7^0	7^1
cache 第1行		2^0	2^1	2^0	2^1	2^2	2^3	2^4	2^5	5^0	5^1	5^2	5^0	5^1	5^2	5^0
cache 第2行			3^0	3^1	3^2	3^0	3^1	3^2	3^3	3^4	6^0	6^1	6^2	6^0	6^1	6^2
cache 第3行								4^0	4^0	4^1	4^2	4^3	4^4	4^5	4^6	4^7
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- 例4.5：假设cache有4行，某程序要访问主存的7个数据块（数据块编号为：1~7），共访问16次，访问次序为：1、2、3、2、1、3、1、4、4、5、6、7、5、6、7、5。请分析LFU和LRU算法的访问过程，并计算命中率。

• 解：（2）

- LRU算法： cache的每行设置一个计数器，当访问某cache行时，该cache行的计数器清0，其它cache行的计数器加1；因此计数器是用于统计未被访问的次数；当需要替换时，比较所有可淘汰行的计数值，将计数值最大的行淘汰掉
- LRU算法的访问过程：

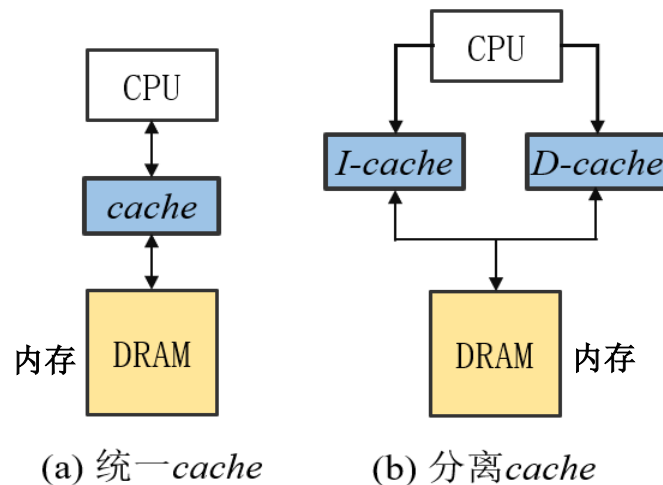
地址流 (访问次序)	1	2	3	2	1	3	1	4	4	5	6	7	5	6	7	5
cache 第0行	1 ⁰	1 ¹	1 ²	1 ³	1 ⁰	1 ¹	1 ⁰	1 ¹	1 ²	1 ³	1 ⁴	7 ⁰	7 ¹	7 ²	7 ⁰	7 ¹
cache 第1行		2 ⁰	2 ¹	2 ⁰	2 ¹	2 ²	2 ³	2 ⁴	2 ⁵	5 ⁰	5 ¹	5 ²	5 ⁰	5 ¹	5 ²	5 ⁰
cache 第2行			3 ⁰	3 ¹	3 ²	3 ⁰	3 ¹	3 ²	3 ³	3 ⁴	6 ⁰	6 ¹	6 ²	6 ⁰	6 ¹	6 ²
cache 第3行								4 ⁰	4 ⁰	4 ¹	4 ²	4 ³	4 ⁴	4 ⁵	4 ⁶	4 ⁷
命中情况	载入	载入	载入	命中	命中	命中	命中	载入	命中	替换	替换	替换	命中	命中	命中	命中

- LRU算法的命中率=9/16=56.25%；显然比LFU算法的命中率高（5/16=31.25%）

4.5.9 cache应用

– 1、分离cache

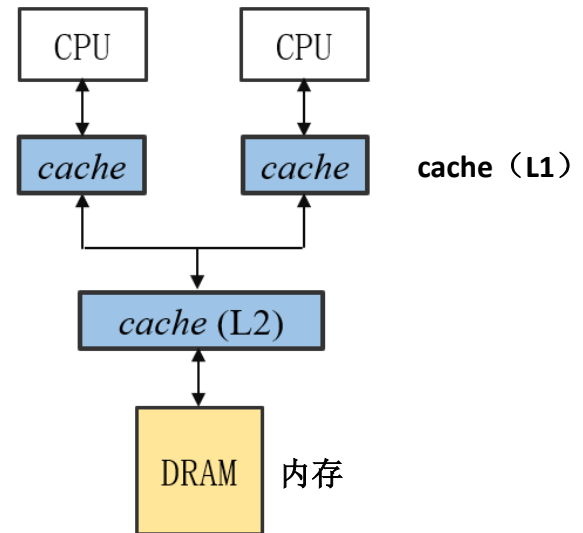
- 早期的cache中既存放指令又存放数据，称为**统一cache**
- 随着**指令预取**等新技术的出现，需要将指令cache（I-cache）和数据cache（D-cache）分开设计，称为**分离cache**
- 分离cache可以实现CPU在存取数据时（访问数据cache），同时完成取指令的功能（访问指令cache），从而提高CPU访问cache的速度



4.5.9 cache应用

– 2、多级cache

- 将cache和处理器集成在同一个芯片内，这个cache称为第一级cache (L1)
- 由于L1 cache在CPU芯片内，减少了对片外总线的访问，加快了存取的速度，提高了系统的性能
- 但是L1 cache的容量通常较小，此时可以在L1与主存之间再设置一个容量较大的第二级cache (L2)
- 随着技术的发展，还出现了L3 cache、L4 cache，称为多级cache

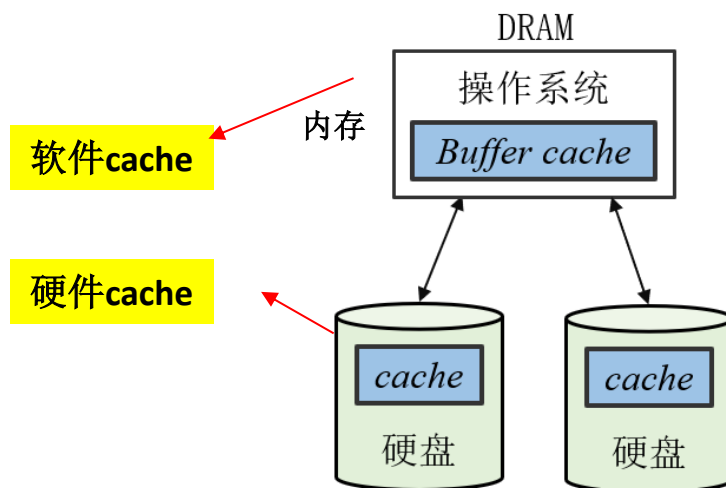


(c) 多级cache

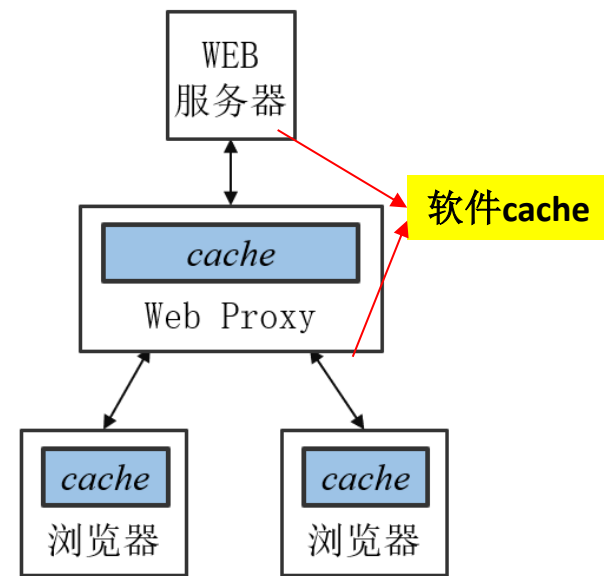
4.5.9 cache应用

– 3、软件cache

- 操作系统中的磁盘缓存buffer cache，其是通过软件方式实现的；磁盘缓存技术可以大大优化磁盘访问性能；此外在磁盘驱动器中也集成了硬件的缓存芯片（cache）
- 在Web访问中也使用了cache技术，包括Web代理服务器（Web Proxy）中的软件cache和本地浏览器中的软件cache



(d) Buffer cache和硬盘缓存芯片



(e) WEB cache

4.6 虚拟存储器

• 4.6.1 虚拟存储器的工作原理

- 存储系统设计的目标是设计一个访问速度快、存储容量大的存储系统
- cache技术可以大大提高主存的访问速度，但是并不能解决主存容量不足的问题
- 一方面，程序员总希望能够有一个大于主存空间的编程空间，这样编写程序时不受实际主存大小的限制
- 另一方面，多任务操作系统的计算机中有多个用户程序的进程在运行，每个进程都需要自己的独立地址空间
- 如何让更多的程序（程序的容量可能大于主存空间）在有限的主存空间中高效并发运行，并互不干扰，是虚拟存储器需要解决的问题
- [计算机体系结构-虚拟存储器 - 知乎 \(zhihu.com\)](#)
- 操作系统之虚拟存储器 - 知乎 <https://www.zhihu.com/zvideo/1618385883492835328>

4.6.1 虚拟存储器的工作原理

- 虚拟存储器是1961年英国曼彻斯特大学Kilburn等人提出的
- 虚拟存储器处于“主存-辅存”存储层次，是为了解决存储系统“存储容量大”的问题；cache处于“CPU-主存”存储层次，是为了解决存储系统“访问速度快”的问题
- 有了虚拟存储器，程序员可以使用虚拟地址（虚拟地址空间比主存的物理地址空间要大很多）进行编程，这样程序员的编程不再受实际主存空间大小的限制
- 虚拟存储器也充分利用了程序的局部性，采用按需加载的方式加载程序代码和数据；根据程序局部性原理，通常程序只需要加载很小一部分空间即可运行，这种方式避免了将程序全部载入内存，大大提高了主存的利用率
- 虚拟存储器也采用了和cache类似的技术，尽量将辅存中经常访问的程序和数据的副本调度到主存储器中，使得大部分数据都可以通过直接访问快速的主存获得
- 如果主存空间消耗完毕（内存满），此时虚拟存储器选择最不经常访问的程序和数据进行淘汰，然后载入新的数据

4.6.1 虚拟存储器的工作原理

- 淘汰时，也存在**主存与辅存数据一致性**的问题；对于静态代码和数据（这部分的代码和数据是不可以修改的），可以**直接丢弃**；对于动态数据（这部分的数据是可以修改的），在丢弃时，需要同时保存到辅助存储器中
- 总结：虚拟存储器的三个特性
 - **多次性**：指一个作业中的程序和数据允许被多次调入内存运行，即只需当前要运行的那部分程序和数据装入内存即可开始运行。
 - **对换性**：在进程运行期间，允许将那些暂不使用的代码和数据从内存调至外存的对换区，待以后需要时再将它们从外存调至内存。
 - **虚拟性**：是指能从逻辑上扩充内存容量

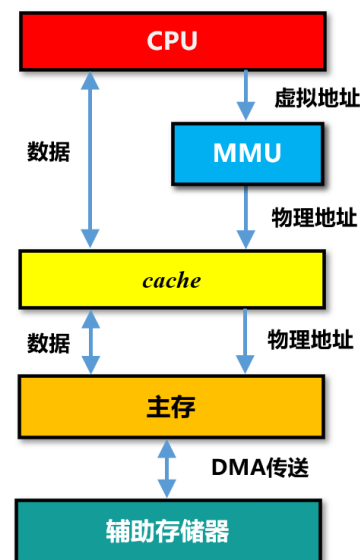


图4.45 虚拟存储器的典型组织结构

MMU: Memory Management Unit, 存储器管理单元

4.6.2 虚拟存储器的地址映射与变换

- 虚拟存储器中有**3种地址空间**:
 - ① 虚拟地址空间：也称为虚拟空间、虚地址空间，它是程序员用来编写程序的地址空间
 - ② 主存地址空间：也称物理地址空间、实地址空间
 - ③ 辅存地址空间：也就是磁盘存储器的地址空间
- 对应**3种地址**：虚拟地址（虚地址）；主存物理地址（实地址）；磁盘存储器地址（磁盘地址、辅存地址）
- 虚拟存储器的**地址映射**：将**虚拟地址空间映射到主存地址空间**，即将用户利用虚拟地址访问的内容按照某种规则，从辅存装入主存中，并建立**虚地址与实地址**之间的对应关系
- 虚拟存储器的**地址变换**（转换）：程序被装入主存后，在实际运行时，将虚拟地址转换为实地址或磁盘地址，以便CPU从主存或磁盘中读取相应的信息
- **MMU**：Memory Management Unit，存储器管理单元，MMU完成虚拟地址到物理地址的转换
- 虚拟存储器分为**3种**不同的类型：**页式**虚拟存储器、**段式**虚拟存储器、**段页式**虚拟存储器

4.6.3 页式虚拟存储器

- 以页（**Page**）为逻辑结构划分信息传送单位的虚拟存储器，称为**页式虚拟存储器**
- 虚拟空间和主存空间均被划分为**固定大小的页**，常见的页大小为**4KB**，也有更大容量的页

– 1、虚拟地址划分

- 虚拟地址（**VA**）= 虚拟页号（**VPN**）+ 虚拟页偏移（**VPO**）
- 物理地址（**PA**）= 物理页号（**PPN**）+ 物理页偏移（**PPO**）

VA: Virtual Address, 虚拟地址
VPN: Virtual Page Number, 虚页号
VPO: Virtual Page Offset, 虚拟页偏移

PA: Physical Address, 物理地址
PPN: Physical Page Number, 物理页号
PPO: Physical Page Offset, 物理页偏移

- 虚拟页号也称为虚页号；物理页号也称为页框号、实页号
- 虚拟页偏移和物理页偏移的大小是相等的，也就是页式虚拟存储器**页的大小**

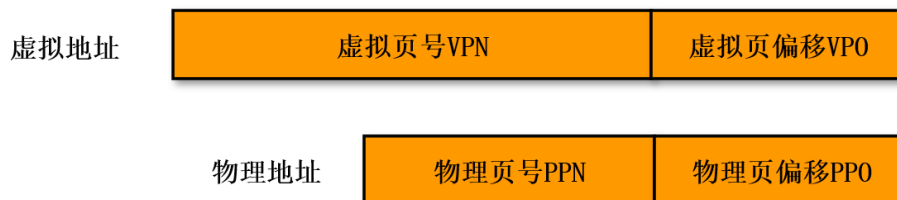


图4.46 页式虚拟存储器的地址划分

4.6.3 页式虚拟存储器

— 2、页表

- 虚拟地址到物理地址的映射，本质上就是将**虚拟页号**（虚页号）转换为**物理页号**（实页号）；页式虚拟存储器中虚拟地址到物理地址之间的转换是**基于页表进行的**

- 页表**是一张保存虚拟页号和物理页号对应关系的查找表，是一个由若干个表项组成的数组

- 页表（图4.47）采用虚拟页号作为索引进行访问，每一个表项主要包括：**有效位和物理页号（PPN）**，另外还包括：修改位、使用位、权限位等信息（图4.47中没有给出）

— **有效位**：有些页在执行程序的时候可能不在内存里，但是这些页肯定要在页表中存有记录，用有效位标记记录是否有效。

- 页表中的每一行称为**页表项**（PTE: Page Table Entry），图4.47中有8个页表项PTE0~PTE7；主存和磁盘都分为固定大小的页面，用于存放**虚拟页面**（vp: virtual page）
- 磁盘中的**交换分区**用于存放主存页面替换出来的**动态修改数据**，磁盘中的**数据分区**用于存放用户程序和数据

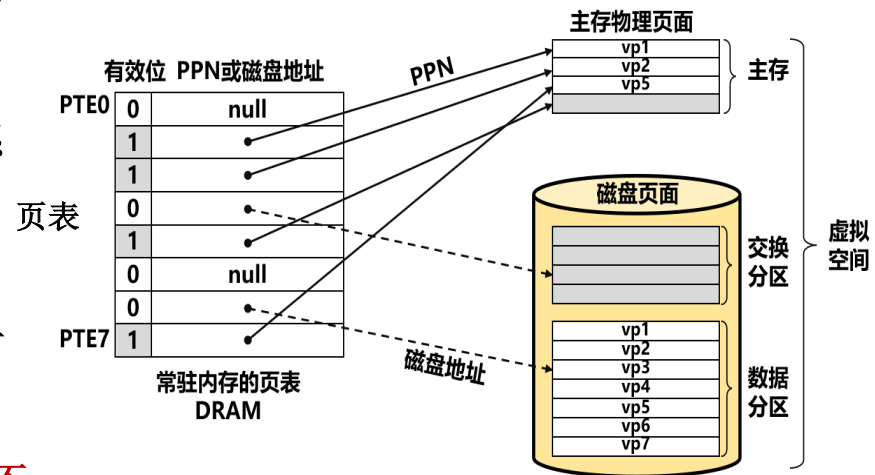


图4.47 页式逻辑结构示意图

PPN: Physical Page Number, 物理页号
PTE: Page Table Entry, 页表项
vp: virtual page, 虚拟页面

4.6.3 页式虚拟存储器

- 页表中有效位=0，且标记为null的页表项，其对应的虚拟页可能是暂未分配页（图4.47中有2个）
- 页表中有效位=0，但没有标记为null的页表项，其对应的虚拟页在磁盘中（图4.47中有2个）
- 页表中有效位=1，其对应的虚拟页在主存中（图4.47中有4个）
- 页表常驻内存（DRAM），即页表存放在主存中

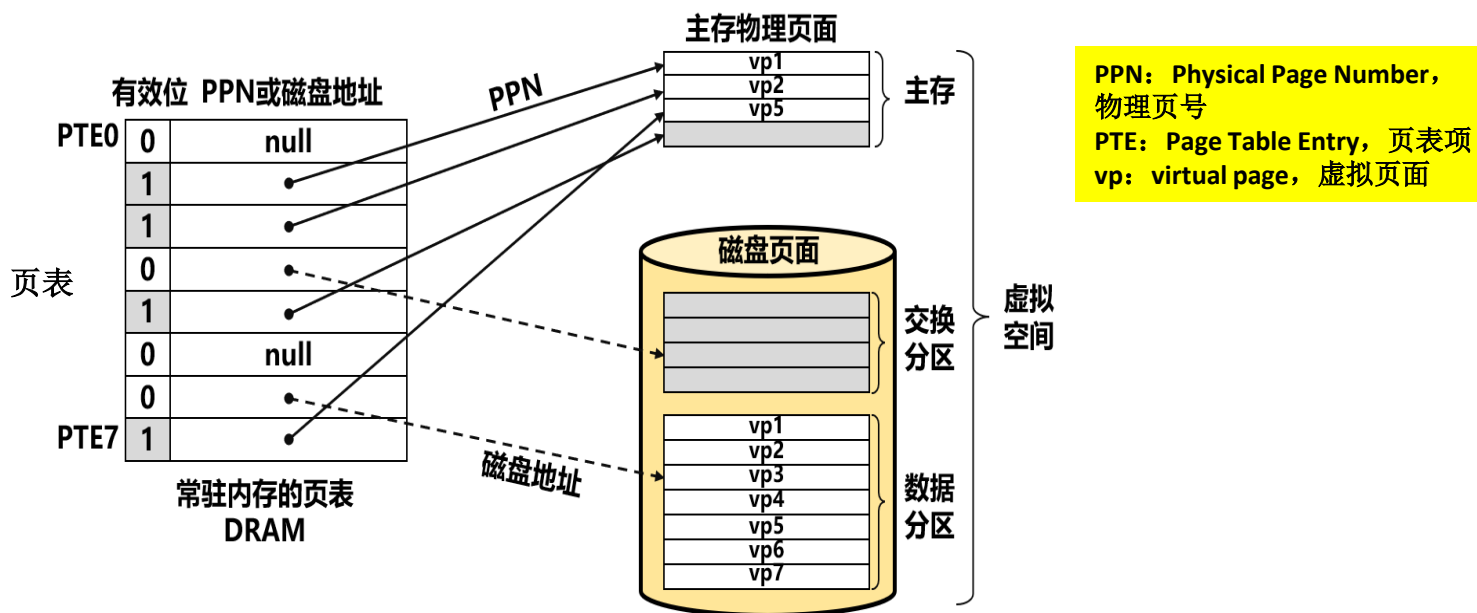


图4.47 页式逻辑结构示意图

4.6.3 页式虚拟存储器

- 假设虚拟地址空间= $2^{32}\text{B}=4\text{GB}$ （32位地址空间），页大小=4KB，则页表项的数目（即页表中有多少行）= $4\text{GB}/4\text{KB}=2^{20}=1\text{M}=1,048,576$ 行
- 假设每个页表项大小（即页表的每行大小）位32位，则该页表的大小= $1\text{M}\times 32\text{位}=4\text{MB}$
- 由于每个进程都拥有独立的虚拟地址空间，因此每个进程都要一张完整的页表
- 页表在主存中的首地址记录在页表基址寄存器（PTBR: Page Table Base Register）中
- 进行进程切换时，只需要简单地切换PTBR的值，就可以实现不同进程的页表的快速切换

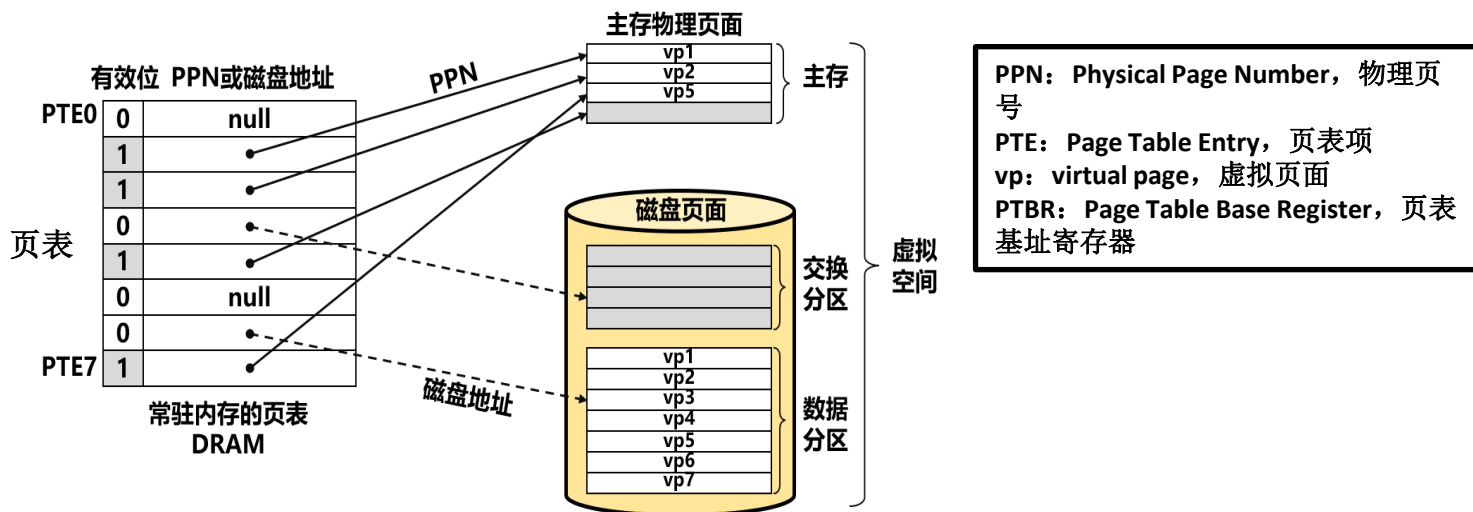


图4.47 页式逻辑结构示意图

- 基于页表的虚拟地址与物理地址的转换过程：

- 虚拟地址 $VA = \text{虚拟页号} VPN + \text{虚拟页偏移} VPO$
- 物理地址 $PA = \text{物理页号} PPN + \text{物理页偏移} PPO$

- 将虚拟地址中的虚拟页号（VPN）与页表基址寄存器的内容（PTBR）相加，结果作为页表项的地址（PTEA），即根据相加的结果找到页表的某一行；如果该页表项（页表行）的**有效位** $V=1$ ，表明当前页的数据在主存中
- 此时直接利用页表中的物理页号（PPN）和虚拟地址中的页内偏移（VPO），就可以得到物理地址（PA）
- 如果**有效位** $V=0$ ，表示缺页，则由操作系统的缺页异常处理程序，将磁盘上对应的页载入主存中

VA: Virtual Address, 虚拟地址
VPN: Virtual Page Number, 虚页号
VPO: Virtual Page Offset, 虚拟页偏移

PA: Physical Address, 物理地址
PPN: Physical Page Number, 物理页号
PPO: Physical Page Offset, 物理页偏移

PTE: Page Table Entry, 页表项
PTBR: Page Table Base Register, 页表基址寄存器
PTEA: Page Table Entry Address, 页表项地址

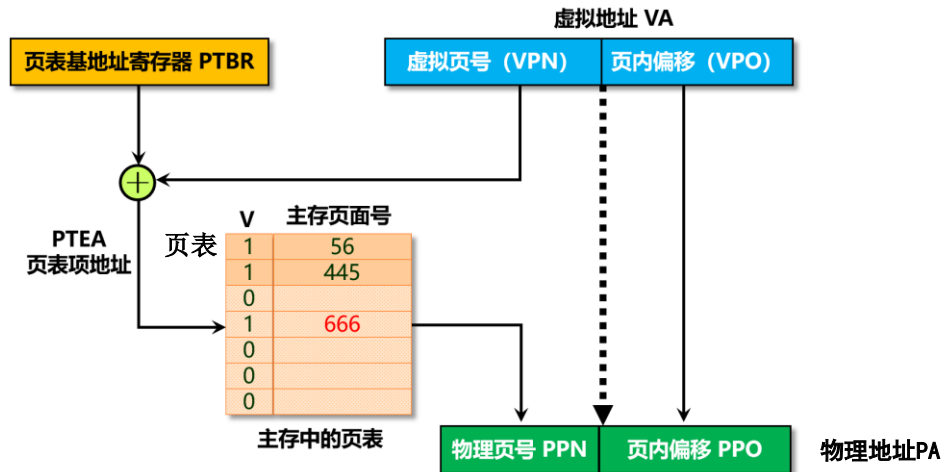


图4.48 基于页表的虚拟地址与物理地址的转换过程

- 例4.6：假设主存容量=16MB，按字节寻址；虚拟存储器容量=4GB，采用页式虚拟存储器，页面大小为4KB。请回答：
 - （1）计算物理页号、页内偏移、虚拟页号字段各为多少位？
 - （2）计算页表中页表项的数量。
 - （3）若部分页表内容如表4.10所示，求对应于虚拟地址00015240H和03FFF180H的物理地址。

虚页号	有效位	实页号	虚页号	有效位	实页号
00010H	0	002H	03FFFH	0	1
00015H	1	035H			
.....					

- 解：
 - （1）
 - 主存容量=16MB，因此物理地址=24位（ $2^{24}=16M$ ）
 - 虚拟存储器容量=4GB，因此虚拟地址=32位（ $2^{32}=4G$ ）
 - 页面大小=4KB，因此页内偏移=12位（ $2^{12}=4K$ ）
 - 物理页号=物理地址-页内偏移=24-12= 12位
 - 虚拟页号=虚拟地址-页内偏移=32-12= 20位

- 例4.6：假设主存容量=16MB，按字节寻址；虚拟存储器容量=4GB，采用页式虚拟存储器，页面大小为4KB。请回答：
 - （1）计算物理页号、页内偏移、虚拟页号字段各为多少位？
 - （2）计算页表中页表项的数量。
 - （3）若部分页表内容如表4.10所示，求对应于虚拟地址00015240H和03FFF180H的物理地址。

虚页号	有效位	实页号	虚页号	有效位	实页号
00010H	0	002H	03FFFH	0	1
00015H	1	035H			
.....					

- 解（续）：
- （2）
 - 页表项的数量（即页表中有多少行）与虚拟页号的位数有关；虚拟页号=20位，则页表项的数量= $2^{20}=1,048,576$ 项（行）
- （3）
 - 虚拟地址=00015240H=00015H（虚拟页号）+ 240H（页内偏移）
 - 虚拟地址=03FFF180H=03FFFH（虚拟页号）+ 180H（页内偏移）
 - 查表4.10，虚页号=00015H，对应的实页号=035H，且有效位=1，因此虚拟地址00015240H对应的物理地址=035H（物理页号）+240H（页内偏移）=035240H
 - 查表4.10，虚页号=03FFFH，对应的有效位=0，表示该虚拟页不在主存中

20bit

12bit

– 3、虚拟存储器访问流程

- 页面命中时的访问流程：

① CPU（处理器）给出一个**虚拟地址VA**，并送到MMU

② MMU根据虚拟地址VA中的虚拟页号VPN，以及页表基址寄存器PTBR的内容，计算得到页表项地址PTEA，并根据**页表项地址PTEA**访问页表

虚拟地址VA = 虚拟页号VPN + 虚拟页偏移VPO
物理地址PA = 物理页号PPN + 物理页偏移PPO

③ 页表向MMU返回**页表项PTE**

④ 若页表项PTE中的有效位=1，则根据页表项PTE中的物理页号PPN和虚拟地址中的页内偏移VPO，构成**物理地址PA**

⑤ 由物理地址PA，从cache/主存中访问得到**数据**，并返回给CPU

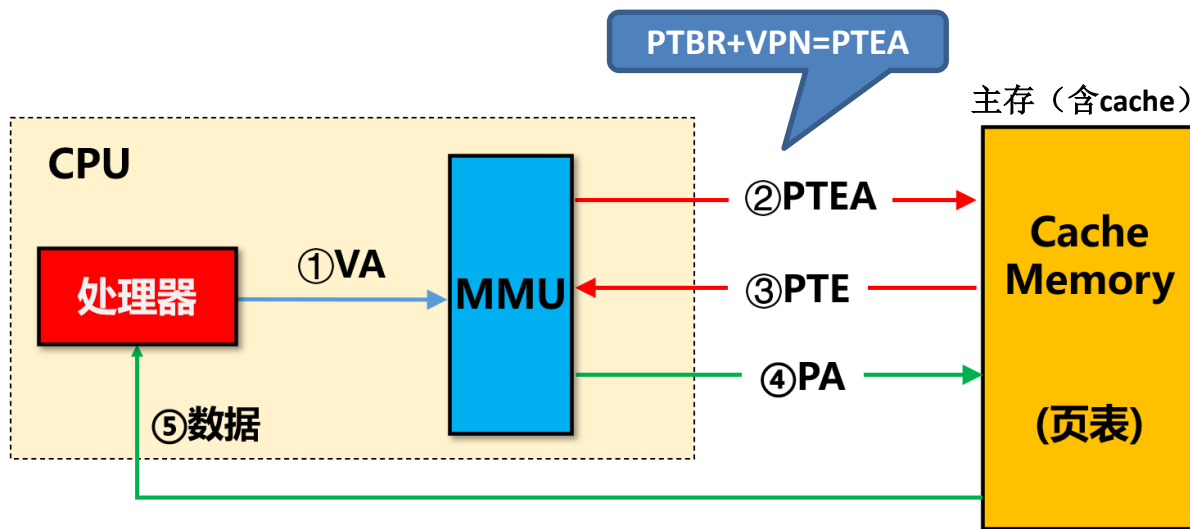


图4.49 (a) 页式虚拟存储器的访问过程（页面命中）

- 页面缺失时的访问流程：

- ① CPU（处理器）给出一个**虚拟地址VA**，并送到MMU
- ② MMU根据虚拟地址VA中的虚拟页号VPN，以及页表基址寄存器PTBR的内容，计算得到页表项地址PTEA，并根据**页表项地址PTEA**访问页表
- ③ 页表向MMU返回**页表项PTE**
- ④ 若页表项PTE中的有效位=0，表示所访问的页不在主存中，MMU触发**缺页异常**，调用操作系统内核中的缺页异常处理程序
- ⑤ 如果主存页满，则需要根据替换算法确定换出页（哪一页被换出）；如果换出页的修改位=1，则将该页换出到磁盘（**调出页**），否则直接丢弃
- ⑥ 缺页处理程序直接从磁盘中调入新的页（**调入页**），并更新存储器中的页表项PTE
- ⑦ 缺页处理程序**返回**到原来的进程

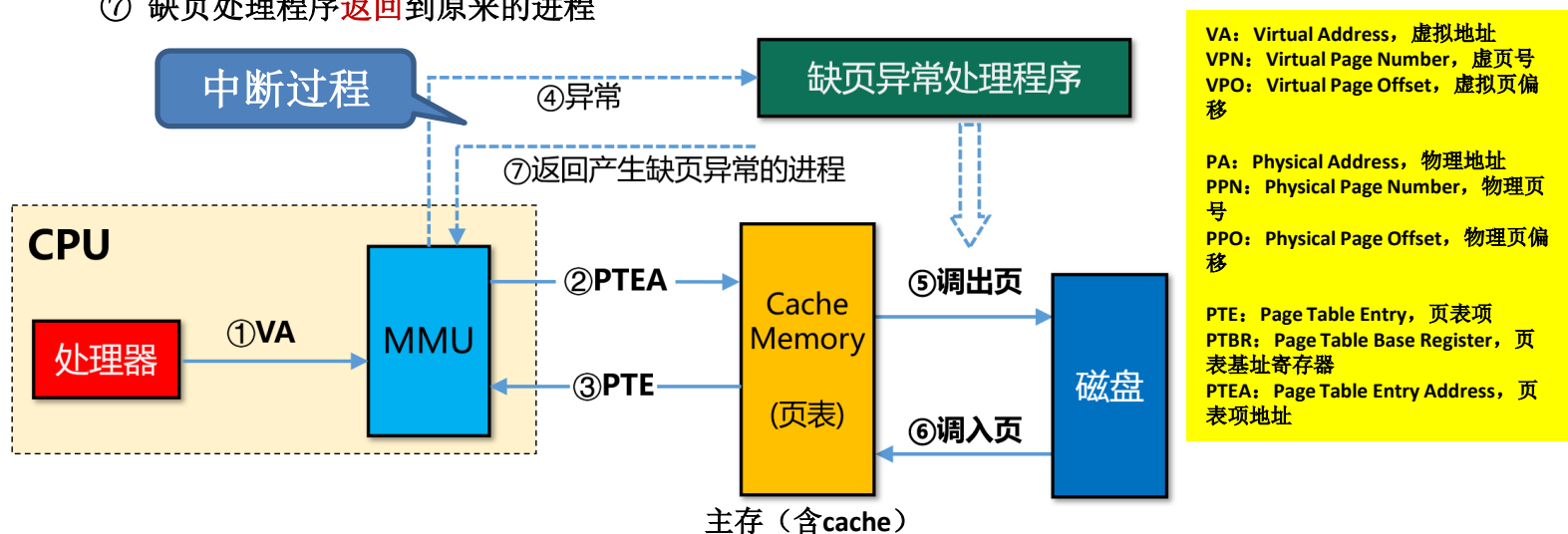


图4.49 (b) 页式虚拟存储器的访问过程（页面缺失）

– 4、结合cache的虚拟存储器访问流程

- 图4.49中的主存和cache是合在一起的；结合cache的虚拟存储器的cache和主存是分开的（图4.50），此时部分页表块放在cache中
- 结合cache的虚拟存储器访问流程：
 - ① CPU（处理器）给出一个**虚拟地址VA**，并送到MMU
 - ② MMU根据虚拟地址VA中的虚拟页号VPN，以及页表基址寄存器PTBR的内容，计算得到页表项地址PTEA，并根据**页表项地址PTEA**访问位于cache中的页表
 - ③ 如果位于cache中的页表有该页表项地址PTEA，则直接向MMU返回**页表项PTE**（表示PTEA命中）；如果cache中的页表没有该页表项地址PTEA（表示PTEA缺失），则需要从位于主存的页表中得到页表项PTE
 - ④ 若页表项PTE中的有效位=1，则根据页表项PTE中的物理页号PPN和虚拟地址中的页内偏移VPO，构成**物理地址PA**
 - ⑤ 由物理地址PA，如果直接从cache中访问得到**数据**（表示PA命中），并返回给CPU；若cache中访问不到数据（表示PA缺失），则需要从主存中访问数据

此图中的cache里面既有data也有页表块；一旦出现miss，则均需要从mem中提取，并更新对应的cache中内容

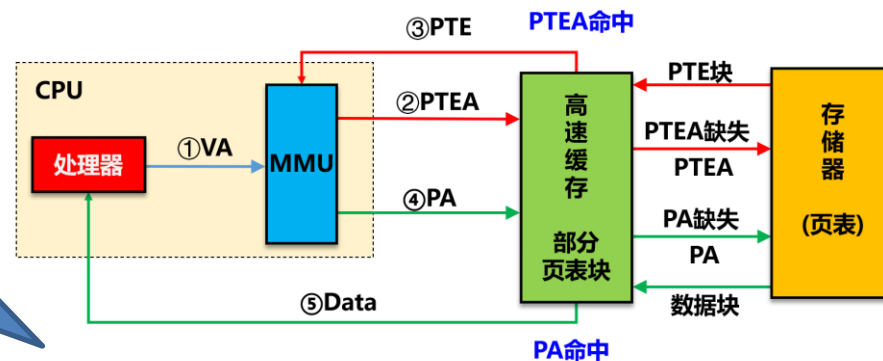


图4.50 结合cache的页式虚拟存储器的访问过程

– 5、利用TLB加速虚拟存储器地址转换

- 为了进一步提高虚拟存储器地址转换的速度，现代计算机中都设置一个**转换旁路缓冲区**（Translation Look-aside Buffer, **TLB**），用于**缓冲经常访问的页表项**（PTE，即页表中经常会被访问的行）
- **TLB**本质上就是一个容量较小的**cache**，为提高查找速度，大多采用全相联或组相联方式，且采用随机替换算法
- **TLB**也称为**快表**，存放在主存中的页表称为**慢表**；快表是按**内容**访问的，慢表是按**地址**访问的；基于**TLB**的虚拟地址与物理地址的转换逻辑如表4.52所示
- 在进行地址转换时，往往同时查快表和慢表，如果查快表命中，则从快表中得到物理页号，同时终止查慢表的过程；如果查快表不命中，则从慢表中得到物理页号

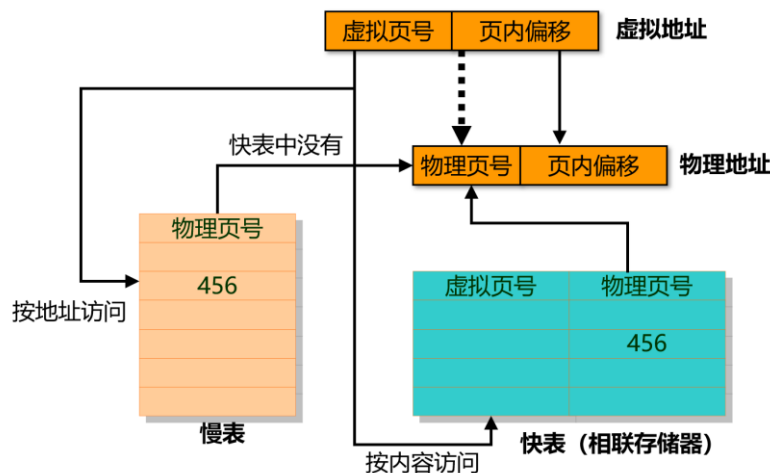


图4.52 基于TLB的虚拟地址与物理地址的转换逻辑

- **TLB命中时的访问流程：**

- ① CPU（处理器）给出一个**虚拟地址VA**，并送到**MMU**
- ② **MMU**根据虚拟地址VA中的**虚拟页号VPN**，查询**TLB**
- ③ 如果**TLB**访问命中，则**TLB**向**MMU**返回与虚拟页号VPN对应的**物理页号PPN**
- ④ **MMU**根据物理页号PPN和虚拟地址中的页内偏移VPO，构成**物理地址PA**
- ⑤ 由物理地址PA，从**cache/主存**中访问得到**数据**，并返回给**CPU**

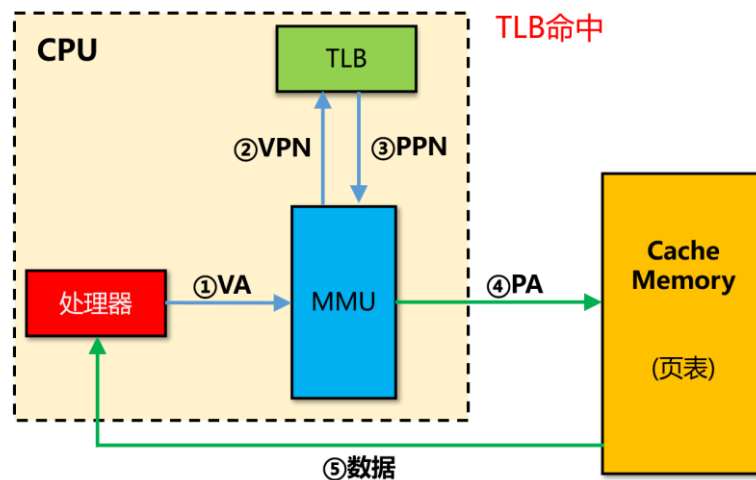


图4.53 TLB命中访问流程

- TLB不命中（缺失）时的访问流程：

- ① CPU（处理器）给出一个**虚拟地址VA**，并送到MMU
- ② MMU根据虚拟地址VA中的**虚拟页号VPN**，查询TLB
- ③ 如果TLB访问不命中（缺失），则TLB向MMU返回**TLB访问缺失**信息
- ④ MMU根据虚拟地址VA中的虚拟页号VPN，以及页表基址寄存器PTBR的内容，计算得到页表项地址PTEA，并根据**页表项地址PTEA**访问页表
- ⑤ 页表向MMU返回**页表项PTE**
- ⑥ 若页表项PTE中的有效位=1，则需要**更新TLB**；同时根据页表项PTE中的物理页号PPN和虚拟地址中的页内偏移VPO，构成**物理地址PA**
- ⑦ 由物理地址PA，从cache/主存中访问得到**数据**，并返回给CPU

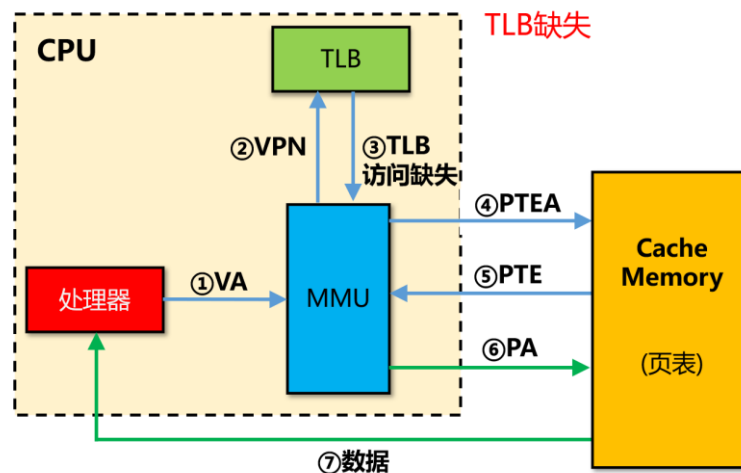


图4.54 TLB不命中（缺失）访问流程

– 6、CPU访问过程

- 实地址模式：**CPU**只能用物理地址访问主存，也称实模式
- 虚地址模式；**CPU**只能使用虚拟地址访问主存，也称保护模式
- 在一个具有**cache**、虚拟存储器的系统中，**CPU**的一次完整的虚存访问操作流程如下所示：

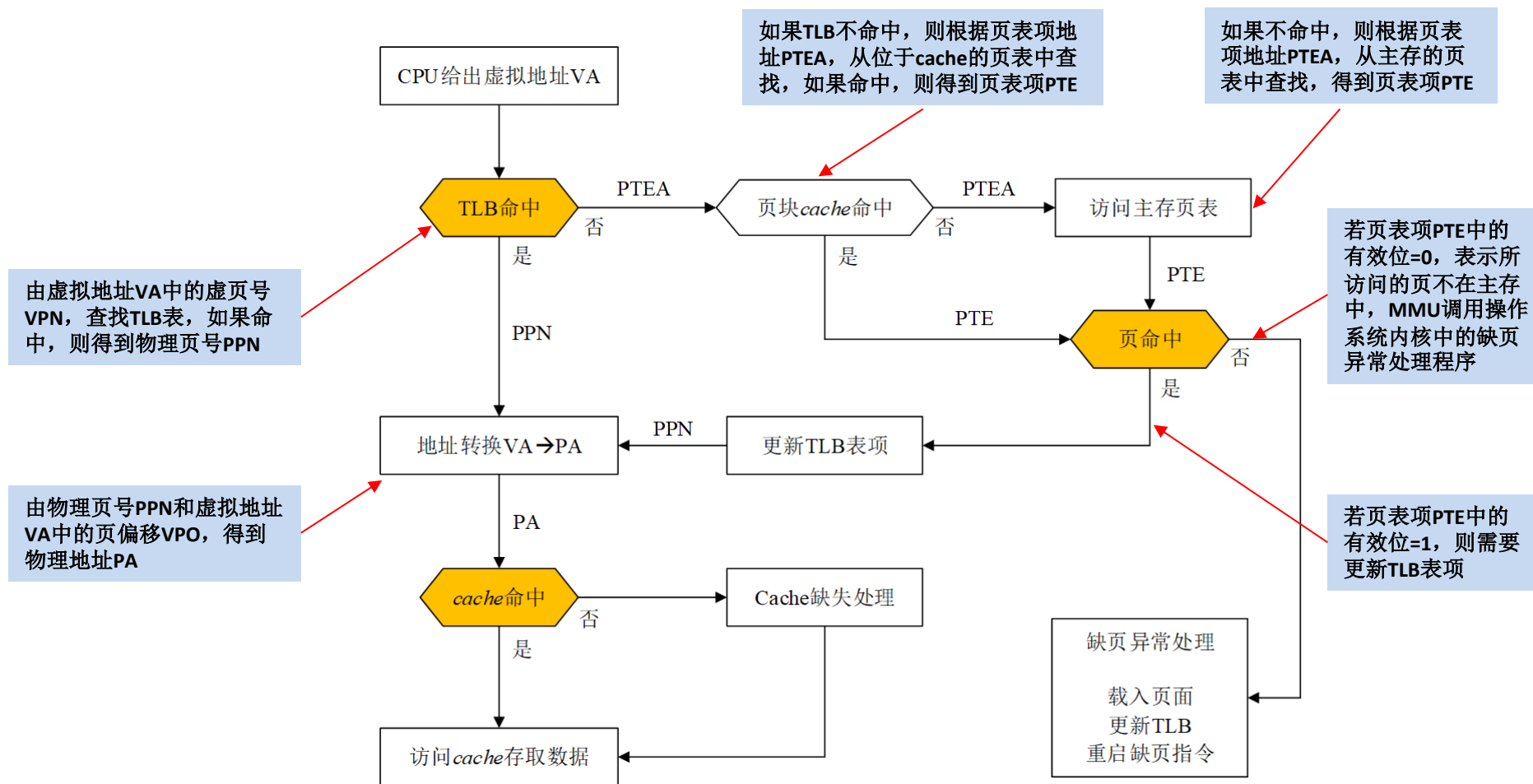


图4.55 虚存访问操作流程

- 最左边的路径（红色）为最短路径，访问性能最优

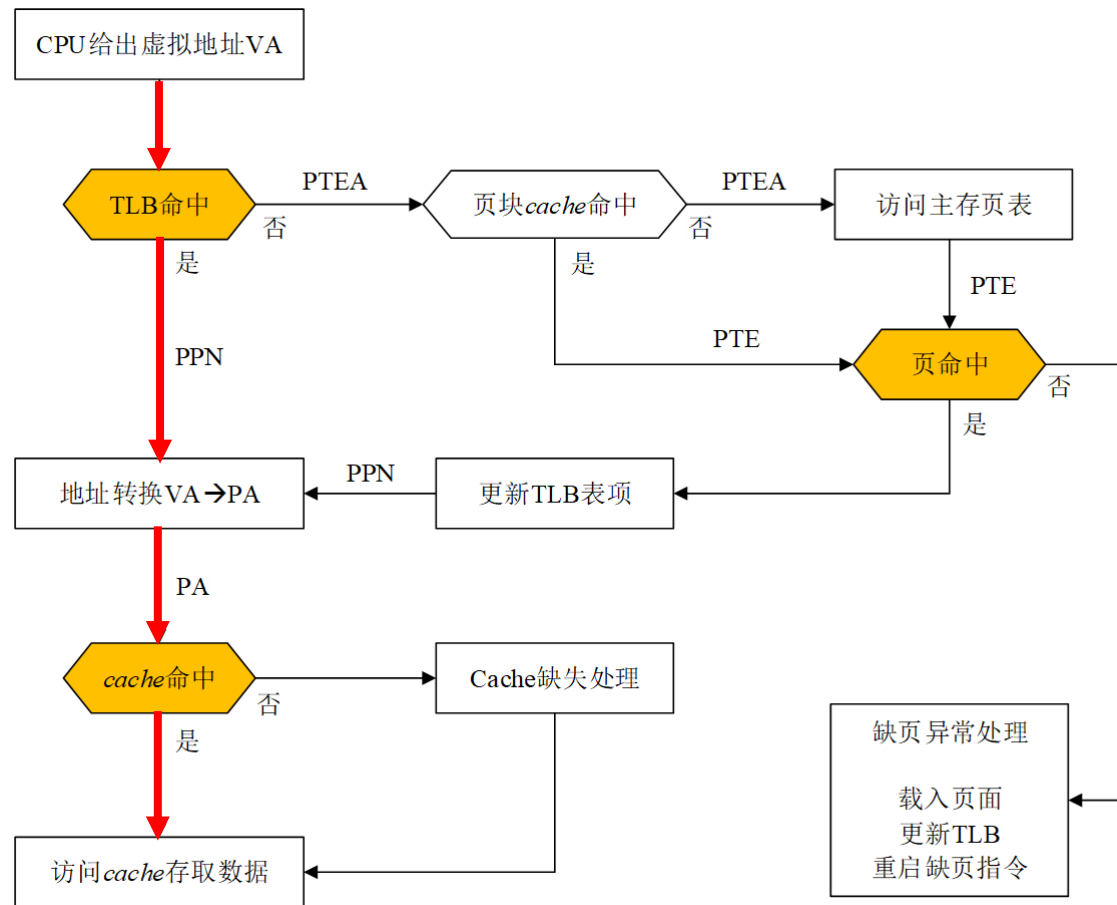


图4.55 虚存访问操作流程

表4.11 TLB命中、页命中、cache命中组合

序号	TLB	页	cache	可能性	说明
1	命中	命中	命中	可能	TLB命中则页一定命中，页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
2	命中	命中	缺失		
3	缺失	命中	命中	可能	TLB缺失后还可以访问慢速页表（慢表），页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
4	缺失	命中	缺失		
5	缺失	缺失	缺失	可能	这是最糟糕的情况，虚存系统初始化时常见
6	命中	缺失	缺失	不可能	页缺失说明页不在主存中，TLB中一定没有对应页表项，TLB不可能命中
7	命中	缺失	命中		
8	缺失	缺失	命中	不可能	页缺失说明数据也不在主存中，所以cache不可能命中

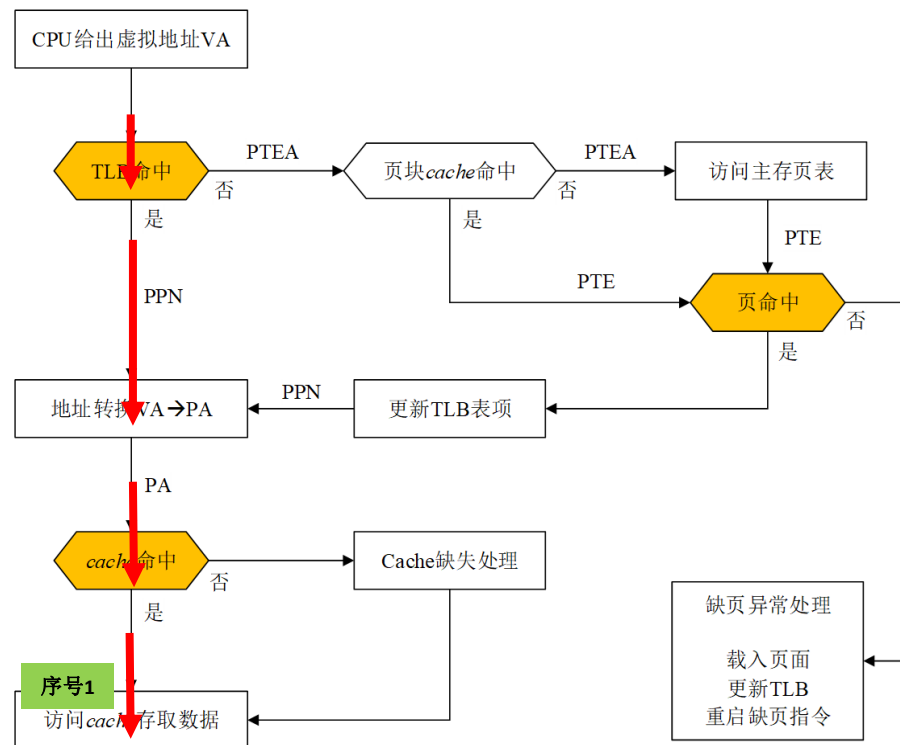


图4.55 虚存访问操作流程

表4.11 TLB命中、页命中、cache命中组合

序号	TLB	页	cache	可能性	说明
1	命中	命中	命中	可能	TLB命中则页一定命中，页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
2	命中	命中	缺失		
3	缺失	命中	命中	可能	TLB缺失后还可以访问慢速页表（慢表），页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
4	缺失	命中	缺失		
5	缺失	缺失	缺失	可能	这是最糟糕的情况，虚存系统初始化时常见
6	命中	缺失	缺失	不可能	页缺失说明页不在主存中，TLB中一定没有对应页表项，TLB不可能命中
7	命中	缺失	命中		
8	缺失	缺失	命中	不可能	页缺失说明数据也不在主存中，所以cache不可能命中

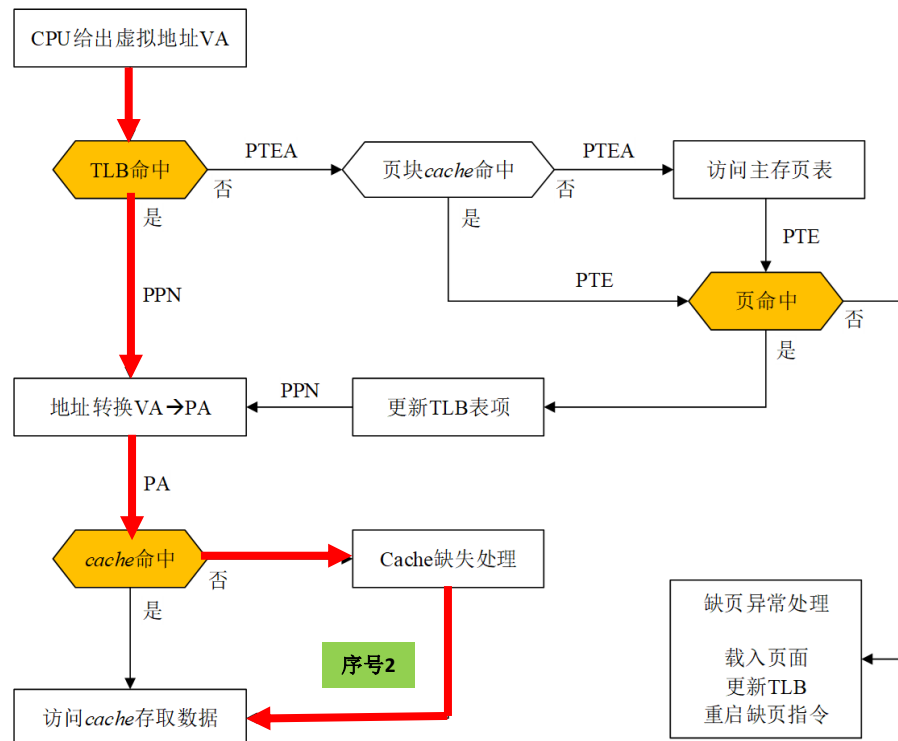


图4.55 虚存访问操作流程

表4.11 TLB命中、页命中、cache命中组合

序号	TLB	页	cache	可能性	说明
1	命中	命中	命中	可能	TLB命中则页一定命中，页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
2	命中	命中	缺失		
3	缺失	命中	命中	可能	TLB缺失后还可以访问慢速页表（慢表），页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
4	缺失	命中	缺失		
5	缺失	缺失	缺失	可能	这是最糟糕的情况，虚存系统初始化时常见
6	命中	缺失	缺失	不可能	页缺失说明页不在主存中，TLB中一定没有对应页表项，TLB不可能命中
7	命中	缺失	命中		
8	缺失	缺失	命中	不可能	页缺失说明数据也不在主存中，所以cache不可能命中

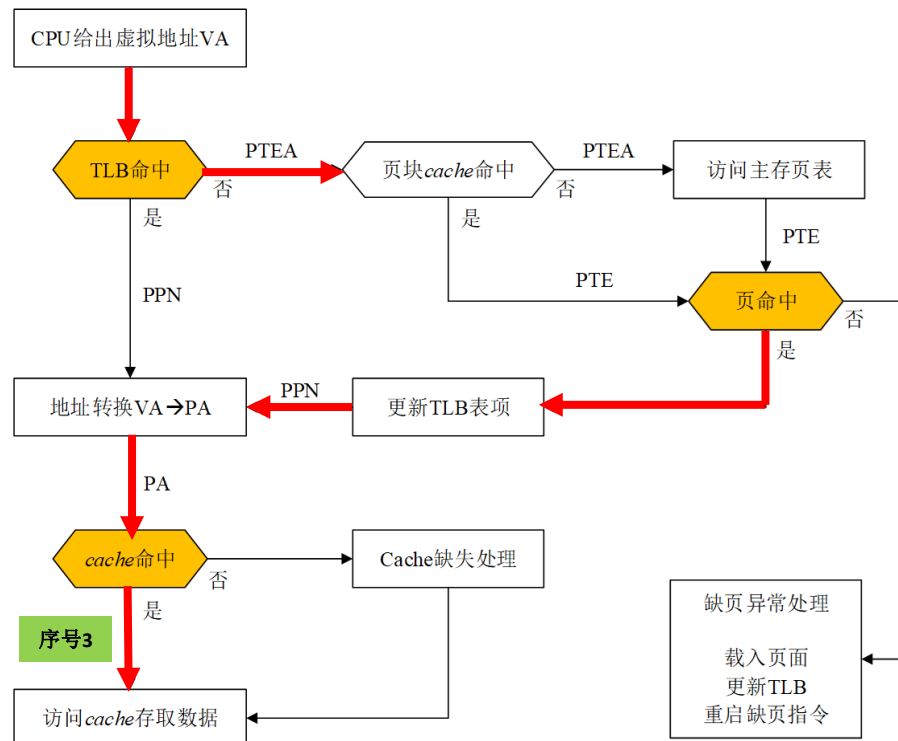


图4.55 虚存访问操作流程

表4.11 TLB命中、页命中、cache命中组合

序号	TLB	页	cache	可能性	说明
1	命中	命中	命中	可能	TLB命中则页一定命中，页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
2	命中	命中	缺失		
3	缺失	命中	命中	可能	TLB缺失后还可以访问慢速页表（慢表），页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
4	缺失	命中	缺失		
5	缺失	缺失	缺失	可能	这是最糟糕的情况，虚存系统初始化时常见
6	命中	缺失	缺失	不可能	页缺失说明页不在主存中，TLB中一定没有对应页表项，TLB不可能命中
7	命中	缺失	命中		
8	缺失	缺失	命中	不可能	页缺失说明数据也不在主存中，所以cache不可能命中

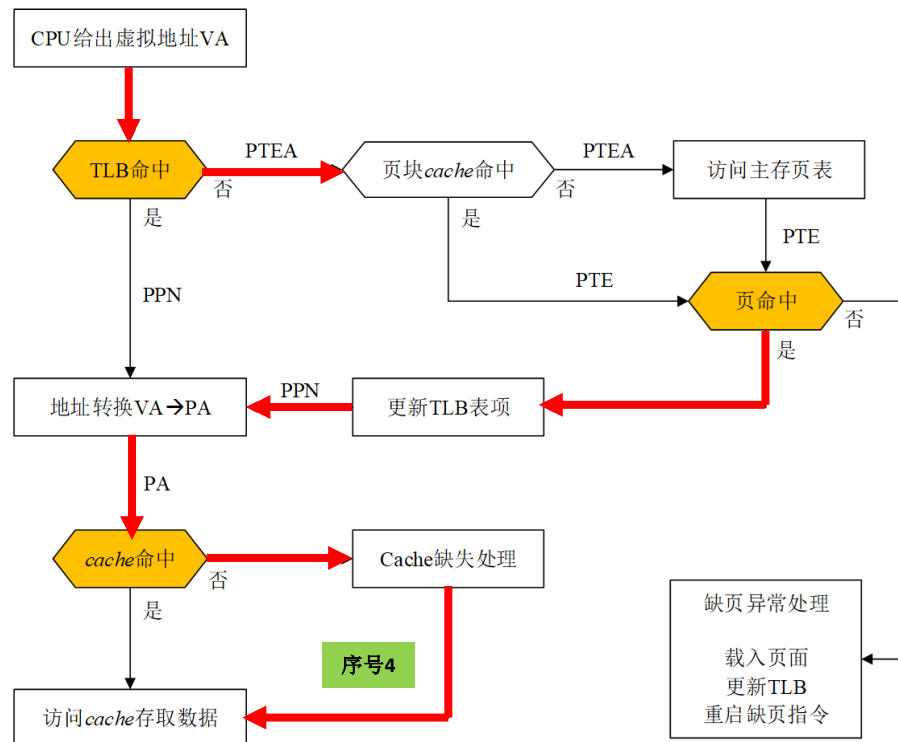


图4.55 虚存访问操作流程

表4.11 TLB命中、页命中、cache命中组合

序号	TLB	页	cache	可能性	说明
1	命中	命中	命中	可能	TLB命中则页一定命中，页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
2	命中	命中	缺失		
3	缺失	命中	命中	可能	TLB缺失后还可以访问慢速页表（慢表），页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
4	缺失	命中	缺失		
5	缺失	缺失	缺失	可能	这是最糟糕的情况，虚存系统初始化时常见
6	命中	缺失	缺失	不可能	页缺失说明页不在主存中，TLB中一定没有对应页表项，TLB不可能命中
7	命中	缺失	命中		
8	缺失	缺失	命中	不可能	页缺失说明数据也不在主存中，所以cache不可能命中

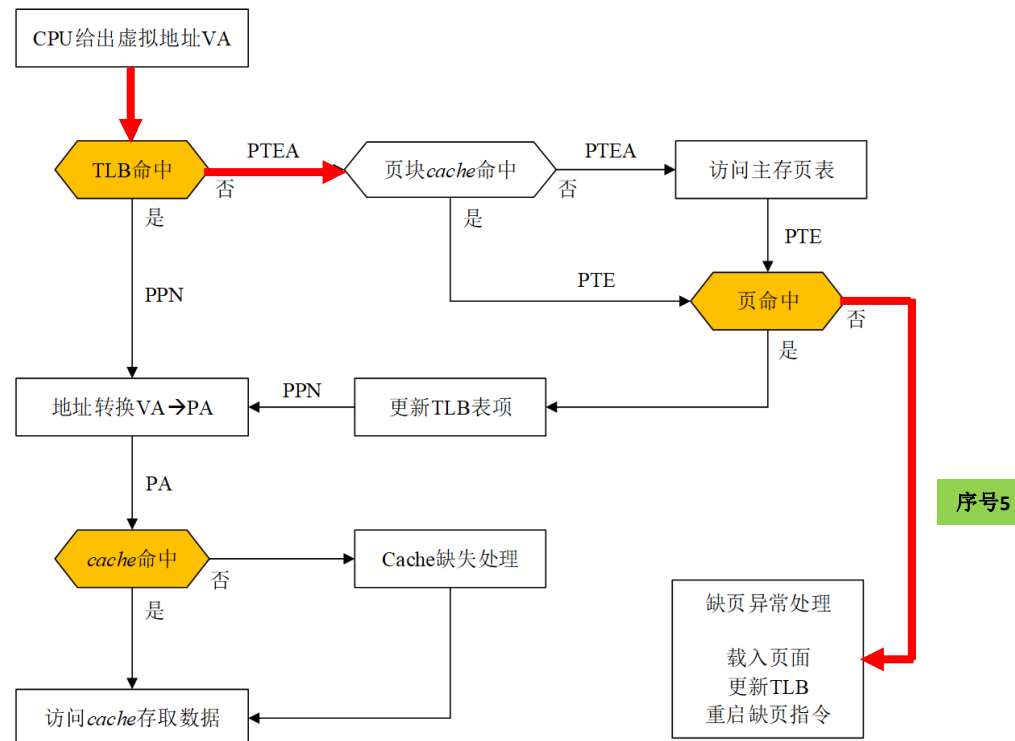


图4.55 虚存访问操作流程

表4.11 TLB命中、页命中、cache命中组合

序号	TLB	页	cache	可能性	说明
1	命中	命中	命中	可能	TLB命中则页一定命中，页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
2	命中	命中	缺失		
3	缺失	命中	命中	可能	TLB缺失后还可以访问慢速页表（慢表），页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
4	缺失	命中	缺失		
5	缺失	缺失	缺失	可能	这是最糟糕的情况，虚存系统初始化时常见
6	命中	缺失	缺失	不可能	页缺失说明页不在主存中，TLB中一定没有对应页表项，TLB不可能命中
7	命中	缺失	命中		
8	缺失	缺失	命中	不可能	页缺失说明数据也不在主存中，所以cache不可能命中

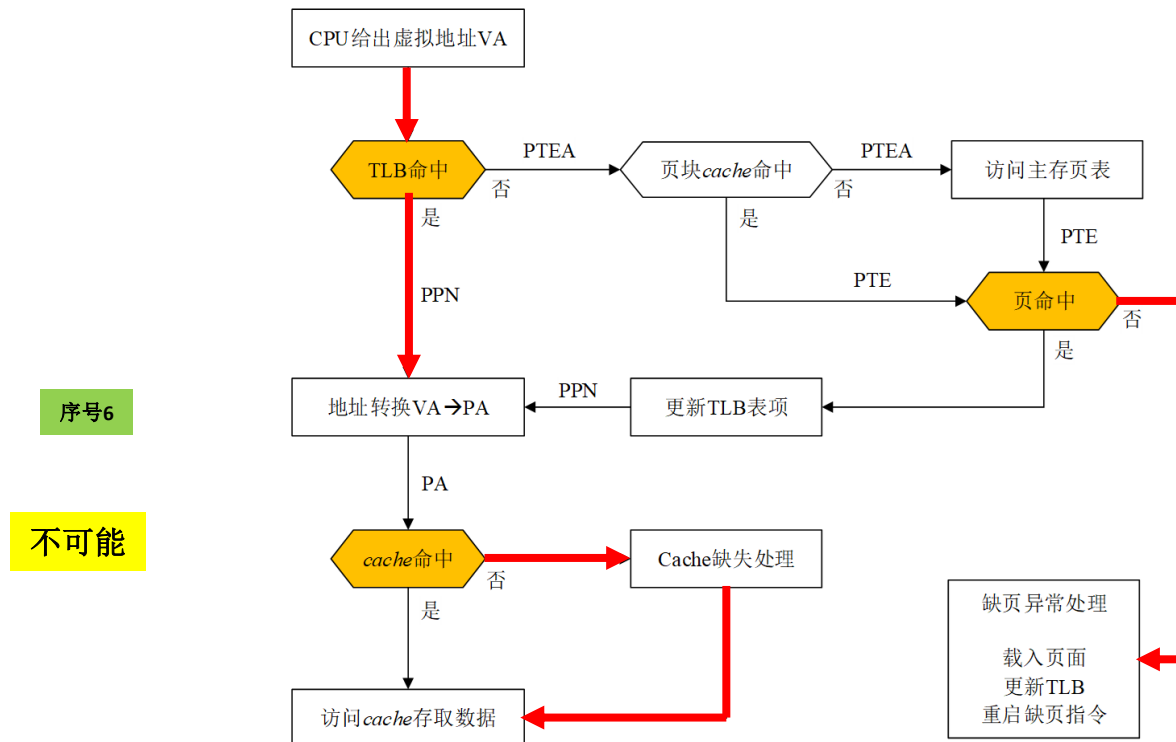


图4.55 虚存访问操作流程

表4.11 TLB命中、页命中、cache命中组合

序号	TLB	页	cache	可能性	说明
1	命中	命中	命中	可能	TLB命中则页一定命中，页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
2	命中	命中	缺失		
3	缺失	命中	命中	可能	TLB缺失后还可以访问慢速页表（慢表），页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
4	缺失	命中	缺失		
5	缺失	缺失	缺失	可能	这是最糟糕的情况，虚存系统初始化时常见
6	命中	缺失	缺失	不可能	页缺失说明页不在主存中，TLB中一定没有对应页表项，TLB不可能命中
7	命中	缺失	命中		
8	缺失	缺失	命中	不可能	页缺失说明数据也不在主存中，所以cache不可能命中

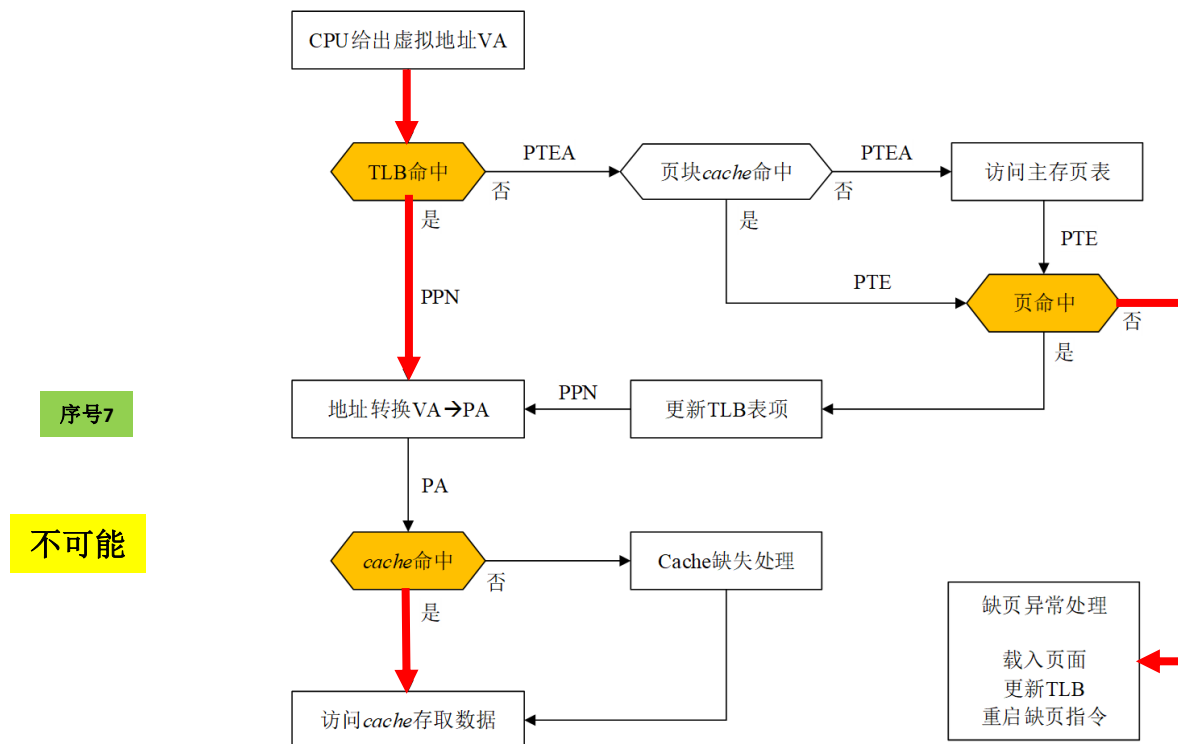


图4.55 虚存访问操作流程

表4.11 TLB命中、页命中、cache命中组合

序号	TLB	页	cache	可能性	说明
1	命中	命中	命中	可能	TLB命中则页一定命中，页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
2	命中	命中	缺失		
3	缺失	命中	命中	可能	TLB缺失后还可以访问慢速页表（慢表），页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
4	缺失	命中	缺失		
5	缺失	缺失	缺失	可能	这是最糟糕的情况，虚存系统初始化时常见
6	命中	缺失	缺失	不可能	页缺失说明页不在主存中，TLB中一定没有对应页表项，TLB不可能命中
7	命中	缺失	命中		
8	缺失	缺失	命中	不可能	页缺失说明数据也不在主存中，所以cache不可能命中

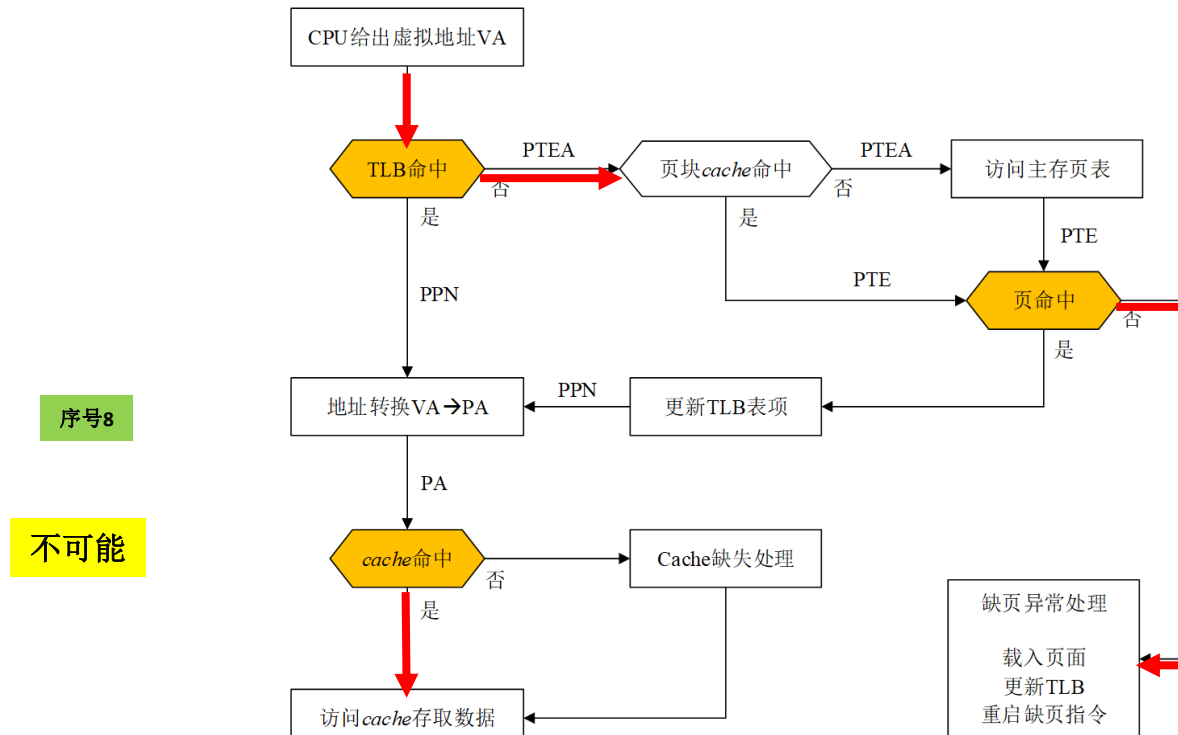


图4.55 虚存访问操作流程

- 例4.7：某计算机存储器按字节编址，虚拟（逻辑）地址空间大小为16MB，主存（物理）地址空间大小为1MB，页面大小为4KB，cache采用直接相联映射方式，共8行；主存与cache之间交换块大小为32B。系统运行到某一时刻时，页表的部分内容和cache的部分内容分别如下图4.56（a）、图4.56（b），图中页框号及标记字段的内容均为十六进制形式。请回答以下问题：
- （1）虚拟地址共有几位？哪几位表示虚页号？物理地址共有几位？哪几位表示页框号（物理页号）？
- （2）使用物理地址访问cache时，物理地址应划分为哪几个字段？请说明每个字段的位数及在物理地址中的位置。
- （3）虚拟地址001C60H所在的页面是否在主存中？若在主存中，则该虚拟地址对应的物理地址是什么？访问该地址时cache是否命中？请说明理由。
- （4）假设为该计算机配置一个四路组相联的TLB（可存放8个页表项），若其当前内容如图4.56（c）所示（十六进制），则此时虚拟地址024BACH所在的页面是否存在于主存中？请说明理由。

虚页号	有效位	页框号	
0	1	06	
1	1	04	
2	1	15	
3	1	02	
4	0		
5	1	2B	
6	0		
7	1	32	

图4.56（a） 页表部分内容

行号	有效位	标记	
0	1	020	
1	0		
2	1	01D	
3	1	105	
4	1	064	
5	1	14D	
6	0		
7	1	27A	

图4.56（b） cache部分内容

组号	有效位	标记	页框号	有效位	标记	页框号	有效位	标记	页框号	有效位	标记	页框号
0	0	—	—	1	001	15	0	—	—	1	012	1F
1	1	013	2D	0	—	—	1	008	7E	0	—	—

图4.56（c） TLB的部分内容

- 例4.7：某计算机存储器按字节编址，虚拟（逻辑）地址空间大小为16MB，主存（物理）地址空间大小为1MB，页面大小为4KB，cache采用直接相联映射方式，共8行；主存与cache之间交换块大小为32B。系统运行到某一时刻时，页表的部分内容和cache的部分内容分别如下图4.56（a）、图4.56（b），图中页框号及标记字段的内容均为十六进制形式。请回答以下问题：
 - （1）虚拟地址共有几位？哪几位表示虚页号？物理地址共有几位？哪几位表示页框号（物理页号）？
 - （2）使用物理地址访问cache时，物理地址应划分为哪几个字段？请说明每个字段的位数及在物理地址中的位置。
 - （3）虚拟地址001C60H所在的页面是否在主存中？若在主存中，则该虚拟地址对应的物理地址是什么？访问该地址时cache是否命中？请说明理由。
 - （4）假设为该计算机配置一个四路组相联的TLB（可存放8个页表项），若其当前内容如图4.56（c）所示（十六进制），则此时虚拟地址024BACH所在的页面是否存在于主存中？请说明理由。

虚页号	有效位	页框号	
0	1	06	
1	1	04	
2	1	15	
3	1	02	
4	0		
5	1	2B	
6	0		
7	1	32	

图4.56（a） 页表部分内容

行号	有效位	标记	
0	1	020	
1	0		
2	1	01D	
3	1	105	
4	1	064	
5	1	14D	
6	0		
7	1	27A	

图4.56（b） cache部分内容

组号	有效位	标记	页框号	有效位	标记	页框号	有效位	标记	页框号	有效位	标记	页框号
0	0	—	—	1	001	15	0	—	—	1	012	1F
1	1	013	2D	0	—	—	1	008	7E	0	—	—

图4.56（c） TLB的部分内容

- 例4.7：某计算机存储器按字节编址，虚拟（逻辑）地址空间大小为16MB，主存（物理）地址空间大小为1MB，页面大小为4KB，cache采用直接相联映射方式，共8行；主存与cache之间交换块大小为32B。系统运行到某一时刻时，页表的部分内容和cache的部分内容分别如下图4.56（a）、图4.56（b），图中页框号及标记字段的内容均为十六进制形式。请回答以下问题：

- （1）虚拟地址共有几位？哪几位表示虚页号？物理地址共有几位？哪几位表示页框号（物理页号）？

• 解：（1）

- 虚拟地址 = 虚页号 + 页偏移
- 虚拟（逻辑）地址空间大小为16MB，因此，虚拟地址=24位（ $2^{24}=16\text{M}$ ）
- 页面大小为4KB，因此，页偏移=12位（ $2^{12}=4\text{K}$ ）
- 因此，虚页号=24-12=12位
- 物理地址 = 实页号（页框号）+ 页偏移
- 主存（物理）地址空间大小为1MB，因此，物理地址=20位（ $2^{20}=1\text{M}$ ）
- 因此，页框号（物理页号）=20-12=8位

- 例4.7：某计算机存储器按字节编址，虚拟（逻辑）地址空间大小为16MB，主存（物理）地址空间大小为1MB，页面大小为4KB，cache采用直接相联映射方式，共8行；主存与cache之间交换块大小为32B。系统运行到某一时刻时，页表的部分内容和cache的部分内容分别如下图4.56（a）、图4.56（b），图中页框号及标记字段的内容均为十六进制形式。请回答以下问题：
 - （2）使用物理地址访问cache时，物理地址应划分为哪几个字段？请说明每个字段的位数及在物理地址中的位置。
- 解：（2）
 - 因为cache采用直接相联映射方式，因此，物理地址（主存地址）= 标记tag + 行索引字段index + 块内偏移offset
 - 主存与cache之间交换块大小为32B，因此块内偏移=5位（ $2^5=32$ ）
 - cache共有8行，因此行索引字段=3位（ $2^3=8$ ）
 - 主存（物理）地址空间大小为1MB，因此，物理地址=20位（ $2^{20}=1M$ ）
 - 因此，标记tag=20-3-5=12位
 - 物理地址（主存地址）= 标记tag + 行索引字段index + 块内偏移offset = 12位 + 3位 + 5位

- 例4.7：某计算机存储器按字节编址，虚拟（逻辑）地址空间大小为16MB，主存（物理）地址空间大小为1MB，页面大小为4KB，cache采用直接相联映射方式，共8行；主存与cache之间交换块大小为32B。系统运行到某一时刻时，页表的部分内容和cache的部分内容分别如下图4.56（a）、图4.56（b），图中页框号及标记字段的内容均为十六进制形式。请回答以下问题：

- （3）虚拟地址001C60H所在的页面是否在主存中？若在主存中，则该虚拟地址对应的物理地址是什么？访问该地址时cache是否命中？请说明理由。

虚页号	有效位	页框号	
0	1	06	
1	1	04	
2	1	15	
3	1	02	
4	0		
5	1	2B	
6	0		
7	1	32	

图4.56（a） 页表部分内容

行号	有效位	标记	
0	1	020	
1	0		
2	1	01D	
3	1	105	
4	1	064	
5	1	14D	
6	0		
7	1	27A	

图4.56（b） cache部分内容

解：（3）

- 虚拟地址=虚页号+页偏移 = 12位 + 12位；虚拟地址001C60H = 001H + C60H；虚页号=001=1，查页表（图4.56（a）），有效位=1，表示命中，即虚拟地址001C60H所在的页面在主存中
- 由图4.56（a），该虚拟地址其对应的页框号=04；因此，物理地址=实页号（页框号）+页偏移 = 04 + C60H = 04C60H
- 又根据：物理地址（主存地址）=标记tag + 行索引字段index + 块内偏移offset=12位+3位+5位
- 物理地址（主存地址）=04C60H = 0000 0100 1100 0110 0000 = 04C + 3 + 0；标记=04C，行索引字段=011=3
- 根据行索引字段=3，查图4.56（b），有效位=1，但是标记=105，与主存地址的标记04C，不相等，表示cache不命中

- 例4.7：某计算机存储器按字节编址，虚拟（逻辑）地址空间大小为16MB，主存（物理）地址空间大小为1MB，页面大小为4KB，cache采用直接相联映射方式，共8行；主存与cache之间交换块大小为32B。系统运行到某一时刻时，页表的部分内容和cache的部分内容分别如下图4.56（a）、图4.56（b），图中页框号及标记字段的内容均为十六进制形式。请回答以下问题：

- （4）假设为该计算机配置一个四路组相联的TLB（可存放8个页表项），若其当前内容如图4.56（c）所示（十六进制），则此时虚拟地址024BACH所在的页面是否存在于主存中？请说明理由。

解：（4）

- 组相联映射的TLB，虚拟地址（24位）=虚页号 + 页偏移 = TLB标记 + TLB索引 + 页偏移
- 因为TLB只有8个页表项，四路组相联映射的TLB有2组、每组4行，即TLB索引=1位（ $2^1=2$ ）
- 页偏移=12位，因此TLB标记=24-1-12=11位
- 虚拟地址=024BACH= 0000 0010 010 0 1011 1010 1100 = TLB标记 + TLB索引 + 页偏移=12H + 0 + BACH
- 即TLB索引=0，表示是TLB的第0组；TLB标记=0000 0010 010=012H，查图4.56（c），有效位=1，表示页面在主存中；页框号=1FH，则主存地址（物理地址）=实页号（页框号）+ 页偏移 = 1FH + BACH = 1FBACH



组号	有效位	标记	页框号	有效位	标记	页框号	有效位	标记	页框号	有效位	标记	页框号
0	0	—	—	1	001	15	0	—	—	1	012	1F
1	1	013	2D	0	—	—	1	008	7E	0	—	—

图4.56（c） TLB的部分内容

实践训练（实验3）

- 在Logisim中利用ROM组件构建能显示16X16点阵的汉字字库存储系统
- 在Logisim中利用RAM组件构建能同时支持8位、16位、32位数据访问的存储器
- 在Logisim中利用给定的cache框架分别设计直接相联、全相联、组相联映射cache